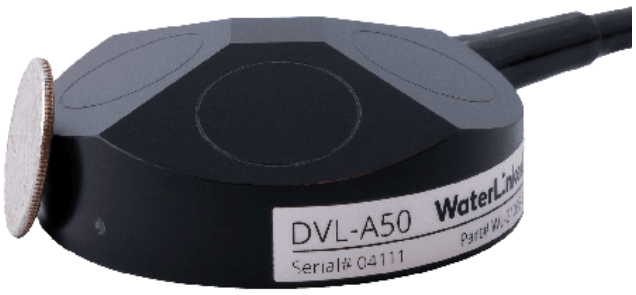
WaterLinked
A NORBIT Company

PRODUCT DOCUMENTATION

DVL



This PDF is generated automatically from the online documentation and contains the same information. The online version is always the most up to date:

https://docs.waterlinked.com/dvl/overview/

Content last changed 2026-10-19 · fbb04d6

Contents

Overview 4
Get started Quickstart guide 7

Models

A50 9
A100 14
A125 17
A250 23

Install and connect

Installation 26
Electrical 30
Network setup 35

Operate the device

Axes 37
Range mode 41
Water tracking 43
Time 46
Settings and configuration 49
Dead reckoning 52
Software updates 54

Web GUI

DVL A50/A125 web GUI 56
DVL A100/A250 web GUI 59

Integration

Integration overview 62

TCP JSON API

A50/A125 65
A100/A250 77

Serial protocol

A50/A125 85
A100/A250 99
PD formats 110
Triggering and synchronization 117
Software resources 119
DVL for BlueROV2 121
DVL with UGPS for BlueROV2 154

Troubleshooting

Diagnostic log 163
How to diagnose 164
Support package checklist 169
FAQ 171

Changelog

DVL A50/A125 changelog 179
DVL A100/A250 changelog 185

Overview

At Water Linked we pride ourselves on building the world's smallest DVLs in their respective categories. Our DVLs combine everything that is important in the smallest possible form factor: price and performance!

What is a DVL?

A DVL estimates velocity relative to the seafloor by sending acoustic waves from four angled transducers and measuring the frequency shift (Doppler effect) of the returned echoes. By combining measurements from all four transducers over time, the DVL can accurately estimate both the speed and direction of movement.



DVL comparison

Here is a quick overview of the differences between the DVL models:

Feature	DVL A50	DVL A100*	DVL A125	DVL A250
Altitude range**	5 cm to 50 m	5 cm to 100 m	5 cm to 125 m	30 cm to 250 m
Maximum velocity	3.75 m/s	5 m/s	9 m/s	15 m/s
Frequency	1 MHz	625 kHz	420 kHz	250 kHz

Feature	DVL A50	DVL A100*	DVL A125	DVL A250
Water tracking	Yes	Yes	Yes	Yes
Diameter / height	66 mm / 25 mm	90 mm / 34 mm	125 mm / 30 mm	149 mm / 40 mm
Weight in air / water	170 g / 105 g	200 g / 125 g	750 g / 500 g	1650 g / 750 g
Power consumption	4 W (average)	8 W (average)	4 W (average)	8 W (average)
Serial protocol	UART TTL (3.3 V, 5 V tolerant)	RS232	UART TTL (3.3 V, 5 V tolerant)	RS232
IMU output	No	Planned	No	Planned
Ping triggering	Software	Soft- and Hardware	Software	Soft- and Hardware
Cable entry	Side entry	Side entry or rear O-ring interface	Side entry	Side entry or rear O-ring interface
Communication	Ethernet and UART TTL	Ethernet and serial RS232	Ethernet and UART TTL	Ethernet and serial RS232
Source level (dB re 1 µPa @ 1 m)	197	210	198	207
Depth rating	300 m or 600 m	1000 m	1000 m or 4000 m	4000 m for side-entry cable configuration; rear O-ring interface version rated to 1000 m
Main use case	Compact platforms where size and payload capacity are limited and operations are conducted close to the bottom	Compact platforms where size and payload capacity are limited but require more range than the A50	Long-range and deep-water subsea operations in high-pressure environments	Long-range and deep-water subsea operations where large operating altitudes are required

*Coming Q4 2026

**Under ideal conditions

TODO: Confirm DVL A100 and DVL A250 source level/SPL before publishing source level/SPL in this comparison table. Keep SPL values in this overview only; FAQ should link here instead of repeating values. ## DVL A50 and DVL A125 comparison The DVL A50 and DVL A125 share the same general web GUI, configuration, electrical, and integration documentation. The main differences are operating range, frequency, velocity limit, depth rating, and physical size. ## DVL A100 and DVL A250 comparison The DVL A100 and DVL A250 share the same general web GUI, configuration, electrical, and integration documentation. The main differences are operating range, frequency, velocity limit, depth rating, and physical size. | Feature | DVL A100 | DVL A250 | | :--- | :--- | :--- | | **Altitude range** | 5 cm to 100 m | 30 cm to 250 m | | **Maximum velocity** | 5 m/s | 15 m/s | | **Frequency** | 625 kHz | 250 kHz | | **Depth rating** | 1,000 m | 4,000 m for side-entry cable configuration; rear O-ring interface version rated to 1,000 m | | **Diameter / height** | 90 mm / 34 mm | 149 mm / 40 mm | | **Weight in air / water** | 0.2 kg / 0.125 kg | 1.65 kg / 0.75 kg | | **Communication** | Ethernet and serial RS232 | Ethernet and serial RS232 | | **Protocols** | TCP JSON API, serial protocol, PD4, PD6. PD0 planned. | TCP JSON API, serial protocol, PD4, PD6. PD0 planned. | | **Main use case** | Compact platforms where size and payload capacity are limited | Long-range and deep-water subsea operations where large altitudes are required | TODO: Update the comparison when PD0 becomes available for DVL A100/A250.

Models

DVL A50

1 MHz · altitude to 50 m · 66 mm diameter

One of the world's smallest commercially available DVLs — an ultra-compact 4-beam unit for size- and payload-limited platforms operating close to the bottom.

Model details

DVL A100 *(coming Q4 2026)*

625 kHz · altitude to 100 m · 90 mm diameter

A compact, high-accuracy DVL for lightweight, integration-friendly platforms that need more range than the A50.

Model details

DVL A125

420 kHz · altitude to 125 m · 125 mm diameter

The step up from the A50: better performance at greater distances while keeping a small form factor.

Model details

DVL A250

250 kHz · altitude to 250 m · 149 mm diameter

The most capable model — long-range, deep-water velocity measurement for larger vehicles and demanding subsea operations.

Model details

Quickstart guide

This guide helps you connect and access the GUI for all DVLs.

1. Unbox and inspect

Check that the DVL, cable, and any interface hardware are undamaged.

2. Keep the DVL in water

The DVL should be submerged before it is powered. This prevents overheating and is required for valid velocity measurements.

> **Tip**
>
> Keep the DVL in a bucket of water when using it on a workbench.

3. Mount the DVL

Secure the DVL to your vehicle using the mounting holes and make sure the transducers have a clear line of sight to the bottom.

See Installation for common guidance on orientation, line of sight, vibration, bubbles, turbulence, and boat-mounted use cases.

Model	Mounting and dimensions
DVL A50	DVL A50
DVL A100	DVL A100
DVL A125	DVL A125
DVL A250	DVL A250

4. Connect power and Ethernet

Wire the DVL according to the correct electrical interface for your model, then connect Ethernet to your computer or network. Ethernet is the recommended interface for first setup because it gives access to the web GUI, diagnostics, configuration, software updates, and the TCP data stream.

Serial is available for vehicle integrations where a direct serial data stream is required.

Model	Electrical interface	Recommended interface	Optional serial interface
DVL A50	Electrical connection	Ethernet	UART TTL
DVL A100	Electrical connection	Ethernet	RS232
DVL A125	Electrical connection	Ethernet	UART TTL
DVL A250	Electrical connection	Ethernet	RS232

5. Configure network

Configure your computer to reach the DVL over Ethernet. See Network Setup for mDNS, fallback IP, static IP, and TCP information.

6. Access the web GUI

Open the DVL web GUI in a web browser.

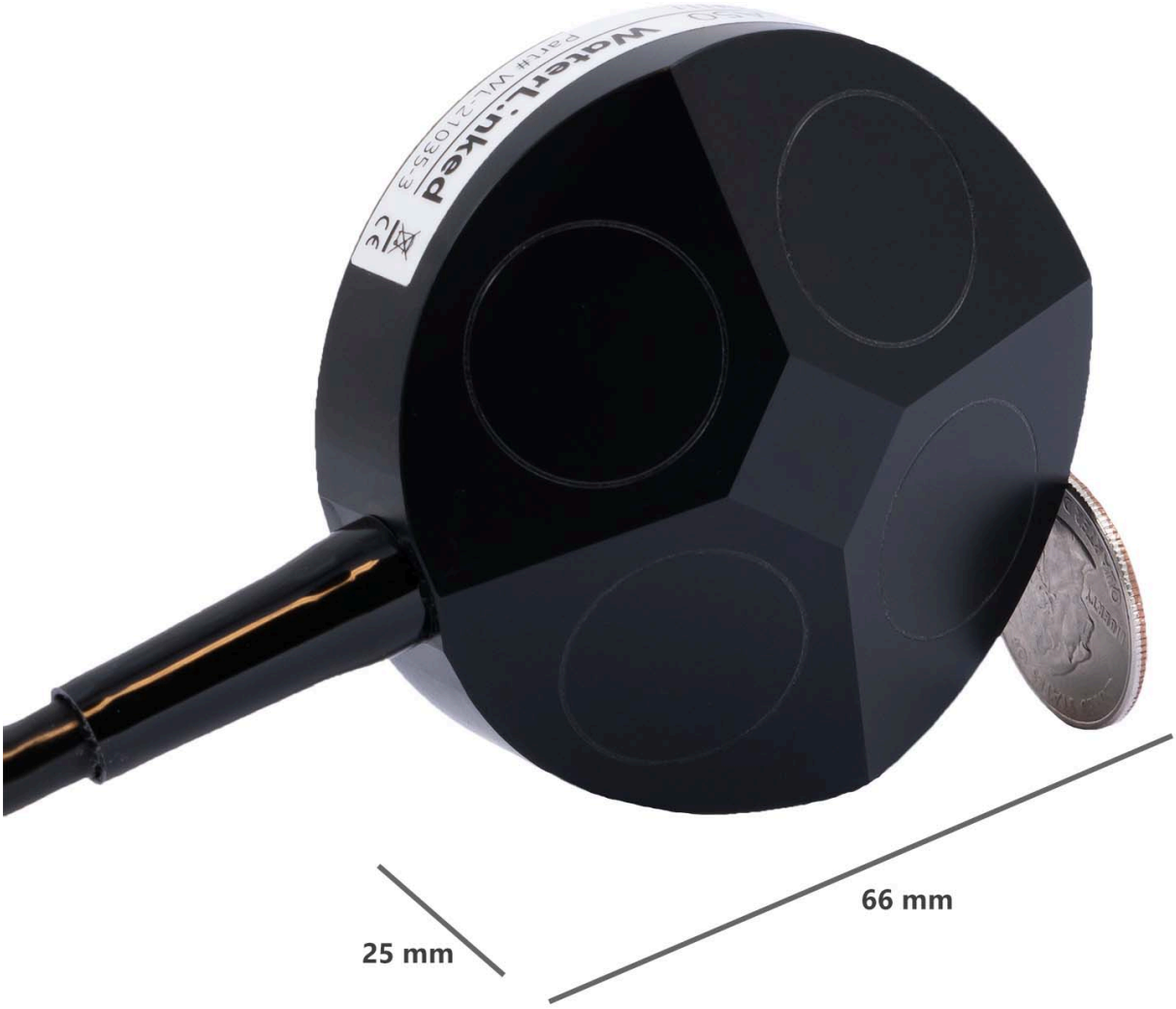
Model	Web GUI documentation
DVL A50/A125	web GUI
DVL A100/A250	web GUI

7. Need help?

Check out How to diagnose and FAQ.

DVL A50

Dimensions

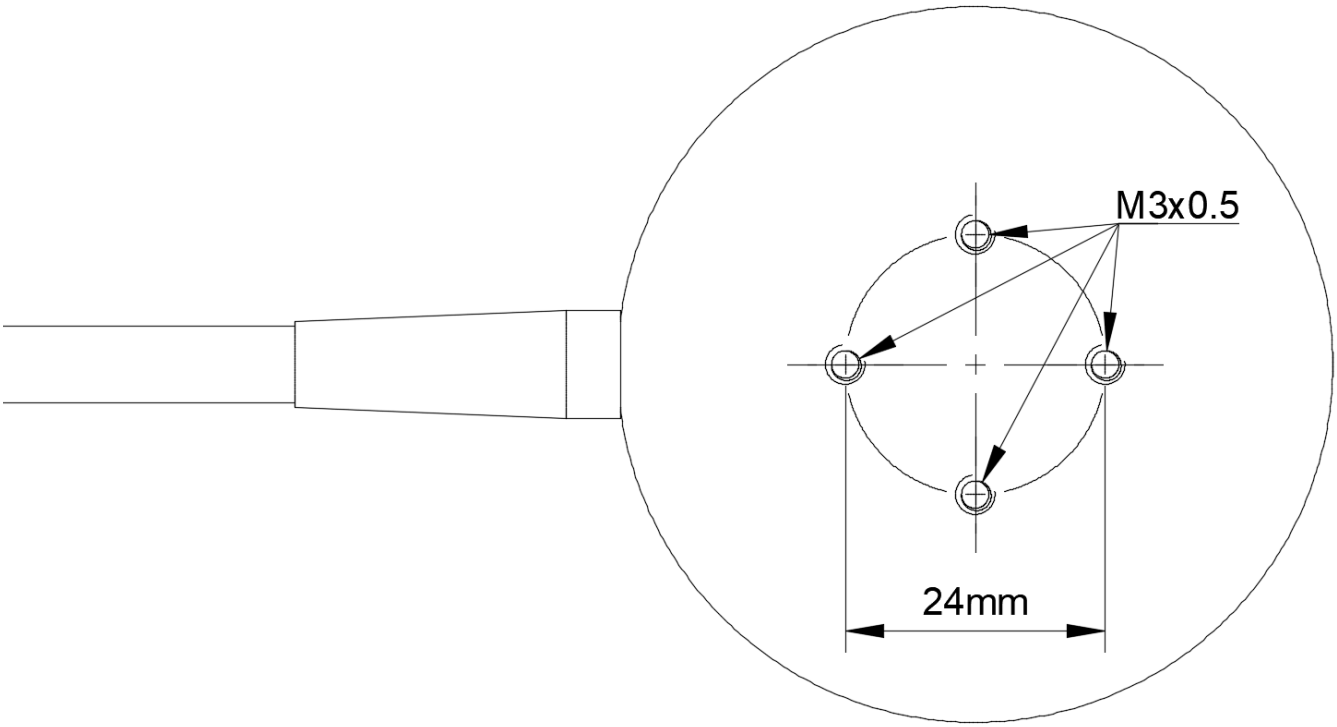


Cable dimensions

Delivered cable length: 3.0 m

Cable diameter: 6.8 mm +/- 0.2 mm

Mounting holes

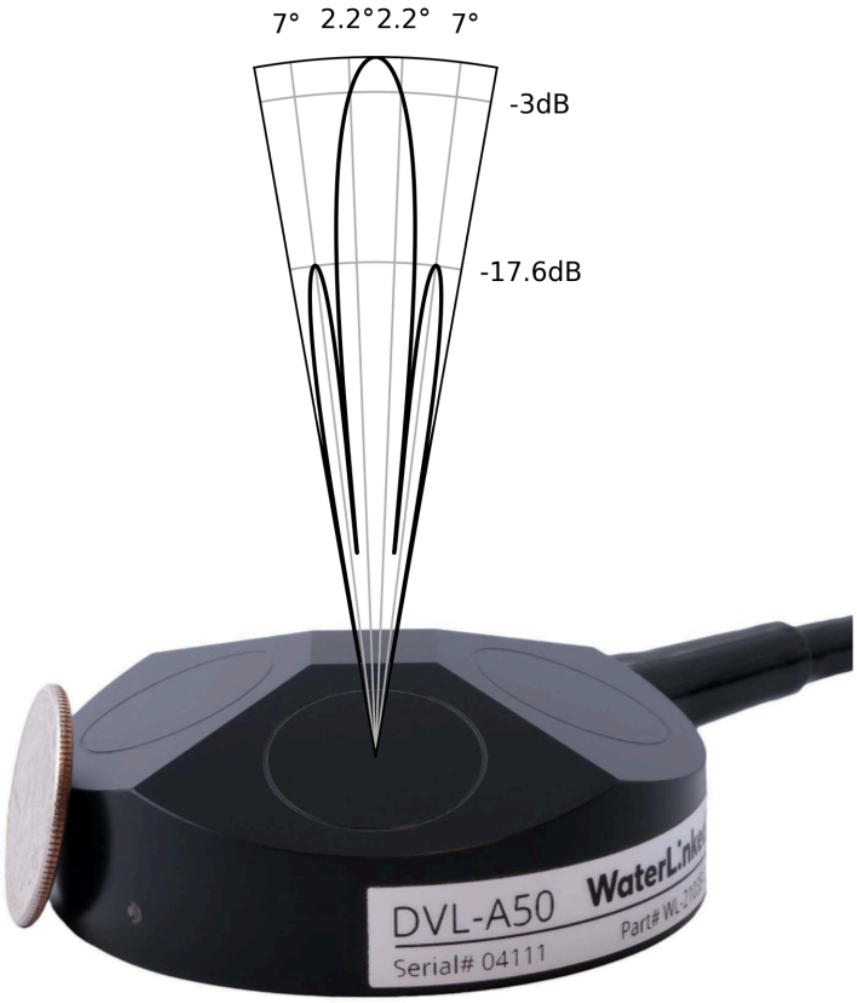


Transducer numbering



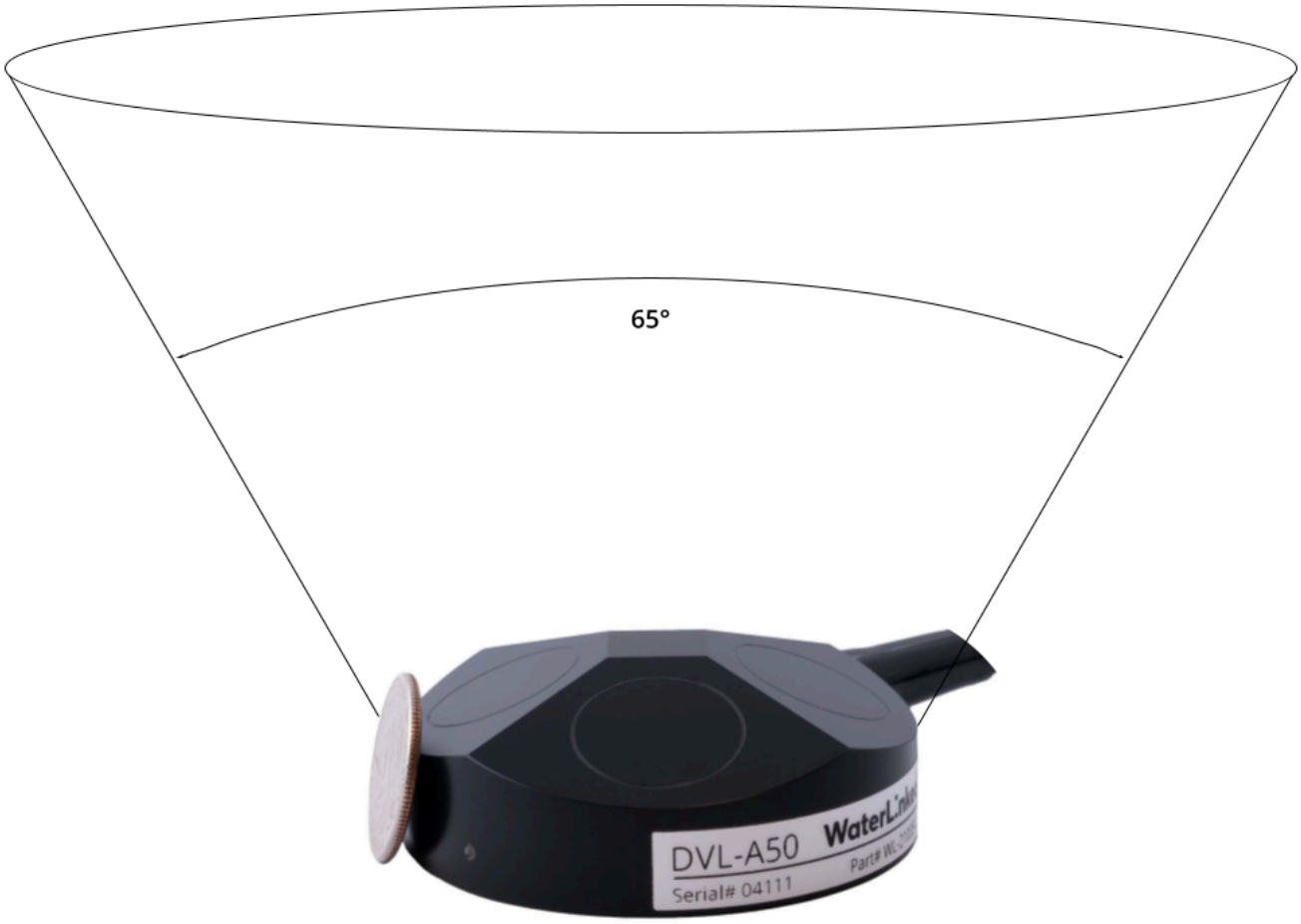
Mechanical drawings number the transducers from 1 to 4. Protocol messages and diagnostic logs use zero-based transducer `id` values from 0 to 3. See Transducer numbering and protocol IDs.

Transducer beam width



Half-power beam width is 4.4°

Line of sight



Datasheet

Datasheet

DVL A100

The DVL A100 is a compact, high-accuracy Doppler Velocity Log for lightweight and integration-friendly platforms.

It is designed for close-range and mid-range operation up to 100 m altitude, with Ethernet and RS232 serial interfaces for vehicle and topside integrations.

The DVL A100 is part of the same product family as the DVL A250. From a web GUI, configuration, protocol, and integration perspective, the two products are documented together where the behavior is shared. The main differences are operating range, frequency, speed limits, size, weight, depth rating, and physical configuration.

TODO: Add product image of DVL A100.

Key features

- Altitude range from 5 cm to 100 m.
- 4-beam convex Janus transducer array.
- 625 kHz transducer frequency.
- Integrated AHRS for dead reckoning.
- Ethernet and RS232 serial communication.
- TCP JSON API, serial protocol, PD4, and PD6. PD0 is planned but not available yet.
- Side cable entry and rear O-ring interface versions.

Mechanical and installation overview

Use the mechanical information together with the common Installation guidance. Before final drawings are added, verify dimensions, mounting hole placement, cable routing, and line of sight against the latest product drawing or datasheet.

Dimensions

- **Diameter**: 90 mm
- **Height**: 34 mm
- **Weight in air**: 0.2 kg
- **Weight in water**: 0.125 kg
- **Depth rating**: 1,000 m
- **Material**: PEEK (housing), Stainless Steel 316 (back plate)

TODO: Add DVL A100 dimensions drawing.

Physical interface versions

The DVL A100 is available in these physical interface versions:

- Side cable entry version with 3 m shielded cable.
- Rear O-ring interface version with 1 m shielded cable.

Cable dimensions

Cable diameter: 7.8 mm +/- 0.2 mm

The cable diameter is the same for the side-entry, rear O-ring interface and connector versions.

TODO: Add DVL A100 cable dimensions drawing, including side cable entry and rear O-ring interface variants.

Mounting holes

Use a rigid mount so the DVL moves with the vehicle or boat. Avoid flexible poles, unsupported brackets, and mounts that can vibrate.

TODO: Add DVL A100 mounting holes drawing.

Transducer numbering

Transducer numbering is useful when comparing web GUI or protocol diagnostics with the physical installation.

Mechanical drawings number the transducers from 1 to 4. Protocol messages and diagnostic logs use zero-based transducer `id` values from 0 to 3. See Transducer numbering and protocol IDs.

TODO: Add DVL A100 transducer numbering drawing.

Transducer beam geometry

Keep all beam paths unobstructed. Avoid mounting locations where brackets, frames, skids, cables, thrusters, propellers, or hull structures can intersect the beam paths.

The DVL A100 beam angle is 22.5 degrees.

TODO: Add DVL A100 transducer beam geometry drawing.

Line of sight

The DVL needs a clear acoustic path to the seabed or another reflecting surface. Strong turbulence, bubbles, vibration, and operation very close to the surface can reduce signal quality or cause loss of bottom lock.

For boat-mounted installations, consider the flow around the hull and avoid positions where bubbles or aerated water pass across the transducers. For ROV/AUV installations, avoid mounting directly in thruster wash or near moving parts.

TODO: Add DVL A100 line-of-sight drawing.

Datasheet

Datasheet

DVL A125

Buy DVL A125 here



Description

The DVL A125 is the next step up from the DVL A50, providing better performance at greater distances while still keeping a small form factor relative to competing DVLs.

The DVL A125 builds on the DVL A50 with increased performance, a small 4-beam setup, an open interface protocol, and mid-to-low cost.

> **Tip**
>
> Keep the DVL A125 in a bucket of water to ensure sufficient cooling when using the DVL on a workbench.

Dimensions

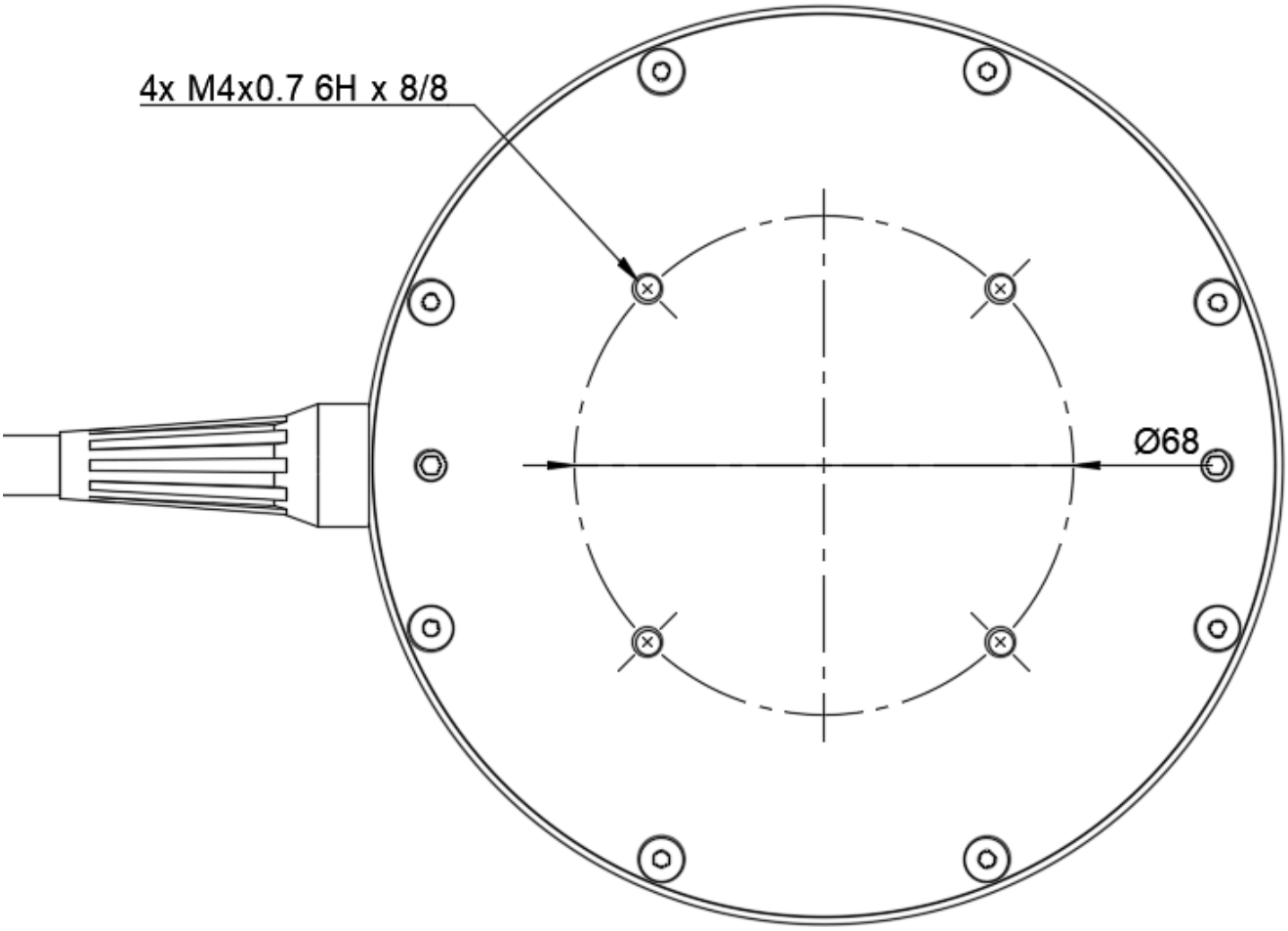


Cable dimensions

Delivered cable length: 3.0 m

Cable diameter: 7.8 mm +/- 0.2 mm

Mounting holes

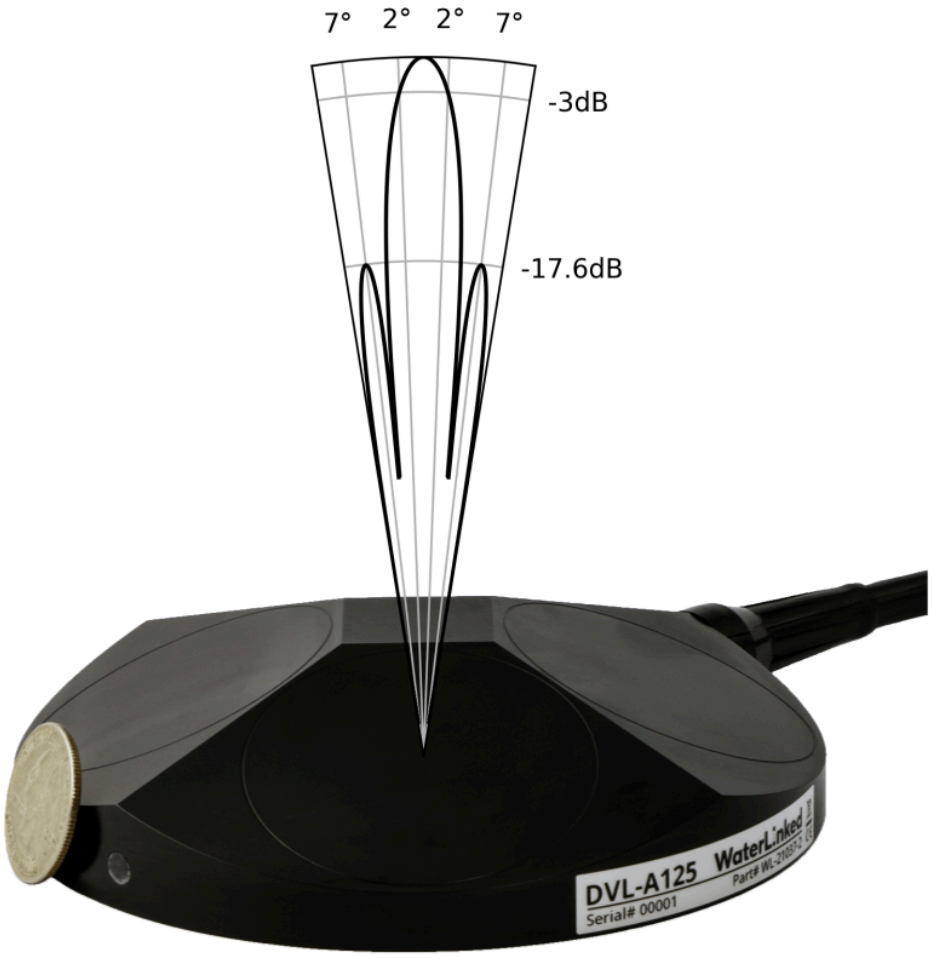


Transducer numbering



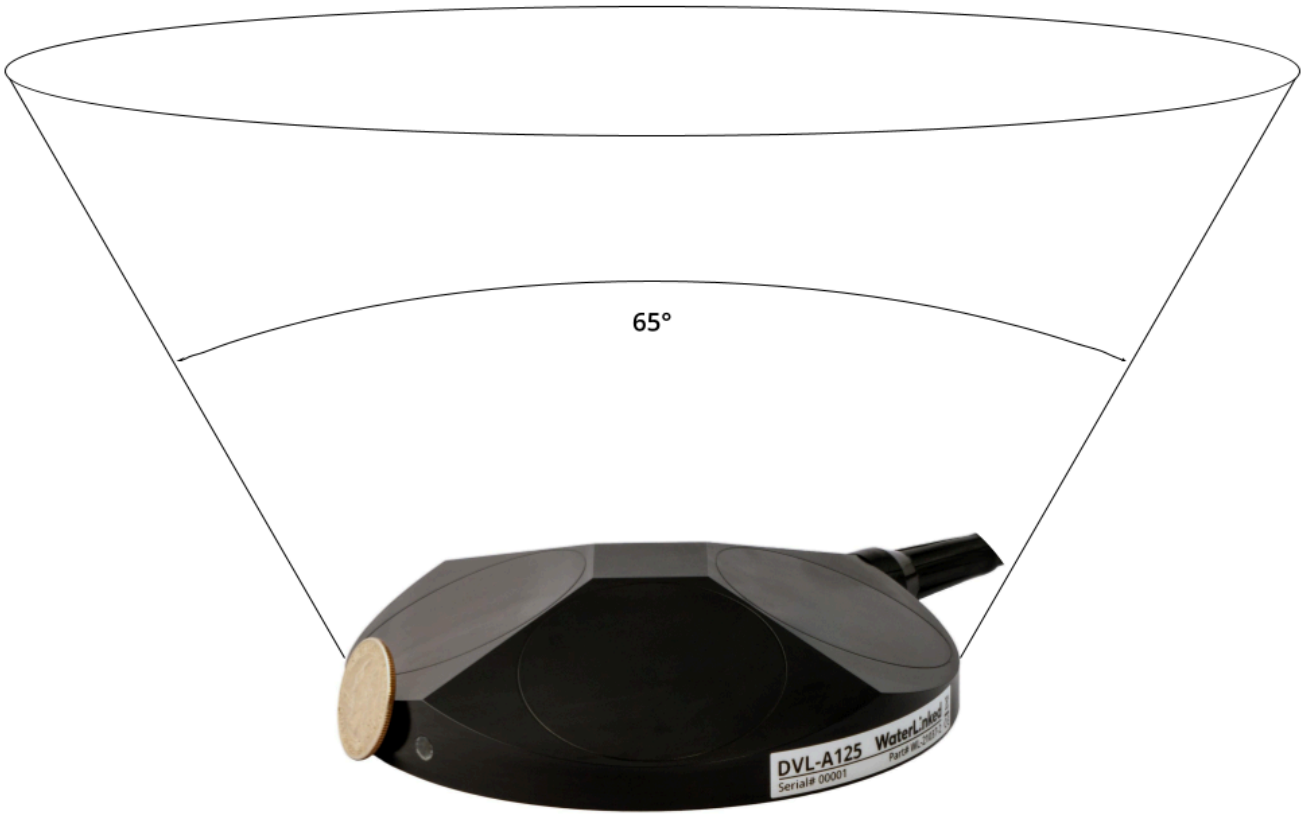
Mechanical drawings number the transducers from 1 to 4. Protocol messages and diagnostic logs use zero-based transducer `id` values from 0 to 3. See Transducer numbering and protocol IDs.

Transducer beam width



Half-power beam width is 4.1°

Line of sight



Datasheet

Datasheet

DVL A250

The DVL A250 is the most capable DVL in the lineup, delivering long-range performance while maintaining a compact and efficient design.

Operating at lower frequency for extended reach, the A250 enables reliable velocity measurements at distances up to 250 m, making it suitable for demanding subsea operations.

The DVL A250 combines high performance, a compact 4-beam setup, open interface protocol, and a competitive cost, making it a powerful solution for larger vehicles and deep-water applications.

TODO: Add product image of DVL A250.

Dimensions and FOV

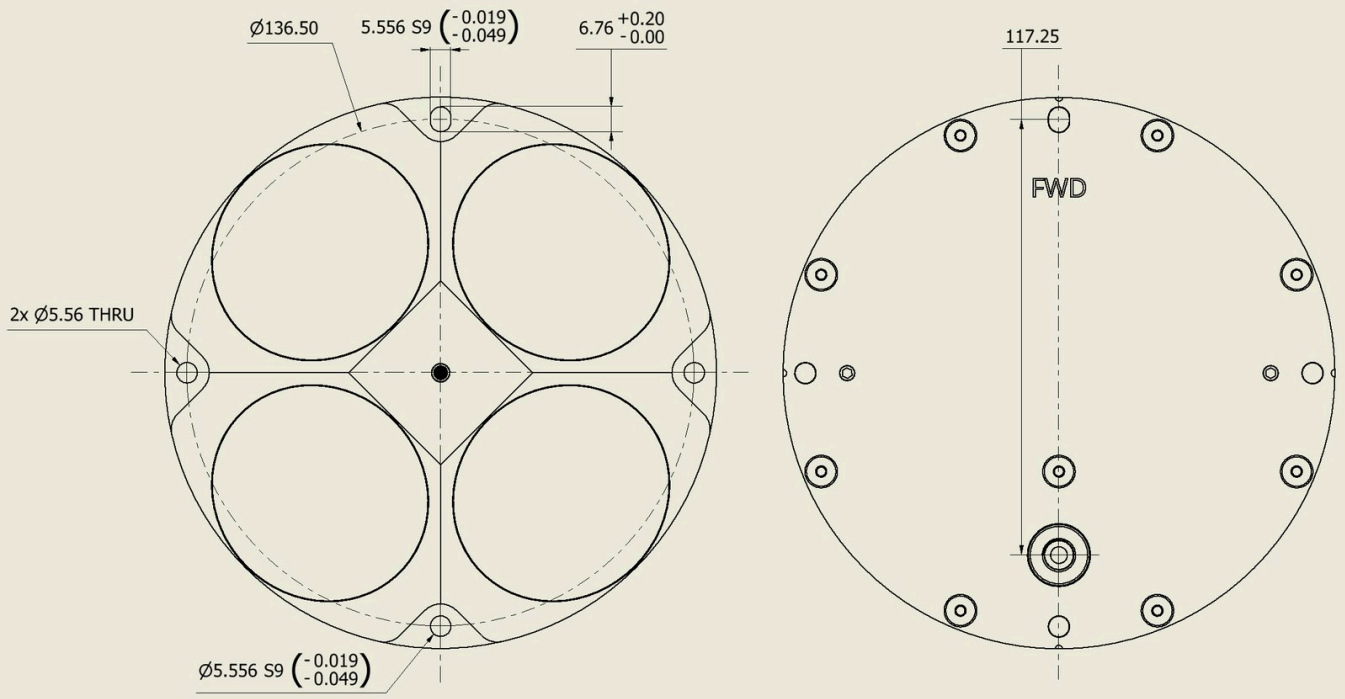
Use the mechanical information together with the common Installation guidance. Verify dimensions, mounting hole placement, cable routing, and line of sight against the latest product drawing or datasheet.

Dimensions

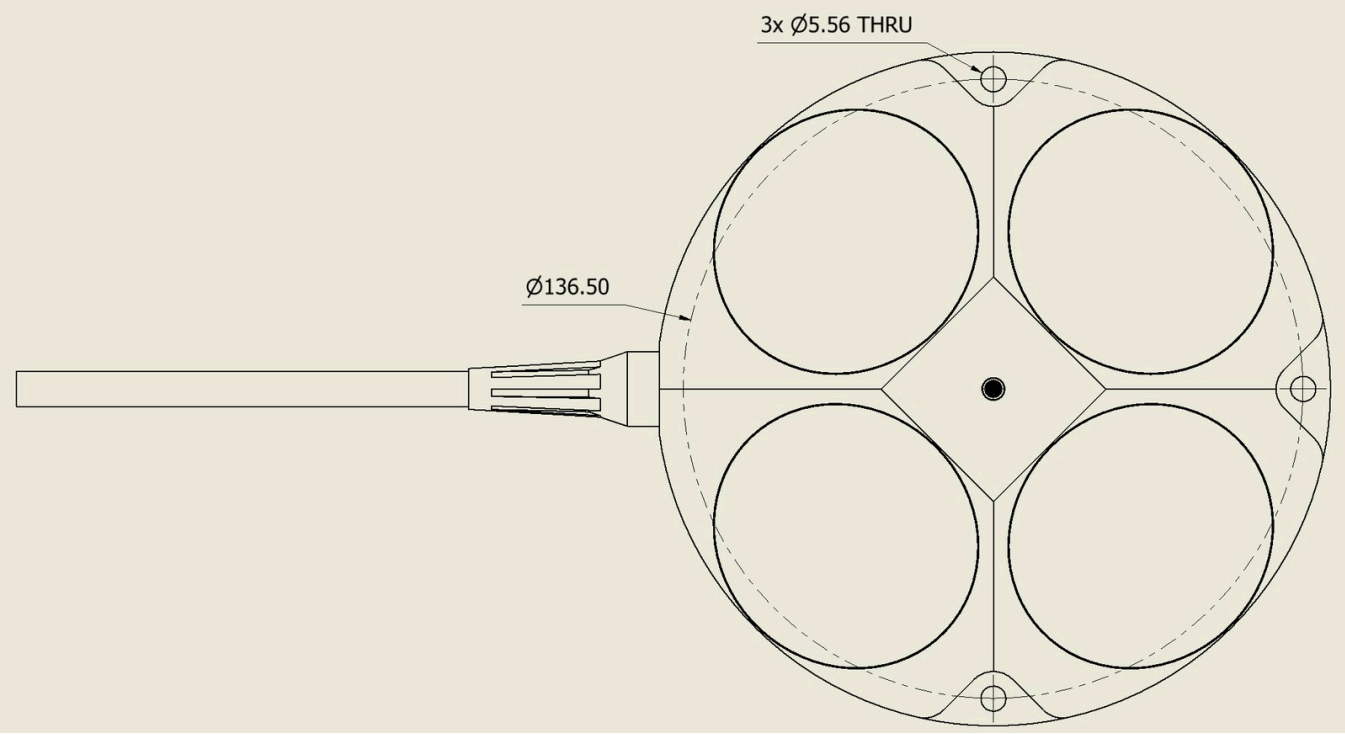
- **Diameter**: 149 mm
- **Height**: 40 mm
- **Weight in air**: 1.65 kg
- **Weight in water**: 0.75 kg
- **Depth rating**: 4,000 m for side-entry cable configuration; rear O-ring interface version rated to 1,000 m
- **Material**: PEEK (housing), Stainless Steel 316 (back plate)

Use the drawings below as mechanical reference for the two A250 interface variants.

Rear O-ring interface version



Side cable entry version



Cable dimensions

Cable diameter: 7.8 mm +/- 0.2 mm

The cable diameter is the same for the side-entry, rear O-ring interface and connector versions.

The DVL A250 is available with side-entry cable and rear O-ring interface options. A Seacon connector option is available for side-entry versions only.

Mounting holes

Use a rigid mount so the DVL moves with the vehicle or boat. Avoid flexible poles, unsupported brackets, and mounts that can vibrate.

Use the mechanical drawings above as the reference for mounting-hole placement.

Transducer numbering

Transducer numbering is useful when comparing web GUI or protocol diagnostics with the physical installation.

Mechanical drawings number the transducers from 1 to 4. Protocol messages and diagnostic logs use zero-based transducer `id` values from 0 to 3. See Transducer numbering and protocol IDs.

TODO: Add DVL A250 transducer numbering drawing.

Transducer beam geometry

Keep all beam paths unobstructed. Avoid mounting locations where brackets, frames, skids, cables, thrusters, propellers, or hull structures can intersect the beam paths.

TODO: Add DVL A250 transducer beam geometry drawing.

Half-power beam width is 22.5°

Line of sight

The DVL needs a clear acoustic path to the seabed or another reflecting surface. Strong turbulence, bubbles, vibration, and operation very close to the surface can reduce signal quality or cause loss of bottom lock.

For boat-mounted installations, consider the flow around the hull and avoid positions where bubbles or aerated water pass across the transducers. For ROV/AUV installations, avoid mounting directly in thruster wash or near moving parts.

TODO: Add DVL A250 line-of-sight drawing.

Datasheet

Datasheet

Installation

This page gives common installation guidance for all our DVLs. Use it together with the mechanical and electrical page for your specific model.

First-time setup

For the full end-to-end path (mount, wire, connect, open the web GUI), follow the Quickstart. This page covers mounting and placement; see also Electrical and Network setup.

Before installation

1. Check that the DVL, cable, connector, and mounting hardware are undamaged.
2. Review the model-specific dimensions, mounting holes, and line-of-sight drawings.
3. Plan cable routing so the cable is protected from strain, sharp bends, and moving parts.
4. Confirm the electrical interface for your model:
   - DVL electrical connection
   - DVL A50/A125 serial interface
   - DVL A100/A250 serial interface

Clear line of sight

The DVL needs a clear acoustic path from each transducer to the seabed or other reflecting surface.

When mounting the DVL:

- Keep all beam paths free from brackets, vehicle frames, payloads, hulls, skids, and cables.
- Make sure the DVL is not looking into a wall, tank side, or nearby structure during testing.
- Check the web GUI or protocol output to see whether the transducers report similar and reasonable distances in a flat test area.

See the model pages for line-of-sight drawings:

- DVL A50 line of sight
- DVL A100 line of sight
- DVL A125 line of sight
- DVL A250 line of sight

TODO: Add final DVL A100 line-of-sight asset when available. TODO: Add final DVL A250 line-of-sight asset when available.

Mounting orientation

Velocity and dead reckoning output use the configured vehicle frame. If the DVL is not mounted with its forward direction aligned with the vehicle forward direction, configure the mounting rotation offset.

See Axes for the common DVL axis convention and mounting rotation offset.

> **Warning**
>
> Incorrect mounting rotation can make valid DVL data appear to point in the wrong direction.

Rigid mounting and vibration

Use a rigid mount so the DVL moves with the vehicle or boat. Avoid flexible poles, long unsupported brackets, loose fasteners, or mounts that can vibrate.

Mechanical vibration can make the velocity estimate noisy or unstable. If measurements look noisy:

1. Check that the DVL cannot move relative to the vehicle.
2. Check for loose fasteners or flexing brackets.
3. Compare measurements with thrusters or motors off and on.

Thrusters, propellers, hulls, and bubbles

Air bubbles and turbulent flow can reduce signal quality and cause invalid velocity or loss of bottom lock.

When possible:

- Mount the DVL away from strong turbulence from thrusters, propellers, pumps, and hull edges.
- Avoid positions where bubbles pass across the transducers.
- Avoid mounting directly behind a propeller or thruster wash.
- Consider the vehicle direction of travel and where bubbles will move during operation.

No strict minimum distance is specified here because it depends on the vehicle, speed, water flow, and installation.

Boat-mounted use

For boat-mounted installations, consider flow and bubbles around the hull:

- Avoid mounting where the hull creates strong aerated flow.
- Avoid locations that come out of the water in waves or when the boat pitches and rolls.
- Use a rigid pole or bracket that does not vibrate.
- Keep the DVL submerged and stable during measurements.
- Check bottom lock at the expected boat speed before relying on the data.

Operation close to the surface

Operation close to the water surface can be affected by bubbles, waves, and changing submersion. Keep the DVL fully submerged and avoid positions where waves or vehicle motion expose the transducers to air.

If the DVL loses bottom lock near the surface, check for:

- Bubbles passing across the transducers.
- Waves causing pitch and roll.
- The DVL or mounting bracket moving relative to the vehicle.
- Beam paths that intersect the hull, dock, tank wall, or other nearby structures.

Other acoustic instruments

Other acoustic instruments can interfere with DVL measurements if they transmit at the same time or in a similar frequency range.

If several acoustic instruments are used together:

1. Test each instrument by itself first.
2. Check DVL performance with the other instruments active.
3. Use trigger or synchronization options when needed. See Integration.

Grounding, power, and cable considerations

Follow the electrical page for your model:

- DVL electrical connection
- DVL A50/A125 serial interface
- DVL A100/A250 serial interface

General checks:

- Use an adequate power supply for the model.
- Protect the cable from strain and abrasion.
- Keep power and communication wiring away from high-noise wiring when practical.
- Follow the shield grounding guidance in the electrical documentation.

First water test

1. Keep the DVL submerged before powering it for normal operation.
2. Open the web GUI and check that the DVL is connected.
3. Confirm that acoustics are enabled.
4. Check that the DVL gets bottom lock in a simple test environment.
5. Check velocity, altitude, and transducer distances.

6. If the output looks wrong, verify mounting rotation, range mode, line of sight, bubbles, vibration, and acoustic interference.

See How to diagnose if the DVL does not behave as expected.

Electrical connection

This page covers electrical connection, LED signals, wiring, shielding, and serial interface for all DVLs.

First-time setup

New to the DVL? The Quickstart covers the full setup path (mount, wire, connect, open the web GUI). This page is the detailed wiring reference; to get on the network afterwards, see Network setup.

Most wiring is common across the DVL models. The serial electrical interface differs:

- DVL A50 and DVL A125 use 3.3 V UART, 5 V tolerant.
- DVL A100 and DVL A250 use RS232.

LED signals

Visualization	LED signal	Description
●	**No green light**	Power is off.
●	**Fixed green light**	The DVL is powering on. Depending on configuration it can take ~30-60 seconds to power on.
●●●●	**Flashing green light** (mostly off, slow)	DVL looking for bottom lock.
●●●●	**Flashing green light** (mostly on, slow)	DVL has bottom lock.
●●●●	**Flashing green light** (fast)	DVL is in thermal shutdown.

Wiring interface

Power and Ethernet wiring are common. The brown and brown/white serial pair depends on the DVL model.

Common power and Ethernet wires

Interface	Color	Visual
Negative/Ground	Black	●

Interface	Color	Visual
Positive (10–30 Vdc)	Red	
ETH RX+	Green/White	
ETH RX-	Green	
ETH TX-	Orange	
ETH TX+	Orange/White	
Shielded wire	Naked/silver	

Model-specific serial wires

Model	Interface	Color	Visual
DVL A50/A125	UART TX	Brown/White	
DVL A50/A125	UART RX	Brown	
DVL A100/A250	Serial (RS232) TX	Brown/White	
DVL A100/A250	Serial (RS232) RX	Brown	

Note

Power must be applied to the power terminals before applying voltage to the serial pins.

Seacon connector option

The Seacon connector option is available for:

- DVL A50.
- DVL A100 side-entry versions only.
- DVL A250 side-entry versions only.

TODO: Confirm whether a Seacon connector option is available for DVL A100.

DVL A50/A100/A250 Seacon MC8M connector

The table below shows the pinout for the DVL A50/A100/A250 Seacon MC8M connector.

Interface	Pin	Color	Visual
Negative/Ground	1	Black	
Positive (10–30 Vdc)	2	Red	
ETH RX+	3	Green/White	
ETH RX-	4	Green	
ETH TX-	5	Orange	
ETH TX+	6	Orange/White	
Serial TX	7	Brown/White	
Serial RX	8	Brown	

TODO: Confirm and publish the DVL A250 side-entry Seacon connector pinout before reusing the A50/A125 table for A250.

Power specifications

Specification	DVL A50	DVL A100	DVL A125	DVL A250
Input voltage	10–30 Vdc	10–30 Vdc	10–30 Vdc	10–30 Vdc
Power consumption, average	4 W	8 W	4 W	8 W
Power-on current surge	35 W	30 W	35 W	30 W

TODO: Add or link DVL A50/A125 electrical specifications if they should be published here.

Shielded cable information

Shielded DVL cable helps protect the communication and power lines from external electromagnetic interference (EMI) and radio frequency interference (RFI). The DVL electronics are not internally connected to the shield; it is up to the user to decide whether and how to connect the shield.

Note

For DVL A50, shielded cables were introduced autumn 2024.

All DVL A125 units are delivered with shielded cable.

TODO: Confirm whether all DVL A250 cable and connector variants use the same shield arrangement.

Connecting the shield

For optimal performance, especially when mounted on an ROV (Remotely Operated Vehicle) or AUV (Autonomous Underwater Vehicle), connect the shield to reduce the impact of interference on communication and power signals. The most common connection methods are:

1. **Chassis ground**: Connect the shield to the chassis of the ROV/AUV. This gives the shield a path to ground for noise and interference, and the vehicle chassis often acts as a common grounding reference.
2. **Common ground**: Connect the shield to the common electrical ground of the ROV/AUV. This may be used when the electrical and signal systems are designed to share a common ground.

Recommendation

We recommend connecting the shield to the **chassis ground** of the ROV or AUV if possible. If your system design uses a common electrical ground, that option can also be considered.

Ground the shield at one end only to avoid ground loops, unless your system design requires grounding at both ends.

Terminal / serial interface

DVL A50 and DVL A125 serial interface

DVL A50 and DVL A125 use the same 3.3 V UART interface, 5 V tolerant.

Setting	Value
Interface	3.3 V UART, 5 V tolerant
Baud rate	115200
Data/parity/stop	8N1
Flow control	None

Warning

There can be a momentary (<10 us) power spike (~5 V) on the UART lines when power is applied to the DVL, which may damage equipment that is not 5 V tolerant.

See the DVL A50/A125 serial protocol.

DVL A100 and DVL A250 serial interface

DVL A100 and DVL A250 use a standard RS232 serial interface.

Setting	Value
Interface	RS232
Baud rate	115200
Data/parity/stop	8N1
Flow control	None

See the DVL A100/A250 serial protocol.

Network Setup

The DVL has several services available over Ethernet:

- Multicast DNS (mDNS) for easy discovery
- Web GUI for live data, configuration, diagnostics, and software updates
- TCP data stream
- Software updates

Use Ethernet for first setup whenever possible. It gives access to the web GUI, diagnostics, configuration, software updates, and the TCP data stream.

First-time setup

Connect power and wiring first (see Electrical), then use this page to get on the network. For the full end-to-end setup path, follow the Quickstart.

Network configuration

Multicast DNS (mDNS)

The DVL runs a DHCP client which attempts to obtain an IP address from a DHCP server on the same network, such as a router.

The DVL also supports mDNS. On a computer that supports mDNS, open the web GUI at:

```
http://waterlinked-dvl
```

Note

If no DHCP server is available on the network, use the fallback IP or configure a static IP address. The DVL can spend up to 5 minutes searching for a DHCP server.

Fallback IP

The DVL is always available with the static fallback IP address:

```
192.168.194.95
```

To connect using the fallback IP:

1. Connect an Ethernet cable directly from the DVL to your computer.
2. Configure the computer Ethernet interface with a static IP address in the same subnet, for example `192.168.194.90` with subnet mask `255.255.255.0`.

3. Activate the Ethernet interface.
4. Open `http://192.168.194.95` in a web browser.

Via the web GUI

The DVL IP configuration can be changed in the web GUI:

- DHCP client: the DVL obtains an IP address from a DHCP server on the network.
- Static IP: the DVL uses a configured static IP address.

After changing the IP configuration, reboot the DVL for the settings to take effect.

> **Note**
>
> Boot time depends on IP configuration. Using the fallback IP, boot time can be as much as 1 min 30 sec. With static IP, it can be as low as 20 sec.

TCP data stream

The DVL can output data and receive commands over Transmission Control Protocol (TCP). A TCP server is available on port **16171**. It outputs the latest data to connected clients and listens for commands from connected clients.

See the DVL TCP JSON API for:

- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API

Axes

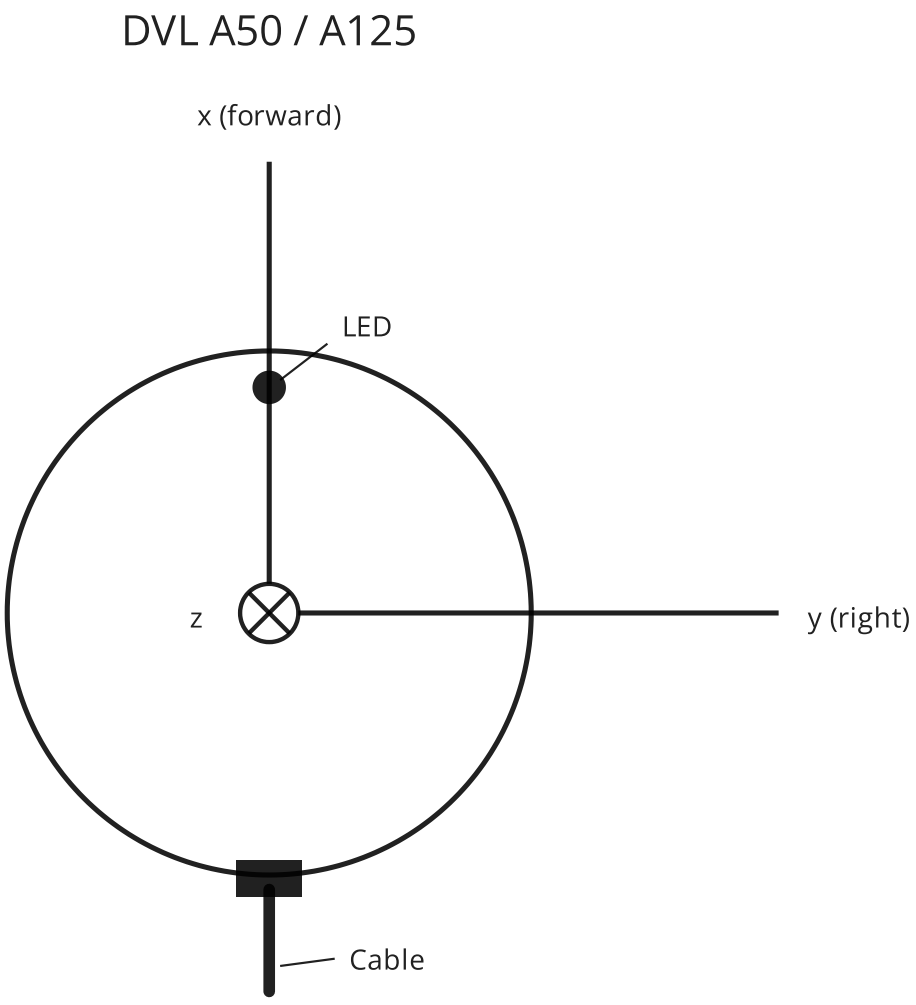
The axis convention is the same for all DVL models. Velocities and dead reckoning output use the vehicle frame. By default, the vehicle frame is the same as the body frame, but it can be adjusted to allow flexible mounting on the vehicle.

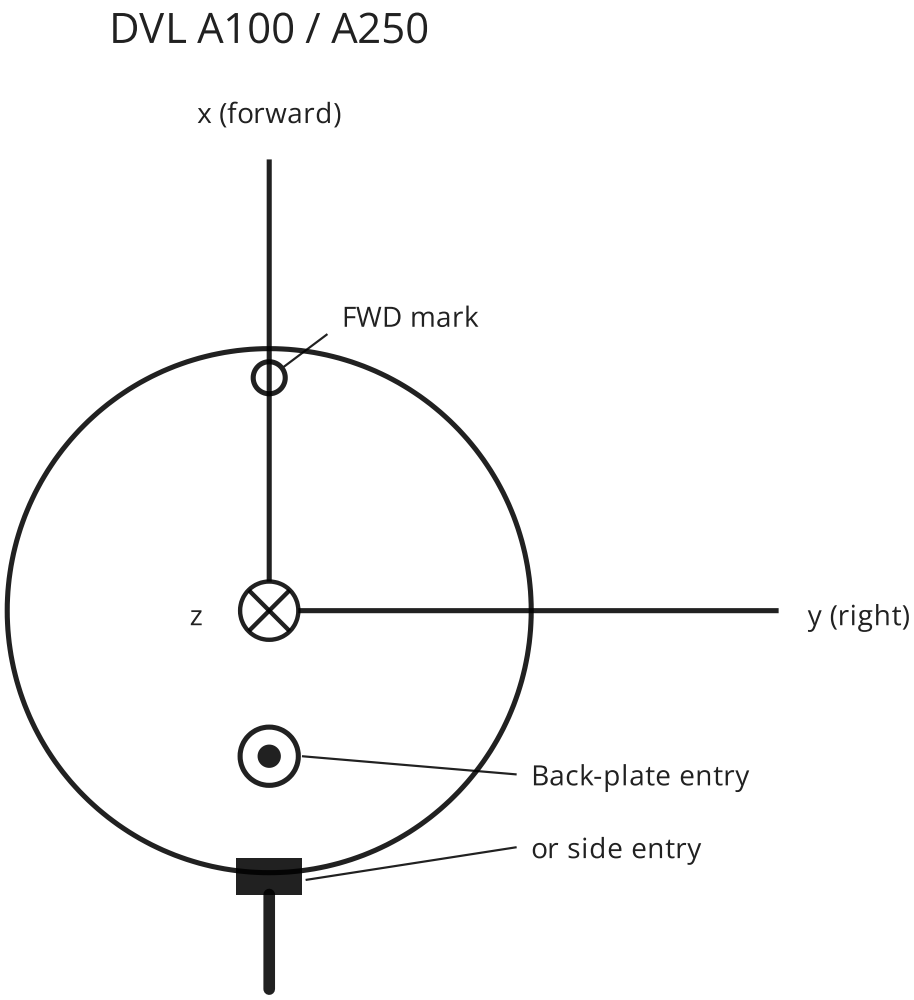
Body frame

The DVL outputs velocity in a right-handed body frame:

- **x** points forward.
- **y** points right (starboard).
- **z** points down, toward the transducers (the metal back plate faces up).

Seen from above, with forward pointing up and ⊗ marking the **z** axis (pointing into the page, toward the seabed), the two model groups indicate forward differently:

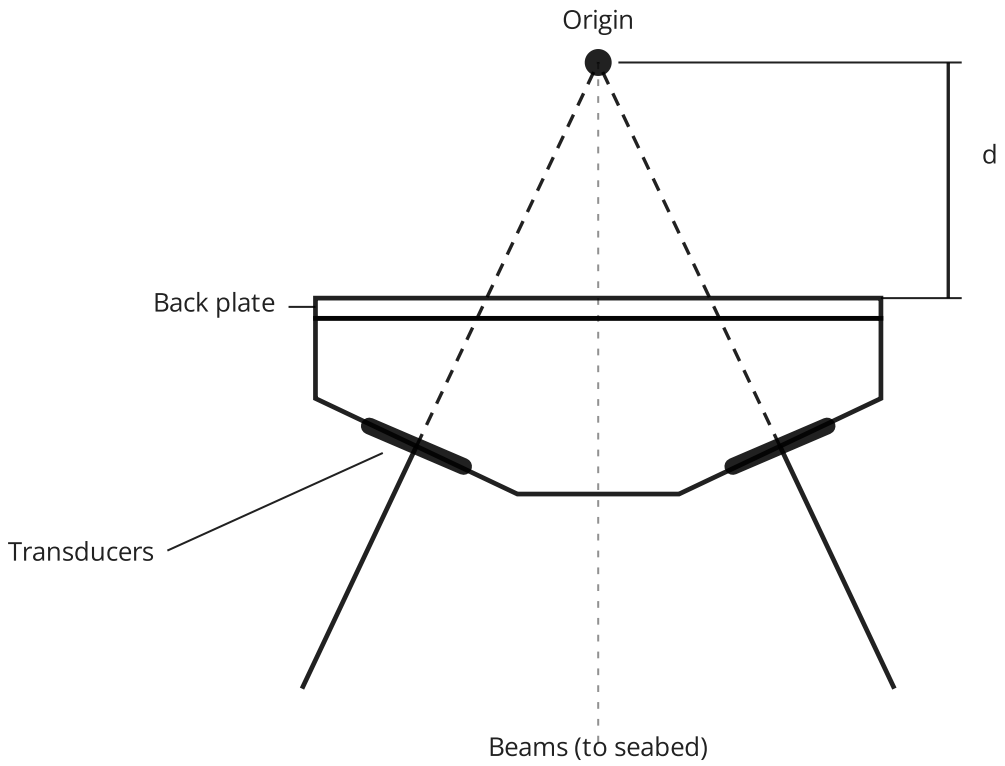




Finding forward on the device. On the DVL A50 and DVL A125, the status LED sits at the forward edge and marks **+x**. On the DVL A100 and DVL A250, the status LED faces down (the **z** direction), so forward is instead stamped **FWD** on the metal back plate. On all models the cable is toward the back of the housing, opposite forward (the **−x** side). On the DVL A100 and DVL A250 it enters either from the side or through the back plate (rear O-ring interface), as shown above. The mechanical drawings on the DVL A100 and DVL A250 pages show the FWD mark and cable position.

Origin

The origin of the body frame is the point at which the DVL measures velocity: the point on the DVL's central axis where the transducer beam axes intersect. In standard mounting orientation (transducers facing down, metal back plate up), it sits above the metal back plate, where the beam axes meet when extended upward:



The distance from the top of the back plate to the origin (shown as **d** above) depends on the model:

Model	Distance from back plate to origin
DVL A50	28 mm
DVL A100	38 mm
DVL A125	70 mm
DVL A250	84 mm

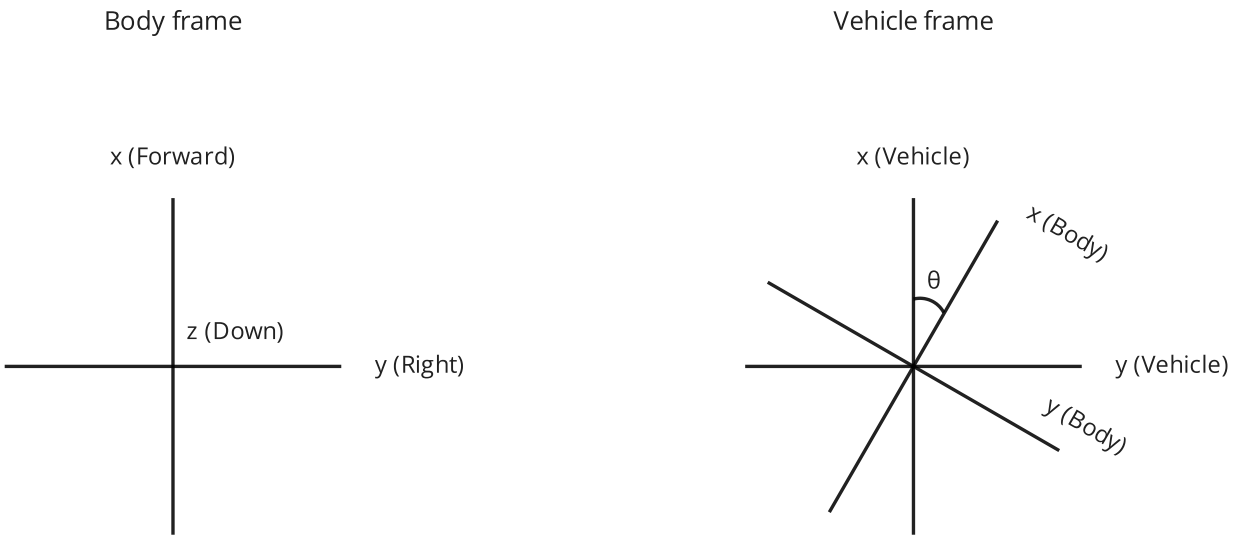
By default, the body frame and vehicle frame are the same and align with the frame used for dead reckoning.

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

Vehicle frame

The DVL can be mounted at an angle to the forward direction of the vehicle (yaw).



The clockwise angle θ in degrees around the Z axis, in the X-Y plane, from the forward axis of the vehicle to the forward axis of the DVL can be entered as a mounting rotation offset in the web GUI, or through the configuration method over the TCP JSON API or serial protocol.

The DVL will then output data in the vehicle frame obtained by rotating the DVL body frame anti-clockwise around the Z axis by θ degrees. The X axis of the velocity output will be aligned with the forward axis of the vehicle. At time zero, the X axis of the DVL's dead-reckoning frame will be aligned with the forward axis of the vehicle.

For example:

- If the DVL is mounted back-to-front (cable aligned with the forward axis of the vehicle), the mounting rotation offset should be set to 180 degrees.
- If the A50/A125 is mounted with the LED at 90 degrees clockwise from the forward axis of the vehicle, the mounting rotation offset should be set to 90 degrees.

Mounting offset in pitch and roll

This is possible, but the translation will have to be done by the user as there are no internal functions to compensate for this.

The DVL will always assume it is facing perpendicular to the surface.

Range mode

Range mode configuration can instruct the DVL to search for bottom lock in a limited altitude range. This can improve performance when the expected minimum and maximum altitude are known, for example in a river, pool, tank, or other constrained environment.

Model applicability

The range-mode concept applies to all DVLs. The table below documents the currently published A50/A125 range modes.

TODO: Confirm final DVL A100 and DVL A250 range mode altitude boundaries and update this page with a model-specific table or model column.

The same configuration parameter can also enable water tracking.

Configuration format

Range specifier	Behavior
`auto`	The DVL searches for bottom lock in its full operational area. This is the default.
`=a`	The DVL is locked to range mode `a`, where `a` is a number from `0` to `4`.
`a<=b`	The DVL searches for bottom lock from range mode `a` through range mode `b`.
`wt`	Enable water tracking on supported software. See Water tracking.

DVL A50/A125 range modes

Range mode	Lower altitude (m)	Upper altitude (m)	Update rate per second (Hz)
0	0.05	0.6	15
1	0.3	3.0	10
2	1.5	14	5 - 6
3	7.7	36	7 - 8
4	15	max	2 - 4

Range mode	Lower altitude (m)	Upper altitude (m)	Update rate per second (Hz)
wt	1.5	4.5	2

DVL A100/A250 range modes

Range mode	Lower altitude (m)	Upper altitude (m)	Update rate per second (Hz)
0	0.05	0.6	15
1	0.3	3.0	10
2	1.5	14	10
3	7.7	72	2 - 3
4	31	max	1 - 2
wt	1.5	4.5	2

Set with TCP JSON API or serial protocol

TCP JSON API

- DVL A50/A125 TCP JSON API configuration
- DVL A100/A250 TCP JSON API configuration

Serial

- DVL A50/A125 serial configuration
- DVL A100/A250 serial configuration

Water tracking

Water tracking is a mode where the DVL estimates velocity relative to the water column instead of the sea bottom. It can be useful when the DVL cannot get bottom lock, or when the movement of the water itself is relevant.

Supported models

Model	Water tracking	Available from software version
DVL A50	Supported	2.7.0
DVL A100	Supported	3.4.0
DVL A125	Supported	2.7.1
DVL A250	Supported	3.3.0

Water tracking requires a software version with water tracking support. See Software updates for update instructions.

Water tracking works the same way on all models.

TODO: Confirm the A100 water-tracking availability version. Water tracking was added in 3.3.0, but A100 support was added in 3.4.0.

Mode of operation

Water tracking uses acoustic reflections from the water column, approximately `1.5 m to 4.5 m` below the DVL. It is intended for use when the distance to the bottom is more than 4.5 m.

When the bottom is within the normal bottom-tracking range of the DVL, water-tracking velocity estimates can be affected by bottom reflections. With fast currents the velocity estimate can become invalid, and with slow currents the estimate can be biased toward the bottom-relative velocity.

The maximum measurable velocity in water tracking mode is 7.5 m/s.

Update rate is `2 Hz`.

Warning

Dead reckoning performance with water tracking is untested and can give large errors.

Web GUI

The dashboard shows the active tracking mode. On supported software, the tracking mode can be selected from the configuration page.

The diagnostic page still shows spatial velocity, per-transducer velocity, and validity. In water tracking mode, the acoustic signal view shows signal strength and noise as separate time-based graphs.

Enable water tracking

Water tracking is enabled by setting `range_mode` to `wt`. Select the interface you integrate with:

TCP JSON API

Available in TCP JSON API version `json_v3.2` and up on DVL A50/A125, and `json_v4.1` and up on DVL A100/A250. The commands and reports are identical on both model families; only the version numbering differs, because DVL A100/A250 have additional capabilities such as hardware triggering.

Use the TCP JSON API configuration method for your model:

- DVL A50/A125 TCP JSON API configuration
- DVL A100/A250 TCP JSON API configuration

Enable water tracking by setting `range_mode` to `wt`:

```
{"command":"set_config","parameters":{"range_mode":"wt"}}
```

Set `range_mode` back to `auto` to return to automatic bottom tracking:

```
{"command":"set_config","parameters":{"range_mode":"auto"}}
```

Water tracking does not introduce a separate report type. The DVL continues to use the normal velocity-and-transducer report, with the field `"type"` showing the active tracking mode. When water tracking is active, the report describes velocity relative to the water column rather than velocity relative to the bottom and the `"type"` will be `"velocity_water"`.

See the TCP JSON API for your model for the report fields:

- DVL A50/A125 TCP JSON API velocity-and-transducer report
- DVL A100/A250 TCP JSON API velocity-and-transducer report

Serial API

Available in serial protocol `2.7.0` and up on DVL A50/A125, and `3.1.0` and up on DVL A100/A250. The commands and reports are identical on both model families; only the version numbering differs, because DVL A100/A250 have additional capabilities such as hardware triggering.

Use the serial configuration method for your model:

- DVL A50/A125 serial configuration

- DVL A100/A250 serial configuration

Enable water tracking by setting `range_mode` to `wt` using the `wcs` command. The `wcs` command takes one more parameter on DVL A100/A250 than on DVL A50/A125, so the number of commas differs between the model families.

On DVL A50/A125:

```
wcs,,,,,wt,
```

On DVL A100/A250:

```
wcs,,,,,wt,,
```

Set `range_mode` back to `auto` to return to automatic bottom tracking.

On DVL A50/A125:

```
wcs,,,,,auto,
```

On DVL A100/A250:

```
wcs,,,,,auto,,
```

Water tracking introduces a serial report called `wrs`, which is similar to the `wrz` report but does not include altitude data. Use this data when water tracking is activated.

```
wrs,[vx],[vy],[vz],[valid],[fom],[covariance],[time_of_validity],[time_of_transmission],
[time],[status]
```

See the serial protocol for your model for the report format:

- DVL A50/A125 serial velocity report
- DVL A100/A250 serial velocity report

> ⚠ **`wrs` is used for two different reports on DVL A50/A125**
>
> On DVL A50/A125 the `wrs` prefix is used for both the time status response and the water tracking velocity report. Inspect the fields after the prefix to tell them apart: a water tracking velocity report begins with a decimal velocity (e.g. `wrs,0.120,...`), while a time status report begins with an integer epoch timestamp (e.g. `wrs,1776409079,...`).

Time

The DVL continously attempts to synchronize time using the Network Time Protocol (NTP). When not synchronized with NTP, it is also possible to set time manually.

Supported models

Model	Support
A50	Supported, starting with software release 2.7.2
A100	Planned. Not supported in 3.2.0
A125	Supported, starting with software release 2.7.2
A250	Planned. Not supported in 3.2.0

See Software updates for update instructions.

Importance of setting time

The DVL does know the current time when booting, and initial timestamps output from the DVL will be relative to some arbitrary time in the past. It is recommended to synchronize the DVL's time before using its output.

NTP server address configuration: auto or custom

The NTP server address can be configured as either "auto" or a specific server address. When NTP server address is configured as "auto", the DVL will attempt to reach a default pool of internet time servers. The configuration is persisted across reboots.

Setting manual time

When the DVL has not yet achieved NTP sync it is possible to set time manually. This is useful in deployment scenarios without a reachable NTP server, or as a fallback until an NTP server becomes available. A manually set time will be disregarded if the DVL later achieves NTP sync.

Time status

The DVL separates between NTP configuration and time status. Time status consists of the following fields:

Field	Description
`system_time`	system time
`ntp_synced`	true if synchronized to NTP source, false otherwise
`ntp_synced_to`	address the sonar is synchronized to if NTP is synchronized, "" otherwise
`ntp_seconds_since_last_sync`	seconds since last NTP sync if NTP is synchronized

NTP Force sync

A NTP force sync operation can be triggered which will attempt to achieve NTP sync within a specified timeout. Upon achieving sync, or when the timeout expires, the operation will respond with the following fields:

Field	Description
`success`	true if NTP sync was achieved within timeout, false otherwise
`message`	human-readable message
`status`	Time status as described here

Warning

`success` true means the DVL achieved NTP sync. To assert the DVL is synchronized to a specific NTP server, check the returned status.

Warning

The operation can cause abrupt jumps in system time, and is not recommended for use while reading DVL output.

Web GUI

On supported models there is a "System time configuration" section in the Configuration page.

TCP JSON API

Time configuration is available in TCP JSON API `json_v3.3` and up for supported models:

- DVL A50/A125 TCP JSON time API

Serial API

Time configuration is available in serial protocol `2.8.0` and up for supported models:

- DVL A50/A125 Serial time API

Settings and configuration

Use settings when the DVL installation, operating environment, or integration interface changes. Avoid changing settings during an operation unless you understand how the change affects velocity output, bottom lock, and dead reckoning.

Open settings

Open the DVL web GUI and go to the settings or configuration page for your model family:

- DVL A50/A125: preview the configuration interface on the demo site
- DVL A100/A250: preview the configuration interface on the demo site

Configuration can also be performed by protocol:

- DVL A50/A125 TCP JSON API configuration
- DVL A50/A125 serial configuration
- DVL A100/A250 TCP JSON API configuration
- DVL A100/A250 serial configuration

For DVL A50/A125, output protocol settings are also available in a separate preview of the output configuration on the demo site.

Operating configurations

Setting	Use
Speed of sound	Set the speed of sound for the water conditions.
Mounting rotation offset	Align DVL output with the vehicle frame. See Axes.
Acoustic enabled	Enable or disable acoustic pinging.
Dark mode	Disable the LED to avoid interference with cameras.
Range mode	Limit the altitude range used when searching for bottom lock. See Range mode.
Periodic cycling	Periodically validate that the DVL is locked to the real bottom. See Periodic cycling.
Water tracking	Use velocity relative to the water column instead of the bottom. See Water tracking.
Hardware trigger	Activate hardware triggering. Only for A100/A250.

> **Warning**
>
> Incorrect speed of sound, mounting rotation, range mode, tracking mode, or output protocol settings can cause the DVL data to appear incorrect, even when the hardware is functioning properly.

Periodic cycling

Periodic cycling is available on all DVL models.

When periodic cycling is enabled, the DVL checks every 10 seconds that the bottom it has locked onto is the actual bottom. The check verifies whether there are closer reflections that should be treated as the real bottom.

Periodic cycling is enabled by default. It is useful in reflective environments with many multipath reflections, such as tanks or pools.

The drawback is that some measurements are lost during the validation check. If you operate in open water with stable bottom conditions and need as many measurements as possible, consider turning periodic cycling off. If multipath or reflections are a concern, keeping periodic cycling on can improve bottom validation.

To change periodic cycling:

1. Open the settings or configuration page for your DVL model.
2. Find the periodic cycling setting.
3. Turn it on or off for the operating environment.
4. Save the configuration.

You can also configure periodic cycling with the `periodic_cycling_enabled` parameter over the TCP JSON API or serial protocol.

Output settings

Use output settings when an external system needs a specific serial protocol, TCP JSON API, or PD format.

1. Choose the protocol required by the receiving system.
2. Configure serial settings if the integration uses the serial interface.
3. Save the configuration.
4. Reconnect or restart the receiving system if required by the integration.

Electrical interfaces differ by model:

- DVL A50 and DVL A125 use a 3.3 V UART interface that is 5 V tolerant.
- DVL A100 and DVL A250 use RS232.

See DVL electrical connection and Integration before connecting external equipment.

Commands

Actions that can be performed on the DVL.

Command	Use
Reset dead reckoning	Reset the origin and attitude from the current DVL position and attitude. See Dead reckoning.
Calibrate gyro	Calibrate the gyro. Only for A50/A125.
Trigger ping	Tell the DVL to send a single ping. This has more latency than hardware triggering.

Network and maintenance

Setting	Use
Network configuration	Use DHCP or a static IP address. See Network Setup.
System time configuration	Configure NTP or set time manually.
Factory reset	Reset all configuration. Reboot is required.
Reboot	Restart the DVL.

Dead reckoning

Introduction

All our DVLs run a dead-reckoning algorithm that estimates the orientation and position of the DVL. This uses the velocities calculated by the DVL and its integrated IMU/AHRS.

Starting dead reckoning

Reset dead reckoning before using the local position estimate.

For DVL A50/A125:

1. Calibrate the gyroscope by pressing *More -> Calibrate gyro* in the web GUI while the DVL is stationary.
2. Click the **Reset** button in the dashboard, or send a reset command over the TCP JSON API or serial protocol.

For DVL A100/A250, gyro calibration is not needed. Reset commands over the TCP JSON API or serial protocol are included for backward compatibility and do not affect dead reckoning on DVL A100/A250.

Frame

The position, speed, roll, pitch, and yaw angles output by the dead-reckoning algorithm are relative to the frame defined at time zero: the time of the last reset, or startup if no reset has been performed. This frame is defined by the axes of the vehicle frame.

Speed and position

The speed and position of the DVL are calculated by integrating its acceleration, corrected by the velocity it calculates with acoustic signal processing. A Kalman filter is used; the roll, pitch, and yaw angles output by the DVL are also part of the input.

When the DVL cannot determine velocity, speed and position are predicted only from acceleration. This will result in large errors, indicated by an increase in the figure of merit (FOM), which is an estimated standard deviation of the position in the X-Y plane.

> **Note**
>
> When the DVL is powered on in the air, its position will drift significantly. This should be ignored, and dead reckoning should be started in water when ready.

The position of the DVL can be viewed in the web GUI dashboard or fetched by API:

- DVL A50/A125 web GUI dashboard

- DVL A100/A250 web GUI dashboard
- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API

Orientation

The orientation of the DVL is based on the accelerometer and gyroscope measurements from its IMU/AHRS. The orientation is represented by roll, pitch, and yaw angles, and can be viewed in the web GUI dashboard or fetched by API.

- Roll is a rotation around the X axis of the DVL
- Pitch is a rotation around the Y axis of the DVL
- Yaw is a rotation around the Z axis of the DVL

For orientation to function correctly, the DVL must be deployed in accordance with its axis conventions.

Note

A small tilt in the DVL mounting, though it will result in almost no position error with respect to the X and Y axes, may lead to a significant error with respect to the Z axis. This change will be a projection of the true horizontal position onto the Z axis. For example, when the DVL is mounted with 5° tilt and the horizontal displacement of the DVL is 100 m since the last reset, the total displayed distance in the X, Y plane will be 99.6 m, and the Z position will be ±9 m.

When dead reckoning is reset, the roll, pitch, and yaw angles are set to zero.

Yaw drift

Note

For DVL A50/A125, yaw drift can be decreased to 0.1-0.3° per minute by calibration of the gyroscope. For DVL A100/A250, gyro calibration is not needed.

Software updates

We recommend running the latest DVL software where possible. Software updates are done from the web GUI **About** page. The latest software can be obtained automatically or manually.

> **Note**
>
> Software updates are performed only through the web GUI over the Ethernet (network) connection. The serial interface cannot be used to update the DVL.

The release notes for each release are found here:

- DVL A50/A125 changelog
- DVL A100/A250 changelog

> **Warning**
>
> During the software update, the thermal protection feature is turned off. To prevent the DVL from overheating, keep the DVL submerged in water throughout the update process.
>
> If the DVL is switched off during an update, it might be permanently damaged.

Automatic software update

The DVL web GUI automatically checks and indicates if a new software version is available from the **About** page.

- Connect the DVL over Ethernet to a network connected to the internet. If you use a personal computer, it may be necessary to make a network bridge between a network interface with internet access, such as Wi-Fi, and the Ethernet interface connected to the DVL.
- Open the DVL web GUI in a web browser.
- Go to the **About** page.
- If a new version is available, start the software update from the **About** page.

Offline software update

If it is not possible to connect the DVL to the internet, use an offline software update.

- Find the current version and chip ID of the DVL on the **About** page in the web GUI. Integrators can also read them with the `get_version_info` command over the TCP JSON API, see DVL A50/A125 or DVL A100/A250.
- Manually download an update package (`.wlup`) from the update server using the chip ID.
- Verify whether the downloaded version is newer than the currently running version.
- Set the system time as described below.
- Open the DVL web GUI in a web browser.

- Go to the **About** page.
- Select **Manual upload** and then upload the downloaded `.wlup` file.

Failure during update (set system time)

When performing an offline software update, the following error might occur: `Failed: Error upgrading: resize: non-zero exit code: exit status 1`. This error is resolved by setting the system time before performing an update.

- Go to the DVL web GUI and click **Configuration** or **Settings** in the left menu.
- Open the **Advanced configuration** menu.
- Scroll to **System time configuration**.
- Select **Manual time** in **System time source** and click **Set manual time**.
- Click **About** in the left menu and click **Software upgrade**.
- Perform the update again.

DVL A50/A125 web GUI

This page applies to DVL A50 and DVL A125.

The DVL A50/A125 web GUI can be used to check live status, configure settings, select outputs, collect diagnostic logs, and view software and device information.

Open the web GUI

1. Connect power and Ethernet to the DVL.
2. Make sure the DVL is submerged before normal operation.
3. Open a web browser and go to the DVL address.
4. See Network Setup if the web GUI does not open.

Demo GUI

Use the public DVL A50/A125 demo GUI to explore the interface without connecting to a DVL.

The demo runs on simulated data. It does not replace testing with your own DVL in water. Hardware status, bottom lock, acoustic conditions, and output integrations must still be verified on the real installation.

Dashboard

Use the dashboard to view the current DVL status, velocity data, bottom lock state, altitude, dead-reckoning information, and other live information.

Open the public dashboard demo page.

Verify bottom lock

1. Confirm that the DVL is submerged and the transducers have a clear line of sight to the bottom.
2. Open the dashboard.
3. Check that acoustics are enabled.
4. Check that velocity is valid.
5. Check that altitude is stable and reasonable for the test environment.
6. Check the figure of merit (FOM) or uncertainty indicator if shown.

> **Note**
>
> Figure of merit is a confidence indicator for the velocity estimate. A high value means the velocity measurement is less certain.

If velocity is invalid or bottom lock is unreliable, see How to diagnose.

View velocity and navigation

1. Open the dashboard.
2. Check that velocity is valid before using the values.
3. Review speed, altitude, ping rate, and FOM.
4. Use the horizontal and vertical velocity views to confirm the movement direction.
5. Use the map view only after checking the dead reckoning limitations.

See Dead reckoning for the reference frame and limitations.

> **Warning**
>
> Dead reckoning will drift over time. Do not use the map view as an absolute position reference unless your system also provides an external position reference.

Outputs

Use the outputs page to configure which data outputs are enabled and how data is sent from the DVL.

Open the public outputs demo page.

1. Open the outputs page.
2. Select the output protocol required by your integration.
3. Configure serial settings if you use the UART interface.
4. Save the configuration.
5. Reconnect or restart the receiving system if required by the integration.

See Settings and configuration, Integration, DVL A50/A125 TCP JSON API, and DVL A50/A125 serial protocol.

Configuration

Use the configuration page to change DVL settings such as speed of sound, range mode, mounting rotation, water tracking, network settings, and maintenance actions.

Open the public configuration demo page.

See Settings and configuration, Axes, Range mode, and Water tracking.

Diagnostics

Use the diagnostics page to view live diagnostic information while the DVL is operating. This can help you see whether status, signal quality, transducer distances, or validity changes when the vehicle moves, thrusters run, or the environment changes.

Open the public diagnostics demo page.

See How to diagnose for symptom-based checks.

Diagnostic log

The DVL A50/A125 web GUI has a separate diagnostic log page.

Use the diagnostic log page to collect a diagnostic log when troubleshooting or when requested by Water Linked support.

Open the public diagnostic log demo page.

See Diagnostic log for collection guidance and the support package checklist before contacting Water Linked support.

About

Use the About page to view software version, chip ID, product information, release notes, and software update actions if available.

Open the public About demo page.

Software updates are covered in Software updates.

DVL A100/A250 web GUI

This page applies to DVL A100 and DVL A250.

The DVL A100/A250 web GUI can be used to check live status, configure settings, view diagnostics, collect diagnostic logs, and view software and device information.

Open the web GUI

1. Connect power and Ethernet to the DVL.
2. Make sure the DVL is submerged before normal operation.
3. Open a web browser and go to the DVL address:
   - `http://waterlinked-dvl` when mDNS is available.
   - `http://192.168.194.95` when using the fallback IP.
4. See Network Setup if the web GUI does not open.

Demo GUI

Use the public DVL A100/A250 demo GUI to explore the interface without connecting to a DVL.

The demo runs on simulated data. It does not replace testing with your own DVL in water. Hardware status, bottom lock, acoustic conditions, and output integrations must still be verified on the real installation.

Dashboard

Use the dashboard to view the current DVL status, velocity data, bottom lock state, dead-reckoning information, and other live information.

Open the public dashboard demo page.

Verify bottom lock

1. Confirm that the DVL is submerged and the transducers have a clear line of sight to the bottom.
2. Open the dashboard.
3. Check that acoustics are enabled.
4. Check that velocity is valid.
5. Check that altitude and transducer distances are stable and reasonable for the test environment.
6. If you use the data stream for integration, compare the web GUI status with the TCP JSON API or serial output.

> **Note**
>
> A DVL can be connected and healthy without having bottom lock. Bottom lock also depends on altitude, bottom conditions, acoustic noise, range mode, and vehicle motion.

If velocity is invalid or bottom lock is unreliable, see How to diagnose.

View velocity and navigation

1. Open the dashboard.
2. Check that velocity is valid before using the values.
3. Review horizontal velocity, vertical velocity, altitude, and transducer validity.
4. Use figure of merit or uncertainty indicators as a confidence check when they are available.
5. Use the local map/dead-reckoning view only after checking the dead reckoning limitations.

See Dead reckoning for the reference frame and limitations.

> **Warning**
>
> Dead reckoning will drift over time. Do not use the map view as an absolute position reference unless your system also provides an external position reference.

Configuration

Use the configuration page to change DVL settings such as speed of sound, range mode, mounting rotation, water tracking, network settings, and maintenance actions.

Open the public configuration demo page.

See Settings and configuration, Axes, Range mode, and Water tracking.

Gyro calibration is not needed for DVL A100/A250.

Outputs The DVL A100/A250 web GUI does not have a separate Outputs page. Use the configuration page and the protocol references for output configuration until the final GUI labels are confirmed. See [Settings and configuration](configuration.md#output-settings), [Integration](integration.md), [DVL A100/A250 TCP JSON API](dvl-a250_a100-json-protocol.md), and [DVL A100/A250 serial protocol](dvl-a250_a100-serial-protocol.md). TODO: Confirm final DVL A100/A250 web GUI label and workflow for output configuration.

Diagnostics and diagnostic log

Use the diagnostics page to view live diagnostic information and collect a diagnostic log when troubleshooting or when requested by Water Linked support.

Open the public diagnostics demo page.

See Diagnostic log for collection guidance, How to diagnose for symptom-based checks, and the support package checklist when preparing a support request.

About

Use the About page to view software version, chip ID, product information, and software update actions if available.

Open the public About demo page.

Software updates are covered in Software updates.

Integration

This page helps you choose an interface for integrating our DVLs into a vehicle, boat, or control system.

Choose an interface

Use case	Recommended interface
First setup, diagnostics, configuration, and testing	Ethernet and the web GUI
Custom software integration over Ethernet	TCP JSON API
Direct vehicle integration over serial	Water Linked serial protocol
Existing systems with PD0, PD4, or PD6 support	PD formats
Multiple acoustic instruments	Triggering and synchronization

TCP JSON API

Use the TCP JSON API when you want a network data stream, command interface, and easy access from custom software. In customer-facing documentation, Water Linked API refers to this TCP JSON API.

The DVL TCP server is available on port `16171`. See Network Setup.

In our measurements, TCP JSON message latency was typically around 4 ms average, with 2 ms standard deviation and an observed maximum of 13 ms. This is not a hard real-time guarantee; actual latency can depend on network setup, host system, load, and integration method.

Protocol references:

- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API

Serial Water Linked protocol

Use the Water Linked serial protocol when the vehicle controller expects a direct serial data stream.

Electrical interfaces differ by model:

- DVL A50 and DVL A125 use the same UART electrical interface. See DVL A50/A125 serial interface.
- DVL A100 and DVL A250 use RS232. See DVL A100/A250 serial interface.

Protocol references:

- DVL A50/A125 serial protocol
- DVL A100/A250 serial protocol

PD formats

Use PD formats only when the receiving system already supports PD0, PD4, or PD6. This can reduce integration work when replacing or adding a DVL to an existing system.

PD4 and PD6 are supported over TCP and serial for all DVL models. PD0 is planned for DVL A100 and DVL A250, but is not available yet.

Protocol references:

- PD formats
- PD6 format
- PD4 format
- PD0 status

Triggering and synchronization

Triggering can be useful when the DVL operates near other acoustic instruments and you want to avoid simultaneous acoustic transmissions.

Triggering and synchronization are handled through protocol or electrical interfaces, not through the web GUI.

TCP JSON API or serial command trigger

The DVL can queue externally requested pings when acoustic pinging is controlled by command.

See:

- DVL A50/A125 TCP JSON API trigger ping
- DVL A50/A125 serial trigger ping
- DVL A100/A250 TCP JSON API trigger ping
- DVL A100/A250 serial trigger ping

RX-line trigger

For DVL A100 and DVL A250, sending a NULL character over the RS232 RX line triggers a pulse on the falling edge.

TODO: Engineering review: confirm timing requirements and valid baud/configuration assumptions for A100/A250 RX-line triggering.

Diagnostics during integration

If the integration does not behave as expected, collect a diagnostic log from the web GUI and include the integration interface in the support request.

See:

- DVL diagnostic log
- How to diagnose

DVL A50/A125 TCP JSON API

The DVL TCP JSON API sends JSON messages over TCP on port 16171. This page applies to DVL A50 and DVL A125.

Terminology

- DVL - Doppler Velocity Log - Uses hydro-acoustic beams to measure the velocity at which the DVL is moving across a surface (typically an unmoving one such as the sea bottom), and the distance to this surface.
- ACK - Acknowledgement. The command issued was successful.
- NAK - Negative acknowledgement. The command issued failed.
- Ping - A pulse of sound sent by the DVL
- Time of validity - Timestamp of the surface reflection ('center of ping')
- Time of transmission - Timestamp taken directly before sending data over the serial or TCP protocols. The difference between time of transmission and time of validity includes both the time for the acoustic wave to travel from the surface from which it was reflected back to the DVL, and the decoding and processing of this signal internally in the DVL.

Version

This document describes TCP JSON API version `json_v3.3` (major.minor):

- MAJOR version increments represent incompatible API changes
- MINOR version increments represent additional backwards-compatible functionality

Version history overview

Software release	Ethernet protocol version	Main protocol improvements
2.7.2	json_v3.3	Add TCP JSON time API. Add get_version_info command
2.7.1	json_v3.2	Add water tracking mode
2.6.1	json_v3.1	Serial baud rate configurable. Add PD4 format support in serial 'wcp' command. Some serial protocol names changed.
2.5.2	json_v3.1	Add PD4 format support (experimental)
2.4.4	json_v3.1	Change gyro calibration to store persistently. Note: gyro calibration commands now take up to 15 seconds.

Software release	Ethernet protocol version	Main protocol improvements
2.4.0	json_v3.1	Add ability to trigger pings (TCP JSON API/serial), add configuration for periodic cycling (TCP JSON API/serial)
2.2.1	json_v3	Add serial output protocol configuration, range mode configuration and calibrate gyro command, Fix missing line ending in configuration (TCP JSON API), fix dark mode enabled naming inconsistency (TCP JSON API), change speed of sound and mounting rotation offset from integer to float
2.1.0	json_v3	Add configuration, add time_of_validity/time_of_transmission, add covariance (TCP JSON API)
2.0.8	json_v2	Add position estimation, Add output of orientation angles
1.6.0	-	Initial (velocity only)

TCP JSON API

Overview

The DVL supports sending velocity, transducer, and position updates using the Transmission Control Protocol (TCP). The DVL runs a TCP server on port 16171.

The format of each packet is JSON.

In our measurements, TCP JSON message latency was typically around 4 ms average, with 2 ms standard deviation and an observed maximum of 13 ms. This is not a hard real-time guarantee; actual latency can depend on network setup, host system, load, and integration method. See Integration.

PD formats over TCP

PD4 and PD6 are also supported over TCP for all DVL models. See PD formats for the canonical PD format descriptions.

Velocity-and-transducer report

A velocity-and-transducer report is sent for each velocity calculation of the DVL. The rate depends on the altitude of the DVL (distance to the sea bottom or other reflecting surface), but will be in the range 2-15 Hz.

The X, Y, and Z axes are with respect to body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a 'mounting rotation offset', from the forward axis of the vehicle.

The messages are delimited by newline.

Variable	Description
time	Milliseconds since last velocity report (ms)
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this (entries in (m/s)^2)
altitude	Distance to the reflecting surface along the Z axis (m)
transducers	Is a list containing information from each transducer: [`id`, velocity (m/s), distance (m), rssi (dBm), nsd (dBm), `beam_valid` (True/False)]
velocity_valid	If true, the DVL has a lock on the reflecting surface, and the altitude and velocities are valid (True/False)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
format	Format type and version for this report: `json_v3.3`
type	Report type: `velocity` or `velocity_water` for water tracking

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

Example of TCP report (indented for legibility)


```
{
  "time": 106.3935775756836,
  "vx": -3.713480691658333e-05,
  "vy": 5.703703573090024e-05,
  "vz": 2.4990416932269e-05,
  "fom": 0.00016016385052353144,
  "covariance": [
```

```
      [
        2.4471841442164077e-08,
        -3.3937477272871774e-09,
        -1.6659699175747278e-09
      ],
      [
        -3.3937477272871774e-09,
        1.4654466085062268e-08,
        4.0409570134514183e-10
      ],
      [
        -1.6659699175747278e-09,
        4.0409570134514183e-10,
        1.5971971523143225e-09
      ]
    ],
    "altitude": 0.4949815273284912,
    "transducers": [
      {
        "id": 0,
        "velocity": 0.00010825289791682735,
        "distance": 0.5568000078201294,
        "rssi": -30.494251251220703,
        "nsd": -88.73271179199219,
        "beam_valid": true
      },
      {
        "id": 1,
        "velocity": -1.4719001228513662e-05,
        "distance": 0.5663999915122986,
        "rssi": -31.095735549926758,
        "nsd": -89.5116958618164,
        "beam_valid": true
      },
      {
        "id": 2,
        "velocity": 2.7863150535267778e-05,
        "distance": 0.537600040435791,
        "rssi": -27.180519104003906,
        "nsd": -96.98075103759766,
        "beam_valid": true
      },
      {
        "id": 3,
        "velocity": 1.9419496311456896e-05,
        "distance": 0.5472000241279602,
        "rssi": -28.006759643554688,
        "nsd": -88.32147216796875,
        "beam_valid": true
      }
    ],
    "velocity_valid": true,
    "status": 0,
    "format": "json_v3.3",
    "type": "velocity",
    "time_of_validity": 1638191471563017,
    "time_of_transmission": 1638191471752336
}
```

Dead reckoning report

A dead reckoning report outputs the current speed, position, and orientation of the DVL as calculated by dead reckoning, with respect to a frame defined by the axes of the DVL's body frame, or vehicle frame if a mounting rotation offset is set, at the start of the dead reckoning run. The expected update rate is 5 Hz.

Variable	Description
ts	Time stamp of report (Unix timestamp in seconds)
x	Distance in X direction (m)
y	Distance in Y direction (m)
z	Distance in downward direction (m)
std	Standard deviation (figure of merit) for position (m)
roll	Rotation around X axis (degrees)
pitch	Rotation around Y axis (degrees)
yaw	Rotation around Z axis, i.e. heading (degrees)
type	Report type: `position_local`
status	Reports if there are any issues with the DVL (0 if no errors, 1 otherwise)
format	Format type and version for this report: `json_v3`

Example of a dead reckoning report.


```
{
  "ts": 49056.809,
  "x": 12.43563613697886467,
  "y": 64.617631152402609587,
  "z": 1.767641898933798075,
  "std": 0.001959984190762043,
  "roll": 0.6173566579818726,
  "pitch": 0.6173566579818726,
  "yaw": 0.6173566579818726,
  "type": "position_local",
  "status": 0,
  "format": "json_v3.3"
}
```


Reset dead reckoning

Dead reckoning can be reset by issuing the `reset_dead_reckoning` command:

```
{"command": "reset_dead_reckoning"}
```

If the request is successfully received the response will have 'success' set to 'true'. The dead reckoning will have a delay of approximately 50ms until the positioning values being zeroed out. If the response is unsuccessful, the 'success' will be 'false' and a non-empty describing text will be returned in 'error_message'.

```
{
  "response_to":"reset_dead_reckoning",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v3.3",
  "type": "response"
}
```

Calibrate gyro

The gyro can be calibrated by issuing the `calibrate_gyro` command:

```
{"command":"calibrate_gyro"}
```

The response will be as follows if the calibration is successful. If unsuccessful, `success` will be `false`, and a non-empty `error_message` will be provided.

```
{
  "response_to": "calibrate_gyro",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v3.3",
  "type": "response"
}
```

Trigger ping

In setups where multiple acoustic sensors are used it can be useful to control the pinging of each acoustic sensor individually. By setting the configuration `acoustic_enabled = false` the pinging of the DVL can be externally controlled. Up to 15 external trigger commands can be queued by issuing the `trigger_ping` command. The DVL will execute each ping in quick succession until no more commands are in the queue.

See Integration for guidance on using triggering with other acoustic instruments.

```
{"command":"trigger_ping"}
```

The response will be as follows if the command is accepted. If the queue is full, `success` will be `false`, and a non-empty `error_message` will be provided.

```
{
  "response_to": "trigger_ping",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v3.3",
```

```
  "type": "response"
}
```

Get version info

Get version info by issuing the `get_version_info` command.

```
{"command":"get_version_info"}
```

A successful response looks like this:

```
{
  "response_to": "get_version_info",
  "success": true,
  "error_message": "",
  "result": {
    "chipid": "0xf1440c09140b04",
    "hardware_revision": 4,
    "product_id": 21035,
    "product_name": "DVL A50",
    "variant": "standard",
    "version_short": "2.7.2",
    "version": "2.7.2 (v2.7.1b-10-g44e26c7.2026-05-29T12:51:02.882778)",
    "is_ready": true
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Time

Setting NTP configuration

Set NTP configuration by issuing the `set_time_ntp` command with a `ntp_address` parameter. Set `ntp_address` either to the address of a NTP server, or as "auto".

```
{
  "command": "set_time_ntp",
  "parameters": {
    "ntp_address": "192.168.0.10"
  }
}
```

A successful response looks like this:

```
{
  "response_to": "set_time_ntp",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v3.3",
  "type": "response"
}
```

Getting NTP configuration

Get NTP configuration by issuing the `get_time_ntp` command.

```
{
  "command": "get_time_ntp"
}
```

A successful response looks like this:

```
{
  "response_to": "get_time_ntp",
  "success": true,
  "error_message": "",
  "result": {
    "ntp_address": "auto"
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Setting manual time

Set time manually by issuing the `set_time_manual` command with the `now` parameter as a RFC3339 timestamp string.

For more information see Setting manual time.

```
{
  "command": "set_time_manual",
  "parameters": {
    "now": "2026-04-20T09:04:13Z"
  }
}
```

A successful response looks like this:

```
{
  "response_to": "set_time_manual",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v3.3",
  "type": "response"
}
```

In the event the DVL is already synchronized via NTP, the response will look like this:

```
{
  "response_to": "set_time_manual",
  "success": false,
  "error_message": "Manual time not allowed when NTP is synchronized",
  "result": null,
  "format": "json_v3.3",
  "type": "response"
}
```

Getting time status

Get time status by issuing the `get_time_status` command.

```
{
  "command": "get_time_status"
}
```

The response contains the following fields:

Field	Description
`system_time`	system time in RFC3339
`ntp_synced`	true if synchronized to NTP source, false otherwise
`ntp_synced_to`	address the sonar is synchronized to if NTP is synchronized, "" otherwise
`ntp_seconds_since_last_sync`	seconds since last NTP sync, or null if not synchronized

Here is an example response where the DVL is not synchornized with NTP:

```
{
  "response_to": "get_time_status",
  "success": true,
  "error_message": "",
  "result": {
    "system_time": "2026-04-20T08:48:25Z",
    "ntp_synced": false,
    "ntp_synced_to": "",
    "ntp_seconds_since_last_sync": null
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Here is an example response where the DVL has achieved sync with NTP:

```
{
  "response_to": "get_time_status",
  "success": true,
  "error_message": "",
  "result": {
    "system_time": "2026-04-20T08:48:25Z",
    "ntp_synced": true,
    "ntp_synced_to": "192.168.0.10",
    "ntp_seconds_since_last_sync": 5
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Forcing NTP sync

A forced NTP sync can be triggered by issuing the `force_sync_ntp` command with the `timeout_seconds` as an integer timeout in seconds.

```
{
  "command": "force_sync_ntp",
  "parameters": {
    "timeout_seconds": 10
  }
}
```

This will trigger a forced NTP sync in the DVL. The DVL will wait up to `timeout_seconds` for sync to be achieved, finally responding with the following fields:

Field	Description
`success`	true if NTP sync was achieved within timeout, false otherwise. Important: callers that want to achieve sync with a specific address should check the returned status
`message`	human-readable message
`status`	time status response body

Here is an example response where the DVL achieved NTP sync:

```
{
  "response_to": "force_sync_ntp",
  "success": true,
  "error_message": "",
  "result": {
    "success": true,
    "message": "NTP force-sync successful",
    "status": {
      "system_time": "2026-04-20T09:04:13Z",
      "ntp_synced": true,
      "ntp_synced_to": "192.168.0.10",
      "ntp_seconds_since_last_sync": 0
    }
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Here is an example response where the DVL did not achieve NTP sync:

```
{
  "response_to": "force_sync_ntp",
  "success": true,
  "error_message": "",
  "result": {
    "success": false,
    "message": "NTP force-sync not successful within timeout",
    "status": {
      "system_time": "2026-04-20T09:04:13Z",
```

```
      "ntp_synced": false,
      "ntp_synced_to": "",
      "ntp_seconds_since_last_sync": null
    }
  },
  "format": "json_v3.3",
  "type": "response"
}
```

Configuration over TCP JSON API

Configuration parameters

Variable	Description
speed_of_sound	Speed of sound (1000-2000 m/s). Integer
mounting_rotation_offset	See the definition of the vehicle frame of the DVL. Typically 0, but can be set to be non-zero if the forward axis of the DVL is not aligned with the forward axis of a vehicle on which it is mounted (0-360 degrees). Integer
acoustic_enabled	`true` for normal operation of the DVL, `false` when the sending of acoustic waves from the DVL is disabled (e.g. to save power or slow down its heating up in air)
dark_mode_enabled	`false` when the LED operates as normal, `true` for no blinking of the LED (e.g. if the LED is interfering with a camera)
range_mode	`auto` when operating as normal, otherwise see range mode configuration or activate water tracking
periodic_cycling_enabled	`true` to enable periodic cycling, `false` to disable it

Fetching current configuration

The current configuration of the DVL can be obtained by issuing the `get_config` command:

```
{"command": "get_config"}
```

If the configuration is successfully fetched, the response will be in the following format. If not, `success` will be false, a non-empty `error_message` string will be provided, and `result` will be `null`.

```
{
  "response_to":"get_config",
  "success":true,
  "error_message":"",
  "result":{
    "speed_of_sound":1475.00,
    "acoustic_enabled":true,
    "dark_mode_enabled":false,
    "mounting_rotation_offset":20.00,
```

```
    "range_mode":"auto",
    "periodic_cycling_enabled":true
  },
  "format":"json_v3.3",
  "type":"response"
}
```

Setting configuration parameters

Setting of configuration parameters can be carried out by issuing a `set_config` in the following format, including those parameters which are to be set:

```
{"command":"set_config","parameters":{"speed_of_sound":1480}}
```

If the parameters are successfully set, the response will be in the following format. If not, `success` will be false, and a non-empty `error_message` string will be provided.

```
{
  "response_to": "set_config",
  "success": true,
  "error_message": "",
  "result" :null,
  "format": "json_v3.3",
  "type": "response"
}
```

Activating water tracking

Water tracking is enabled by setting `range_mode` to `wt`:

```
{"command":"set_config","parameters":{"range_mode":"wt"}}
```

Deactivate water tracking

Water tracking is deactivated by setting `range_mode` to `auto`:

```
{"command":"set_config","parameters":{"range_mode":"auto"}}
```

DVL A100/A250 TCP JSON API

The DVL TCP JSON API sends JSON messages over TCP on port 16171. This page applies to DVL A100 and DVL A250.

Terminology

- DVL - Doppler Velocity Log - Uses hydro-acoustic beams to measure the velocity at which the DVL is moving across a surface (typically an unmoving one such as the sea bottom), and the distance to this surface.
- ACK - Acknowledgement. The command issued was successful.
- NAK - Negative acknowledgement. The command issued failed.
- Ping - A pulse of sound sent by the DVL
- Time of validity - Timestamp of the surface reflection ('center of ping')
- Time of transmission - Timestamp taken directly before sending data over the serial or TCP protocols. The difference between time of transmission and time of validity includes both the time for the acoustic wave to travel from the surface from which it was reflected back to the DVL, and the decoding and processing of this signal internally in the DVL.

Version

This document describes TCP JSON API `json_v4.1` (major.minor):

- MAJOR version increments represent incompatible API changes
- MINOR version increments represent additional backwards-compatible functionality

Version history overview

Software release	Ethernet protocol version	Main protocol improvements
3.3.0	json_v4.1	Add TCP JSON time API. Add get_version_info command. Add water tracking mode.
3.2.0	json_v4.0	Initial public release.

TCP JSON API

Overview

The DVL supports sending velocity, transducer, and position updates using the Transmission Control Protocol (TCP). The DVL runs a TCP server on port 16171.

The format of each packet is JSON.

In our measurements, TCP JSON message latency was typically around 4 ms average, with 2 ms standard deviation and an observed maximum of 13 ms. This is not a hard real-time guarantee; actual latency can depend on network setup, host system, load, and integration method. See Integration.

PD formats over TCP

PD4 and PD6 are also supported over TCP for all DVL models. PD0 is planned for DVL A100 and DVL A250 over TCP and serial, but is not available yet. See PD formats for the canonical PD format descriptions.

Velocity-and-transducer report

A velocity-and-transducer report is sent for each velocity calculation of the DVL. The rate depends on the model, altitude, and configuration. See the model page and Range mode for product-specific operating limits.

The X, Y, and Z axes are with respect to the body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a mounting rotation offset, from the forward axis of the vehicle.

The messages are delimited by newline.

Variable	Description
time	Milliseconds since last velocity report (ms)
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this (entries in (m/s)^2)
altitude	Distance to the reflecting surface along the Z axis (m)
transducers	Is a list containing information from each transducer: [`id`, velocity (m/s), distance (m), rssi (dBm), nsd (dBm), `beam_valid` (True/False)]
velocity_valid	If true, the DVL has a lock on the reflecting surface, and the altitude and velocities are valid (True/False)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)

Variable	Description
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
format	Format type and version for this report: `json_v4.1`
type	Report type: `velocity` or `velocity_water` for water tracking

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

Example of TCP report (indented for legibility)


```
{
  "time": 106.3935775756836,
  "vx": -3.713480691658333e-05,
  "vy": 5.703703573090024e-05,
  "vz": 2.4990416932269e-05,
  "fom": 0.00016016385052353144,
  "covariance": [
    [
      2.4471841442164077e-08,
      -3.3937477272871774e-09,
      -1.6659699175747278e-09
    ],
    [
      -3.3937477272871774e-09,
      1.4654466085062268e-08,
      4.0409570134514183e-10
    ],
    [
      -1.6659699175747278e-09,
      4.0409570134514183e-10,
      1.5971971523143225e-09
    ]
  ],
  "altitude": 0.4949815273284912,
  "transducers": [
    {
      "id": 0,
      "velocity": 0.00010825289791682735,
      "distance": 0.5568000078201294,
      "rssi": -30.494251251220703,
      "nsd": -88.73271179199219,
      "beam_valid": true
    },
    {
      "id": 1,
      "velocity": -1.4719001228513662e-05,
      "distance": 0.5663999915122986,
      "rssi": -31.095735549926758,
      "nsd": -89.5116958618164,
      "beam_valid": true
    },
```

```
    {
      "id": 2,
      "velocity": 2.7863150535267778e-05,
      "distance": 0.537600040435791,
      "rssi": -27.180519104003906,
      "nsd": -96.98075103759766,
      "beam_valid": true
    },
    {
      "id": 3,
      "velocity": 1.9419496311456896e-05,
      "distance": 0.5472000241279602,
      "rssi": -28.006759643554688,
      "nsd": -88.32147216796875,
      "beam_valid": true
    }
  ],
  "velocity_valid": true,
  "status": 0,
  "format": "json_v4.1",
  "type": "velocity",
  "time_of_validity": 1638191471563017,
  "time_of_transmission": 1638191471752336
}
```


Dead reckoning report

A dead reckoning report outputs the current speed, position, and orientation of the DVL as calculated by dead reckoning, with respect to a frame defined by the axes of the DVL's body frame, or vehicle frame if a mounting rotation offset is set, at the start of the dead reckoning run. The expected update rate is 5 Hz.

Variable	Description
ts	Time stamp of report (Unix timestamp in seconds)
x	Distance in X direction (m)
y	Distance in Y direction (m)
z	Distance in downward direction (m)
std	Standard deviation (figure of merit) for position (m)
roll	Rotation around X axis (degrees)
pitch	Rotation around Y axis (degrees)
yaw	Rotation around Z axis, i.e. heading (degrees)
type	Report type: `position_local`
status	Reports if there are any issues with the DVL (0 if no errors, 1 otherwise)

Variable	Description
format	Format type and version for this report: `json_v4.1`

Example of a dead reckoning report.


```
{
  "ts": 49056.809,
  "x": 12.43563613697886467,
  "y": 64.617631152402609587,
  "z": 1.767641898933798075,
  "std": 0.001959984190762043,
  "roll": 0.6173566579818726,
  "pitch": 0.6173566579818726,
  "yaw": 0.6173566579818726,
  "type": "position_local",
  "status": 0,
  "format": "json_v4.1"
}
```


Reset dead reckoning

Dead reckoning can be reset by issuing the `reset_dead_reckoning` command:

```
{"command": "reset_dead_reckoning"}
```

If the request is successfully received the response will have 'success' set to 'true'. The dead reckoning will have a delay of approximately 50ms until the positioning values being zeroed out. If the response is unsuccessful, the 'success' will be 'false' and a non-empty describing text will be returned in 'error_message'.


```
{
  "response_to":"reset_dead_reckoning",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v4.1",
  "type": "response"
}
```


Calibrate gyro

Gyro calibration is not needed as a normal setup step for DVL A100/A250.

The gyro can be calibrated by issuing the `calibrate_gyro` command:

```
{"command":"calibrate_gyro"}
```

The response will be as follows if the calibration is successful. If unsuccessful, `success` will be `false`, and a non-empty `error_message` will be provided.


```
{
  "response_to": "calibrate_gyro",
```

```
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v4.1",
  "type": "response"
}
```


Trigger ping

In setups where multiple acoustic sensors are used it can be useful to control the pinging of each acoustic sensor individually. By setting the configuration `acoustic_enabled = false` the pinging of the DVL can be externally controlled. Up to 15 external trigger commands can be queued by issuing the `trigger_ping` command. The DVL will execute each ping in quick succession until no more commands are in the queue.

See Integration for guidance on using triggering with other acoustic instruments.


```
{"command":"trigger_ping"}
```


The response will be as follows if the command is accepted. If the queue is full, `success` will be `false`, and a non-empty `error_message` will be provided.


```
{
  "response_to": "trigger_ping",
  "success": true,
  "error_message": "",
  "result": null,
  "format": "json_v4.1",
  "type": "response"
}
```


Get version info

Get version info by issuing the `get_version_info` command.


```
{"command":"get_version_info"}
```


A successful response looks like this:


```
{
  "response_to": "get_version_info",
  "success": true,
  "error_message": "",
  "result": {
    "chipid": "0x123456789",
    "hardware_revision": 1,
    "product_id": 21062,
    "product_name": "DVL A100",
    "variant": "standard",
    "version_short": "3.4.5",
    "version": "3.4.5 (v3.4.5-rc6.2026-03-27T10:49:10.704464)",
    "is_ready": true
  },
  "format": "json_v4.1",
  "type": "response"
}
```

Variable	Description
chipid	Chip ID of the DVL. Also shown on the About page in the web GUI, and used to download offline software updates
hardware_revision	Hardware revision of the DVL
product_id	Water Linked product number: 21062 for DVL A100, 21061 for DVL A250
product_name	Product name: `DVL A100` or `DVL A250`
variant	Product variant
version_short	Software version
version	Software version including build information
is_ready	`true` when the DVL is fully operational

Configuration over TCP JSON API

Configuration parameters

Variable	Description
speed_of_sound	Speed of sound (1000-2000 m/s). Integer
mounting_rotation_offset	See the definition of the vehicle frame of the DVL. Typically 0, but can be set to be non-zero if the forward axis of the DVL is not aligned with the forward axis of a vehicle on which it is mounted (0-360 degrees). Integer
acoustic_enabled	`true` for normal operation of the DVL, `false` when the sending of acoustic waves from the DVL is disabled (e.g. to save power or slow down its heating up in air)
dark_mode_enabled	`false` when the LED operates as normal, `true` for no blinking of the LED (e.g. if the LED is interfering with a camera)
range_mode	`auto` when operating as normal, otherwise see range mode configuration or activate water tracking
periodic_cycling_enabled	`true` to enable periodic cycling, `false` to disable it
hardware_trigger_enabled	`true` to enable hardware triggering, `false` to disable

Fetching current configuration

The current configuration of the DVL can be obtained by issuing the `get_config` command:

```
{"command": "get_config"}
```

If the configuration is successfully fetched, the response will be in the following format. If not, `success` will be false, a non-empty `error_message` string will be provided, and `result` will be `null`.

```
{
  "response_to":"get_config",
  "success":true,
  "error_message":"",
  "result":{
    "speed_of_sound":1475.00,
    "acoustic_enabled":true,
    "dark_mode_enabled":false,
    "mounting_rotation_offset":20.00,
    "range_mode":"auto",
    "periodic_cycling_enabled":true
  },
  "format":"json_v4.1",
  "type":"response"
}
```

Setting configuration parameters

Setting of configuration parameters can be carried out by issuing a `set_config` in the following format, including those parameters which are to be set:

```
{"command":"set_config","parameters":{"speed_of_sound":1480}}
```

If the parameters are successfully set, the response will be in the following format. If not, `success` will be false, and a non-empty `error_message` string will be provided.

```
{
  "response_to": "set_config",
  "success": true,
  "error_message": "",
  "result" :null,
  "format": "json_v4.1",
  "type": "response"
}
```

Activating water tracking

Water tracking is enabled by setting `range_mode` to `wt`:

```
{"command":"set_config","parameters":{"range_mode":"wt"}}
```

Deactivate water tracking

Water tracking is deactivated by setting `range_mode` to `auto`:

```
{"command":"set_config","parameters":{"range_mode":"auto"}}
```

DVL A50/A125 serial protocol

Describes the Water Linked DVL serial protocol for DVL A50 and DVL A125. For PD4 and PD6 output formats, see PD formats.

Terminology

- DVL - Doppler Velocity Log - Uses hydro-acoustic beams to measure the velocity at which the DVL is moving across a surface (typically an unmoving one such as the sea bottom), and the distance to this surface.
- ACK - Acknowledgement. The command issued was successful.
- NAK - Negative acknowledgement. The command issued failed.
- Ping - A pulse of sound sent by the DVL
- Time of validity - Timestamp of the surface reflection ('center of ping')
- Time of transmission - Timestamp taken directly before sending data over the serial or TCP protocols. The difference between time of transmission and time of validity includes both the time for the acoustic wave to travel from the surface from which it was reflected back to the DVL, and the decoding and processing of this signal internally in the DVL.

Version

This document describes serial protocol versions `2.6.x` and `2.7.x` (major.minor.patch):

- MAJOR version increments represent incompatible API changes
- MINOR version increments represent additional backwards-compatible functionality
- PATCH version increments represent backwards-compatible bug fixes

Version history overview

Software release	Serial protocol version	Main protocol improvements
2.7.2	2.8.0	Add serial time API
2.7.1	2.7.0	Add water tracking mode
2.6.1	2.6.0	Serial baud rate configurable. Add PD4 format support in serial 'wcp' command. Some serial protocol names changed.
2.5.2	2.5.0	Add PD4 format support (experimental)

Software release	Serial protocol version	Main protocol improvements
2.4.4	2.5.0	Change gyro calibration to store persistently. Note: gyro calibration commands now take up to 15 seconds.
2.4.0	2.5.0	Add ability to trigger pings (TCP JSON API/serial), add configuration for periodic cycling (TCP JSON API/serial)
2.2.1	2.4.0	Add serial output protocol configuration, range mode configuration and calibrate gyro command, Fix missing line ending in configuration (TCP JSON API), fix dark mode enabled naming inconsistency (TCP JSON API), change speed of sound and mounting rotation offset from integer to float
2.1.0	2.3.0	Add configuration, add time_of_validity/time_of_transmission, add covariance (TCP JSON API)
2.0.8	2.2.0	Add position estimation, Add output of orientation angles
1.6.0	2.1.0	Initial (velocity only)

Serial protocol

Overview

The default serial communication format is 115200 8-N-1 (no hardware flow control). Release 2.6.1 and later allow different baud rates through web GUI configuration.

DVL A50 and DVL A125 use a 3.3 V UART interface that is 5 V tolerant. See DVL A50/A125 serial interface.

Packets sent to and received from the DVL start with a `w` and end with LF or CR+LF. The packet format is:

Start byte	Direction	Command	Options (0 to many)	Checksum	End byte
`w`	`c` or `r`	`x`	`,[option]`	`*xx`	`\n` or `\r\n`

`Direction` is `c` (short for 'command') for packets sent to the DVL, and `r` (short for 'response') for packets sent from the DVL. The DVL always replies with CR+LF as the line ending. The commands can be sent from a terminal program that supports sending a full line at a time. The timeout between characters is approximately 10 ms.

Note

Please verify the baud rate configuration in the web GUI if you are unable to communicate with the default baud rate.

> **Note**
>
> The checksum is optional when sending commands to the DVL. The DVL always returns a checksum. The checksum algorithm is CRC-8 and it is formatted as a hexadecimal number using 2 lower-case characters (ex: `*c3`). See below for details.

Command overview

The commands in the table are shown without the checksum and without the mandatory line ending (`\n` or `\r\n`) for readability. In all cases, the response to a submitted command may be `wrn`, `wr?`, or `wr!` (see below for details), but only the format of a successful response is listed.

Command	Description	Response	Description
`wcv`	Get protocol version	`wrv,` *[major],[minor],[patch]*	Protocol version. eg: `wrv,2.5.0`
`wcw`	Get product detail	`wrw,` *[name],[version],[chipID],[IP address]*	Where type is dvl, name is product name, version is software version, chip ID is the chip ID and *optionally* the IP address if connected to DHCP server: eg: `wrw,dvl-a50,2.2.1,0xfedcba98765432` or `wrw,dvl-a50,2.2.1,0xfedcba98765432,10.11.12.140`
`wcs,` *[configuration parameters]*	Set configuration parameters	`wra`	Successfully set the specified configuration parameters. See Configuration for details
`wcc`	Get current configuration	`wrc,` *[configuration parameters]*	Entire current configuration. See Configuration for details
`wcr`	Reset dead reckoning	`wra`	Successfully started a new dead reckoning run
`wcx`	Trigger ping	`wra`	Successfully queued a ping
`wcg`	Calibrate gyro	`wra`	Successfully calibrated gyro
`wcp`	Change serial output protocol	`wra`	Successfully changed output protocol
`wcn,` *[ntp_address]*	Set NTP server configuration. `ntp_address` is an address, or "auto"	`wra`	Successfully set NTP server

Command	Description	Response	Description
`wcN`	Get NTP address configuration	`wrN,` *[ntp_address]*	Configured NTP server address, or "auto"
`wcm,` *[timestamp]*	Set manual time, where `timestamp` is UTC epoch time in microseconds	`wra`	Successfully set manual time
`wct`	Get time status	`wrs,` *[details below]*	Time status
`wcT,` *[timeout_seconds]*	Force NTP sync within given timeout	`wrT,` *[success]*	Where `success` is y if sync was achieved within timeout, and n otherwise
		`wrz,` *[details below]*	Velocities calculated
		`wrs,` *[details below]*	Velocities calculated during water tracking
		`wru,` *[details below]*	Transducer information
		`wrp,` *[details below]*	Dead reckoning report
		`wrx,` *[details below]*	DEPRECATED: Velocities calculated (old format)
		`wrt,` *[details below]*	DEPRECATED: Transducer information (old format)
		`wr?`	Malformed request: packet cannot be understood or no newline received before timeout
		`wr!`	Malformed request: packet does not match the given checksum
		`wrn`	Not acknowledged (nack): an error occurred when handling the packet

> ⚠ **`wrs` is used for two different reports**
>
> The `wrs` prefix is used for both the time status response and the water tracking velocity report. You must inspect the fields after the prefix to tell them apart: a water tracking velocity report begins with a decimal velocity (e.g. `wrs,0.120,...`), while a time status report begins with an integer epoch timestamp (e.g. `wrs,1776409079,...`).

Time status (wrs)

Time status is returned by the command `wct`.

The report has the following format: `wrs,` *[system_time],[ntp_synced],[ntp_synced_to],[ntp_seconds_since_last_sync]*

Variable	Description
system_time	UTC epoch time in microseconds
ntp_synced	`y` if synchronized, `n` if not
ntp_synced_to	NTP server address the DVL is synced to
ntp_seconds_since_last_sync	Seconds since last sync if synced

Examples:

```
wrs,1776409079,0,,
wrs,1776409079,1,192.168.0.10,45
```

Velocity report (wrz)

A velocity report is output for each velocity calculation of the DVL. The rate depends on the altitude of the DVL (distance to the sea bottom or other reflecting surface), but will be in the range 2-15 Hz.

The X, Y, and Z axes are with respect to body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a 'mounting rotation offset', from the forward axis of the vehicle.

The report has the following format: `wrz,` *[vx],[vy],[vz],[valid],[altitude],[fom],[covariance],[time_of_validity],[time_of_transmission],[time],[status]

Variable	Description
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
valid	If `y`, the DVL has a lock on the reflecting surface, and the altitude and velocities are valid (y/n)

Variable	Description
altitude	Measured altitude to the bottom (m)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this. 9 entries ((m/s)^2) separated by ;
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
time	Milliseconds since last velocity report (ms)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.

Example where all velocities are valid:

```
wrz,0.120,-0.400,2.000,y,1.30,1.855,1e-07;0;1.4;0;1.2;0;0.2;0;1e+09,7,14,123.00,1*50
```

Velocity report (wrs)

A velocity report is output for each velocity calculation of the DVL in this format when water tracking is activated. The update rate will be 2 Hz.

The X, Y, and Z axes are with respect to body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a 'mounting rotation offset', from the forward axis of the vehicle.

The report has the following format: `wrs,` *[vx],[vy],[vz],[valid],[fom],[covariance],[time_of_validity],[time_of_transmission],[time],[status]

Variable	Description
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
valid	If `y`, the DVL has a lock on the reflecting surface, and the altitude and velocities are valid (y/n)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)

Variable	Description
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this. 9 entries ((m/s)^2) separated by ;
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
time	Milliseconds since last velocity report (ms)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.

Example where all velocities are valid:

```
wrs,0.120,-0.400,2.000,y,1.855,1e-07;0;1.4;0;1.2;0;0.2;0;1e+09,7,14,123.00,1*7b
```

Water tracking is enabled by setting `range_mode` to `wt`:

```
wcs,,,,,wt,
```

Transducer report (wru)

A transducer report is output for each of the four transducers of the DVL for each velocity calculation of the DVL. The rate will be the same as that of the velocity report. If the transducer did not receive a signal that could be successfully decoded, `distance` will be set to -1, and `velocity` will be set to 0. RSSI and NSD will be output in all cases.

The report has the following format: `wru,` *[id],[velocity],[distance],[rssi],[nsd]*

Variable	Description
id	Zero-based transducer `id`, from 0 to 3
velocity	Velocity in the direction of the transducer (m/s)
distance	Distance (parallel to the transducer beam, i.e. not the vertical distance) to the reflecting surface from this transducer (m)
rssi	Received signal strength indicator: strength of the signal received by this transducer (dBm)
nsd	Noise spectral density: strength of the background noise received by this transducer (dBm)

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

Example where all data is valid:

```
wru,0,0.070,1.10,-40,-95*9c
wru,1,-0.500,1.25,-62,-104*f0
wru,2,2.200,1.40,-56,-98*18
wru,3,1.800,1.35,-58,-96*a3
```

Dead reckoning report (wrp)

A dead reckoning report outputs the current speed, position, and orientation of the DVL as calculated by dead reckoning, with respect to a frame defined by the axes of the DVL's body frame, or vehicle frame if a mounting rotation offset is set, at the start of the dead reckoning run. The expected update rate is 5 Hz.

The format is: `wrp,` *[time_stamp],[x],[y],[z],[pos_std],[roll],[pitch],[yaw],[status]*

Variable	Description
time_stamp	Time stamp of report (Unix timestamp in seconds)
x	Distance in X direction (m)
y	Distance in Y direction (m)
z	Distance in downward direction (m)
pos_std	Standard deviation (Figure of merit) for position (m)
roll	Rotation around X axis (degrees)
pitch	Rotation around Y axis (degrees)
yaw	Rotation around Z axis, i.e. heading (degrees)
status	Reports if there are any issues with the DVL (0 if no errors, 1 otherwise)

Example:

```
wrp,49056.809,0.41,0.15,1.23,0.4,53.9,13.0,19.3,0*de
wrp,49057.269,0.39,0.18,1.23,0.4,53.9,13.0,19.3,0*e2
```

Reset dead reckoning (wcr)

Dead reckoning can be reset by issuing the `wcr` command. The reply will be an ack (`wra`) if the reset is successful, and a nak (`wrn`) if not.

Calibrate gyro (wcg)

The gyro can be calibrated by issuing the `wcg` command. The reply will be an ack (`wra`) if the reset is successful, and a nak (`wrn`) if not.

Trigger ping (wcx)

In setups where multiple acoustic sensors are used it can be useful to control the pinging of each acoustic sensor individually. By setting the configuration `acoustic_enabled = n` the pinging of the DVL can be externally controlled. Up to 15 external trigger commands can be queued by issuing the `wcx` command. The DVL will execute each ping in quick succession until no more commands are in the queue.

See Integration for guidance on using triggering with other acoustic instruments.

For DVL A100/A250 RX-line hardware triggering, see Integration.

The reply will be an ack (`wra`) if the command is successful, and a nak (`wrn`) if queue is full.

Change serial output protocol (wcp)

The serial output protocol in use can be configured by issuing the `wcp` command. The selected protocol is persistent over reboots.

```
wcp,[protocol number]
```

The supported protocols are:

Protocol number	Name	Description
0	Output disabled	No output on serial. Recommended if serial port is not used to lower latency on Ethernet protocols.
1	WL - Serial V1 and V2	All output including the deprecated `wrx` and `wrt` sentences.
2	PD6	PD6 format output. See PD6 format description
3	WL - Serial V2	All output excluding the deprecated `wrx` and `wrt` sentences.
4	Not used	Not used
5	Not used	Not used

Protocol number	Name	Description
6	PD4	PD4 format output. See PD4 format description

The reply will be an ack (`wra`) if the protocol change is successful, and a nak (`wrn`) if not.

Example setting configuring output to use protocol number 3:

```
wcp,3
```

Note

Prior to release 2.6.1 protocol number 1 was named `Backward compatible` and protocol number 3 was named `Latest`. There is no change in functionality.

Configuration over serial

Configuration parameters

Variable	Description
speed_of_sound	Speed of sound (1000-2000 m/s). Float
mounting_rotation_offset	See the definition of the vehicle frame of the DVL. Typically 0, but can be set to be non-zero if the forward axis of the DVL is not aligned with the forward axis of a vehicle on which it is mounted (0-360 degrees). Float
acoustic_enabled	`y` for normal operation of the DVL, `n` when the sending of acoustic waves from the DVL is disabled (e.g. to save power or slow down its heating up in air)
dark_mode_enabled	`n` when the LED operates as normal. `y` for no blinking of LED (e.g. if the LED is interfering with a camera)
range_mode	`auto` when operating as normal, otherwise see range mode configuration
periodic_cycling_enabled	`y` to enable periodic cycling, `n` to disable it. See Configuration over TCP JSON API for details

Note

For backward compatibility the `range_mode` and `periodic_cycling_enabled` parameters are optional when setting the configuration. They will always be returned when reading the configuration (`wcc`). Speed of sound and mounting rotation was changed from integer to float in serial protocol 2.4.0

Fetching current configuration

The current configuration of the DVL can be obtained by issuing the `wcc` command.

If the configuration is successfully fetched, the response will be in the following format. If not, a nak `wrn` will be returned.

```
wrc,[speed_of_sound],[mounting_rotation_offset],[acoustic_enabled],[dark_mode_enabled],
[range_mode],[periodic_cycling_enabled]
```

Setting configuration parameters

Setting of configuration parameters can be carried out by issuing the `wcs` command in the following format.

```
wcs,[speed_of_sound],[mounting_rotation_offset],[acoustic_enabled],[dark_mode_enabled],
[range_mode],[periodic_cycling_enabled]
```

Those parameters which are not to be set can be left blank.

Example for setting dark mode without changing the other parameters:

```
wcs,,,,y,,
```

Example of setting water tracking:

Water tracking is enabled by setting `range_mode` to `wt`:

```
wcs,,,,,wt,
```

Example for setting speed of sound to 1450 m/s and disabling acoustics, without changing the other parameters:

```
wcs,1450,,n,,,
```

The response will be an ack `wra` if the parameters are successfully set, a nak `wrn` if the command was successfully parsed but the parameters were not successfully set, and a malformed request `wr?` if the command was not successfully parsed, e.g. if the wrong number of parameters was used, or either `speed_of_sound` or `mounting_rotation_offset` was not an integer.

The new configuration will not be returned in the response, but can be obtained by issuing a `wcc` command as above.

Velocity report, old format (wrx) [Deprecated]

Same purpose as the velocity report, but in an older format:

`wrx,` *[time],[vx],[vy],[vz],[fom],[altitude],[valid],[status]*

Variable	Description
time	Milliseconds since last velocity report (ms)

Variable	Description
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
altitude	Distance to the reflecting surface along Z axis (m)
valid	If `y`, the DVL has lock on the reflecting surface, and the altitude and velocities are valid (y/n)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.

Example where velocities are valid:

```
wrx,112.83,0.007,0.017,0.006,0.000,0.93,y,0*d2
wrx,140.43,0.008,0.021,0.012,0.000,0.92,y,0*b7
wrx,118.47,0.009,0.020,0.013,0.000,0.92,y,0*54
```

Example where velocities and altitude are not valid and a high temperature warning occurs:

```
wrx,1075.51,0.000,0.000,0.000,2.707,-1.00,n,1*04
wrx,1249.29,0.000,0.000,0.000,2.707,-1.00,n,1*6a
wrx,1164.94,0.000,0.000,0.000,2.707,-1.00,n,1*39
```

Transducer report, old format (wrt) [Deprecated]

Same purpose as the transducer report, but in an older format, and combining the data of all four transducers:

`wrt,` *[dist_1],[dist_2],[dist_3],[dist_4]*

Variable	Description
dist_1	Distance (parallel to the transducer beam, i.e. not the vertical distance) to reflecting surface from transducer 1 (m)
dist_2	Distance to reflecting surface from transducer 2 (m)
dist_3	Distance to reflecting surface from transducer 3 (m)
dist_4	Distance to reflecting surface from transducer 4 (m)

Example where all distances are valid:

```
wrt,15.00,15.20,14.90,14.20*b1
wrt,14.90,15.10,14.80,14.10*ac
```

Example where distance is not valid for transducer 4:

```
wrt,14.90,15.10,14.80,-1.00*53
wrt,15.00,15.20,14.90,-1.00*71
```

Checksum

The checksum algorithm is CRC-8 (Polynomal: 0x07, Init: 0x00, RefIn/RefOut: false, XorOut: 0x00, Check: 0xf4). Checksum is formatted as a hexadecimal number using 2 lower-case characters (ex: `*c3`).

Compatible implementations:

- Python 3: crcmod `crcmod.predefined.mkPredefinedCrcFun("crc-8")`
- Golang: github.com/sigurn/crc8 `crc8.MakeTable(crc8.CRC8)`

Example for how to verify checksum using Python 3 and crcmod:

```python
crc = crcmod.predefined.mkPredefinedCrcFun("crc-8")
sentence = b"wrx,1164.94,0.000,0.000,0.000,2.707,-1.00,n,1*39"
data, checksum = sentence.split(b"*")

if crc(data) == int(checksum, 16):
    print("CRC valid")
else:
    print("CRC invalid")
```

The crcmod python package can generate code in other languages. Here is an example (subject to the MIT License) for C which should be straightforward to adapt to other languages.

```
static const uint8_t lookup_table[256] = {
    0x00U,0x07U,0x0EU,0x09U,0x1CU,0x1BU,0x12U,0x15U,
    0x38U,0x3FU,0x36U,0x31U,0x24U,0x23U,0x2AU,0x2DU,
    0x70U,0x77U,0x7EU,0x79U,0x6CU,0x6BU,0x62U,0x65U,
    0x48U,0x4FU,0x46U,0x41U,0x54U,0x53U,0x5AU,0x5DU,
    0xE0U,0xE7U,0xEEU,0xE9U,0xFCU,0xFBU,0xF2U,0xF5U,
    0xD8U,0xDFU,0xD6U,0xD1U,0xC4U,0xC3U,0xCAU,0xCDU,
    0x90U,0x97U,0x9EU,0x99U,0x8CU,0x8BU,0x82U,0x85U,
    0xA8U,0xAFU,0xA6U,0xA1U,0xB4U,0xB3U,0xBAU,0xBDU,
    0xC7U,0xC0U,0xC9U,0xCEU,0xDBU,0xDCU,0xD5U,0xD2U,
    0xFFU,0xF8U,0xF1U,0xF6U,0xE3U,0xE4U,0xEDU,0xEAU,
    0xB7U,0xB0U,0xB9U,0xBEU,0xABU,0xACU,0xA5U,0xA2U,
    0x8FU,0x88U,0x81U,0x86U,0x93U,0x94U,0x9DU,0x9AU,
    0x27U,0x20U,0x29U,0x2EU,0x3BU,0x3CU,0x35U,0x32U,
    0x1FU,0x18U,0x11U,0x16U,0x03U,0x04U,0x0DU,0x0AU,
    0x57U,0x50U,0x59U,0x5EU,0x4BU,0x4CU,0x45U,0x42U,
    0x6FU,0x68U,0x61U,0x66U,0x73U,0x74U,0x7DU,0x7AU,
    0x89U,0x8EU,0x87U,0x80U,0x95U,0x92U,0x9BU,0x9CU,
    0xB1U,0xB6U,0xBFU,0xB8U,0xADU,0xAAU,0xA3U,0xA4U,
    0xF9U,0xFEU,0xF7U,0xF0U,0xE5U,0xE2U,0xEBU,0xECU,
    0xC1U,0xC6U,0xCFU,0xC8U,0xDDU,0xDAU,0xD3U,0xD4U,
    0x69U,0x6EU,0x67U,0x60U,0x75U,0x72U,0x7BU,0x7CU,
    0x51U,0x56U,0x5FU,0x58U,0x4DU,0x4AU,0x43U,0x44U,
    0x19U,0x1EU,0x17U,0x10U,0x05U,0x02U,0x0BU,0x0CU,
    0x21U,0x26U,0x2FU,0x28U,0x3DU,0x3AU,0x33U,0x34U,
```

```
    0x4EU,0x49U,0x40U,0x47U,0x52U,0x55U,0x5CU,0x5BU,
    0x76U,0x71U,0x78U,0x7FU,0x6AU,0x6DU,0x64U,0x63U,
    0x3EU,0x39U,0x30U,0x37U,0x22U,0x25U,0x2CU,0x2BU,
    0x06U,0x01U,0x08U,0x0FU,0x1AU,0x1DU,0x14U,0x13U,
    0xAEU,0xA9U,0xA0U,0xA7U,0xB2U,0xB5U,0xBCU,0xBBU,
    0x96U,0x91U,0x98U,0x9FU,0x8AU,0x8DU,0x84U,0x83U,
    0xDEU,0xD9U,0xD0U,0xD7U,0xC2U,0xC5U,0xCCU,0xCBU,
    0xE6U,0xE1U,0xE8U,0xEFU,0xFAU,0xFDU,0xF4U,0xF3U,
};

uint8_t crc8(uint8_t *message, int message_length) {
    uint8_t checksum = 0;
    while (message_length > 0) {
        checksum = lookup_table[*message ^ checksum];
        message++;
        message_length--;
    }
    return checksum;
}
```

PD6 format (TCP/Serial)

PD6 is supported over serial and TCP for all DVL models.

See the canonical PD6 format description.

PD4 format (TCP/Serial)

PD4 is supported over serial and TCP for all DVL models.

See the canonical PD4 format description.

DVL A100/A250 serial protocol

Describes the Water Linked DVL serial protocol for DVL A100 and DVL A250. For PD0, PD4, and PD6 output formats, see PD formats.

Terminology

- DVL - Doppler Velocity Log - Uses hydro-acoustic beams to measure the velocity at which the DVL is moving across a surface (typically an unmoving one such as the sea bottom), and the distance to this surface.
- ACK - Acknowledgement. The command issued was successful.
- NAK - Negative acknowledgement. The command issued failed.
- Ping - A pulse of sound sent by the DVL
- Time of validity - Timestamp of the surface reflection ('center of ping')
- Time of transmission - Timestamp taken directly before sending data over the serial or TCP protocols. The difference between time of transmission and time of validity includes both the time for the acoustic wave to travel from the surface from which it was reflected back to the DVL, and the decoding and processing of this signal internally in the DVL.

Version

This document describes serial protocol version `3.1.x` (major.minor.patch):

- MAJOR version increments represent incompatible API changes
- MINOR version increments represent additional backwards-compatible functionality
- PATCH version increments represent backwards-compatible bug fixes

Version history overview

Software release	Serial protocol version	Main protocol improvements
3.3.0	3.1.0	Add serial time API. Add water tracking mode. Remove deprecated protocol "WL - Serial V1 and V2".
3.2.0	3.0.0	Initial public release.

Serial protocol

Overview

The default serial communication format is 115200 8-N-1 (no hardware flow control). Different baud rates can be configured through the web GUI.

DVL A100 and DVL A250 use RS232. See DVL A100/A250 serial interface.

Packets sent to and received from the DVL start with a `w` and end with LF or CR+LF. The packet format is:

Start byte	Direction	Command	Options (0 to many)	Checksum	End byte
`w`	`c` or `r`	`x`	`,[option]`	`*xx`	`\n` or `\r\n`

`Direction` is `c` (short for 'command') for packets sent to the DVL, and `r` (short for 'response') for packets sent from the DVL. The DVL always replies with CR+LF as the line ending. The commands can be sent from a terminal program that supports sending a full line at a time. The timeout between characters is approximately 10 ms.

Note

Please verify the baud rate configuration in the web GUI if you are unable to communicate with the default baud rate.

Note

The checksum is optional when sending commands to the DVL. The DVL always returns a checksum. The checksum algorithm is CRC-8 and it is formatted as a hexadecimal number using 2 lower-case characters (ex: `*c3`). See below for details.

Command overview

The commands in the table are shown without the checksum and without the mandatory line ending (`\n` or `\r\n`) for readability. In all cases, the response to a submitted command may be `wrn`, `wr?`, or `wr!` (see below for details), but only the format of a successful response is listed.

Command	Description	Response	Description
`wcv`	Get protocol version	`wrv,` *[major],[minor],[patch]*	Protocol version. eg: `wrv,3.1.0`
`wcw`	Get product detail	`wrw,` *[name],[version],[chipID],[IP address]*	Where type is dvl, name is product name, version is software version, chip ID is the chip ID and *optionally* the IP address if connected to DHCP server: eg: `wrw,dvl-a250,3.4.0,0xfedcba98765432` or `wrw,dvl-a250,3.4.0,0xfedcba98765432,10.11.12.140`

Command	Description	Response	Description
`wcs,` *[configuration parameters]*	Set configuration parameters	`wra`	Successfully set the specified configuration parameters. See Configuration for details
`wcc`	Get current configuration	`wrc,` *[configuration parameters]*	Entire current configuration. See Configuration for details
`wcr`	Reset dead reckoning	`wra`	Successfully started a new dead reckoning run
`wcx`	Trigger ping	`wra`	Successfully queued a ping
`wcg`	Calibrate gyro	`wra`	Successfully calibrated gyro
`wcp`	Change serial output protocol	`wra`	Successfully changed output protocol
		`wrz,` *[details below]*	Velocities calculated
		`wrs,` *[details below]*	Velocities calculated during water tracking
		`wru,` *[details below]*	Transducer information
		`wrp,` *[details below]*	Dead reckoning report
		`wr?`	Malformed request: packet cannot be understood or no newline received before timeout
		`wr!`	Malformed request: packet does not match the given checksum
		`wrn`	Not acknowledged (nack): an error occurred when handling the packet

Velocity report (wrz)

A velocity report is output for each velocity calculation of the DVL. The rate depends on the model, altitude, and configuration. See the model page and Range mode for product-specific operating limits.

The X, Y, and Z axes are with respect to the body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a mounting rotation offset, from the forward axis of the vehicle.

The report has the following format: `wrz,` *[vx],[vy],[vz],[valid],[altitude],[fom],[covariance],[time_of_validity],[time_of_transmission],[time],[status]*

Variable	Description
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
valid	If `y`, the DVL has a lock on the reflecting surface, and the altitude and velocities are valid (y/n)
altitude	Measured altitude to the bottom (m)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this. 9 entries ((m/s)^2) separated by ;
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
time	Milliseconds since last velocity report (ms)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.

Example where all velocities are valid:

```
wrz,0.120,-0.400,2.000,y,1.30,1.855,1e-07;0;1.4;0;1.2;0;0.2;0;1e+09,7,14,123.00,1*50
```

Velocity report (wrs)

A velocity report is output for each velocity calculation of the DVL in this format when water tracking is activated. The update rate will be 2 Hz.

The X, Y, and Z axes are with respect to the body frame of the DVL, or the vehicle frame if the DVL is mounted on a vehicle at an angle, specified as a mounting rotation offset, from the forward axis of the vehicle.

The report has the following format: `wrs,` *[vx],[vy],[vz],[valid],[fom],[covariance],[time_of_validity],[time_of_transmission],[time],[status]*

Variable	Description
vx	Velocity in x direction (m/s)
vy	Velocity in y direction (m/s)
vz	Velocity in z direction (m/s)
valid	If `y`, the DVL has a lock on the reflecting surface, and the velocities are valid (y/n)
fom	Figure of merit, a measure of the accuracy of the velocities (m/s)
covariance	Covariance matrix for the velocities. The figure of merit is calculated from this. 9 entries ((m/s)^2) separated by ;
time_of_validity	Timestamp of the surface reflection, aka 'center of ping' (Unix timestamp in microseconds)
time_of_transmission	Timestamp from immediately before sending of the report over TCP (Unix timestamp in microseconds)
time	Milliseconds since last velocity report (ms)
status	8 bit status mask. Bit 0 is set to 1 for high temperature and DVL will soon enter thermal shutdown. Remaining bits are reserved for future use.

Example where all velocities are valid:

```
wrs,0.120,-0.400,2.000,y,1.855,1e-07;0;1.4;0;1.2;0;0.2;0;1e+09,7,14,123.00,1*7b
```

Water tracking is enabled by setting `range_mode` to `wt`:

```
wcs,,,,,wt,,
```

Transducer report (wru)

A transducer report is output for each of the four transducers of the DVL for each velocity calculation of the DVL. The rate will be the same as that of the velocity report. If the transducer did not receive a signal that could be successfully decoded, `distance` will be set to -1, and `velocity` will be set to 0. RSSI and NSD will be output in all cases.

The report has the following format: `wru,` *[id],[velocity],[distance],[rssi],[nsd]*

Variable	Description
id	Zero-based transducer `id`, from 0 to 3

Variable	Description
velocity	Velocity in the direction of the transducer (m/s)
distance	Distance (parallel to the transducer beam, i.e. not the vertical distance) to the reflecting surface from this transducer (m)
rssi	Received signal strength indicator: strength of the signal received by this transducer (dBm)
nsd	Noise spectral density: strength of the background noise received by this transducer (dBm)

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

Example where all data is valid:

```
wru,0,0.070,1.10,-40,-95*9c
wru,1,-0.500,1.25,-62,-104*f0
wru,2,2.200,1.40,-56,-98*18
wru,3,1.800,1.35,-58,-96*a3
```

Dead reckoning report (wrp)

A dead reckoning report outputs the current speed, position, and orientation of the DVL as calculated by dead reckoning, with respect to a frame defined by the axes of the DVL's body frame, or vehicle frame if a mounting rotation offset is set, at the start of the dead reckoning run. The expected update rate is 5 Hz.

The format is: `wrp,` *[time_stamp],[x],[y],[z],[pos_std],[roll],[pitch],[yaw],[status]*

Variable	Description
time_stamp	Time stamp of report (Unix timestamp in seconds)
x	Distance in X direction (m)
y	Distance in Y direction (m)
z	Distance in downward direction (m)
pos_std	Standard deviation (Figure of merit) for position (m)
roll	Rotation around X axis (degrees)
pitch	Rotation around Y axis (degrees)

Variable	Description
yaw	Rotation around Z axis, i.e. heading (degrees)
status	Reports if there are any issues with the DVL (0 if no errors, 1 otherwise)

Example:

```
wrp,49056.809,0.41,0.15,1.23,0.4,53.9,13.0,19.3,0*de
wrp,49057.269,0.39,0.18,1.23,0.4,53.9,13.0,19.3,0*e2
```

Reset dead reckoning (wcr)

Dead reckoning can be reset by issuing the `wcr` command. The reply will be an ack (`wra`) if the reset is successful, and a nak (`wrn`) if not.

Calibrate gyro (wcg)

Gyro calibration is not needed as a normal setup step for DVL A100/A250.

The gyro can be calibrated by issuing the `wcg` command. The reply will be an ack (`wra`) if the reset is successful, and a nak (`wrn`) if not.

Trigger ping (wcx)

In setups where multiple acoustic sensors are used it can be useful to control the pinging of each acoustic sensor individually. By setting the configuration `acoustic_enabled = n` the pinging of the DVL can be externally controlled. Up to 15 external trigger commands can be queued by issuing the `wcx` command. The DVL will execute each ping in quick succession until no more commands are in the queue.

See Integration for guidance on using triggering with other acoustic instruments.

For DVL A100 and DVL A250 hardware triggering, sending a NULL character over the RS232 RX line triggers a pulse on the falling edge.

TODO: Engineering review: confirm RX-line trigger timing requirements and valid serial configuration assumptions.

The reply will be an ack (`wra`) if the command is successful, and a nak (`wrn`) if queue is full.

Change serial output protocol (wcp)

The serial output protocol in use can be configured by issuing the `wcp` command. The selected protocol is persistent over reboots.

```
wcp,[protocol number]
```

The supported protocols are:

Protocol number	Name	Description
0	Output disabled	No output on serial. Recommended if serial port is not used to lower latency on Ethernet protocols.
1	Not used	Not used
2	PD6	PD6 format output. See PD6 format description
3	WL - Serial V2	All Water Linked serial output, including velocity, transducer, and dead reckoning reports.
4	Not used	Not used
5	Not used	Not used
6	PD4	PD4 format output. See PD4 format description

The reply will be an ack (`wra`) if the protocol change is successful, and a nak (`wrn`) if not.

Example setting configuring output to use protocol number 3:

```
wcp,3
```

Configuration over serial

Configuration parameters

Variable	Description
speed_of_sound	Speed of sound (1000-2000 m/s). Float
mounting_rotation_offset	See the definition of the vehicle frame of the DVL. Typically 0, but can be set to be non-zero if the forward axis of the DVL is not aligned with the forward axis of a vehicle on which it is mounted (0-360 degrees). Float
acoustic_enabled	`y` for normal operation of the DVL, `n` when the sending of acoustic waves from the DVL is disabled (e.g. to save power or slow down its heating up in air)
dark_mode_enabled	`n` when the LED operates as normal. `y` for no blinking of LED (e.g. if the LED is interfering with a camera)
range_mode	`auto` when operating as normal, otherwise see range mode configuration or activate water tracking

Variable	Description
periodic_cycling_enabled	`y` to enable periodic cycling, `n` to disable it. See Configuration over TCP JSON API for details
hardware_triggered_enabled	`y` to enable hardware triggering, `n` to disable

Note

For backward compatibility the `range_mode` and `periodic_cycling_enabled` parameters are optional when setting the configuration. They will always be returned when reading the configuration (`wcc`).

Fetching current configuration

The current configuration of the DVL can be obtained by issuing the `wcc` command.

If the configuration is successfully fetched, the response will be in the following format. If not, a nak `wrn` will be returned.

```
wrc,[speed_of_sound],[mounting_rotation_offset],[acoustic_enabled],[dark_mode_enabled],
[range_mode],[periodic_cycling_enabled],[hardware_trigger_enabled]
```

Setting configuration parameters

Setting of configuration parameters can be carried out by issuing the `wcs` command in the following format.

```
wcs,[speed_of_sound],[mounting_rotation_offset],[acoustic_enabled],[dark_mode_enabled],
[range_mode],[periodic_cycling_enabled],[hardware_trigger_enabled]
```

Those parameters which are not to be set can be left blank.

Example for setting dark mode without changing the other parameters:

```
wcs,,,,y,,,
```

Example of setting water tracking:

Water tracking is enabled by setting `range_mode` to `wt`:

```
wcs,,,,,wt,,
```

Example for setting speed of sound to 1450 m/s and disabling acoustics, without changing the other parameters:

```
wcs,1450,,n,,,,
```

The response will be an ack `wra` if the parameters are successfully set, a nak `wrn` if the command was successfully parsed but the parameters were not successfully set, and a malformed request `wr?` if the command was not successfully parsed, e.g. if the wrong number of parameters was used, or either `speed_of_sound` or `mounting_rotation_offset` was not a float.

The new configuration will not be returned in the response, but can be obtained by issuing a `wcc` command as above.

Checksum

The checksum algorithm is CRC-8 (Polynomal: 0x07, Init: 0x00, RefIn/RefOut: false, XorOut: 0x00, Check: 0xf4). Checksum is formatted as a hexadecimal number using 2 lower-case characters (ex: `*c3`).

Compatible implementations:

- Python 3: crcmod `crcmod.predefined.mkPredefinedCrcFun("crc-8")`
- Golang: github.com/sigurn/crc8 `crc8.MakeTable(crc8.CRC8)`

Example for how to verify checksum using Python 3 and crcmod:

```
crc = crcmod.predefined.mkPredefinedCrcFun("crc-8")
sentence = b"wrz,0.120,-0.400,2.000,y,1.30,1.855,1e-
07;0;1.4;0;1.2;0;0.2;0;1e+09,7,14,123.00,1*50"
data, checksum = sentence.split(b"*")

if crc(data) == int(checksum, 16):
    print("CRC valid")
else:
    print("CRC invalid")
```

The crcmod python package can generate code in other languages. Here is an example (subject to the MIT License) for C which should be straightforward to adapt to other languages.

```
static const uint8_t lookup_table[256] = {
    0x00U,0x07U,0x0EU,0x09U,0x1CU,0x1BU,0x12U,0x15U,
    0x38U,0x3FU,0x36U,0x31U,0x24U,0x23U,0x2AU,0x2DU,
    0x70U,0x77U,0x7EU,0x79U,0x6CU,0x6BU,0x62U,0x65U,
    0x48U,0x4FU,0x46U,0x41U,0x54U,0x53U,0x5AU,0x5DU,
    0xE0U,0xE7U,0xEEU,0xE9U,0xFCU,0xFBU,0xF2U,0xF5U,
    0xD8U,0xDFU,0xD6U,0xD1U,0xC4U,0xC3U,0xCAU,0xCDU,
    0x90U,0x97U,0x9EU,0x99U,0x8CU,0x8BU,0x82U,0x85U,
    0xA8U,0xAFU,0xA6U,0xA1U,0xB4U,0xB3U,0xBAU,0xBDU,
    0xC7U,0xC0U,0xC9U,0xCEU,0xDBU,0xDCU,0xD5U,0xD2U,
    0xFFU,0xF8U,0xF1U,0xF6U,0xE3U,0xE4U,0xEDU,0xEAU,
    0xB7U,0xB0U,0xB9U,0xBEU,0xABU,0xACU,0xA5U,0xA2U,
    0x8FU,0x88U,0x81U,0x86U,0x93U,0x94U,0x9DU,0x9AU,
    0x27U,0x20U,0x29U,0x2EU,0x3BU,0x3CU,0x35U,0x32U,
    0x1FU,0x18U,0x11U,0x16U,0x03U,0x04U,0x0DU,0x0AU,
    0x57U,0x50U,0x59U,0x5EU,0x4BU,0x4CU,0x45U,0x42U,
    0x6FU,0x68U,0x61U,0x66U,0x73U,0x74U,0x7DU,0x7AU,
    0x89U,0x8EU,0x87U,0x80U,0x95U,0x92U,0x9BU,0x9CU,
    0xB1U,0xB6U,0xBFU,0xB8U,0xADU,0xAAU,0xA3U,0xA4U,
    0xF9U,0xFEU,0xF7U,0xF0U,0xE5U,0xE2U,0xEBU,0xECU,
    0xC1U,0xC6U,0xCFU,0xC8U,0xDDU,0xDAU,0xD3U,0xD4U,
    0x69U,0x6EU,0x67U,0x60U,0x75U,0x72U,0x7BU,0x7CU,
    0x51U,0x56U,0x5FU,0x58U,0x4DU,0x4AU,0x43U,0x44U,
    0x19U,0x1EU,0x17U,0x10U,0x05U,0x02U,0x0BU,0x0CU,
    0x21U,0x26U,0x2FU,0x28U,0x3DU,0x3AU,0x33U,0x34U,
    0x4EU,0x49U,0x40U,0x47U,0x52U,0x55U,0x5CU,0x5BU,
    0x76U,0x71U,0x78U,0x7FU,0x6AU,0x6DU,0x64U,0x63U,
    0x3EU,0x39U,0x30U,0x37U,0x22U,0x25U,0x2CU,0x2BU,
    0x06U,0x01U,0x08U,0x0FU,0x1AU,0x1DU,0x14U,0x13U,
    0xAEU,0xA9U,0xA0U,0xA7U,0xB2U,0xB5U,0xBCU,0xBBU,
```

```
    0x96U,0x91U,0x98U,0x9FU,0x8AU,0x8DU,0x84U,0x83U,
    0xDEU,0xD9U,0xD0U,0xD7U,0xC2U,0xC5U,0xCCU,0xCBU,
    0xE6U,0xE1U,0xE8U,0xEFU,0xFAU,0xFDU,0xF4U,0xF3U,
};

uint8_t crc8(uint8_t *message, int message_length) {
    uint8_t checksum = 0;
    while (message_length > 0) {
        checksum = lookup_table[*message ^ checksum];
        message++;
        message_length--;
    }
    return checksum;
}
```

PD0 format

PD0 is planned for DVL A100 and DVL A250. It is not available yet.

When ready, PD0 is planned for both TCP and serial output.

See PD formats.

PD6 format (TCP/Serial)

PD6 is supported over serial and TCP for all DVL models.

See the canonical PD6 format description.

PD4 format (TCP/Serial)

PD4 is supported over serial and TCP for all DVL models.

See the canonical PD4 format description.

PD formats

The DVL can output PD-compatible formats for integration with systems that already support these formats.

PD formats are data formats. They can be available over different transports depending on model, software version, and configuration.

For new integrations, the TCP JSON API is usually the easiest starting point:

- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API

Supported formats

Format	Status	Notes
PD4	Supported	Supported over TCP and serial for DVL A50, DVL A125, DVL A100, and DVL A250.
PD6	Supported	Supported over TCP and serial for DVL A50, DVL A125, DVL A100, and DVL A250.
PD0	Planned for DVL A100/A250	Not available yet. Planned over TCP and serial when ready.

Transport support

Format	TCP	Serial	Models
PD4	Yes	Yes	DVL A50, DVL A125, DVL A100, DVL A250
PD6	Yes	Yes	DVL A50, DVL A125, DVL A100, DVL A250
PD0	Planned	Planned	DVL A100, DVL A250

PD4 and PD6 over TCP are always enabled. The TCP port is configurable in the web GUI.

Serial PD output is configured through the serial output protocol setting:

- DVL A50/A125 serial protocol
- DVL A100/A250 serial protocol

When to use PD formats

Use PD formats when integrating with software or systems that already expect PD0, PD4, or PD6 output.

For custom software integrations, use the TCP JSON API unless the receiving system specifically requires a PD format:

- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API

PD6

PD6 support allows integration with equipment that already has a PD6-compatible interface, reducing the need to create a driver based on the Water Linked serial protocol.

PD6 is supported for output over serial and Ethernet. PD6 over TCP is always enabled. The port in use is configurable in the web GUI. The default port is TCP 1037.

The serial output can be configured to output PD6 using the serial output protocol setting.

Command overview

Sentences TS, BI, and BD are filled with relevant numbers. All other sentences are set to zero values.

Timing and scaling data (TS)

```
:TS,YYMMDDHHmmsshh,SS.S,+TT.T,DDDD.D,CCCC.C,BBB <CR><LF>
```

Field	Explanation	Value
YYMMDDHHmmsshh	Year, month, day, hour, minute, second, hundredths of seconds	Report timestamp
SS.S	Salinity in parts per thousand (ppt).	Always 0
DDDD.D	Depth of transducer face in meters.	Always 0
CCCC.C	Speed of sound in meters per second.	Configured speed of sound
BBB	Built-in Test (BIT) result code.	Always 0

Bottom track, instrument referenced velocity data (BI)

```
:BI,±XXXXX,±YYYYY,±ZZZZZ,±EEEEE,S <CR><LF>
```

Field	Explanation	Value
±XXXXX	X-axis velocity data in mm/s	Current speed
±YYYYY	Y-axis velocity data in mm/s	Current speed
±ZZZZZ	Z-axis velocity data in mm/s	Current speed
±EEEEE	Error in velocity data in mm/s	Current error
S	Status of velocity	A = good. V = bad

Note

Axis used in the BI sentence is the vehicle frame.

Bottom track, earth referenced distance data (BD)

`:BD,±EEEEEEEE.EE,±NNNNNNNN.NN,±UUUUUUUU.UU,DDDD.DD,TTT.TT <CR><LF>`

Field	Explanation	Value
±EEEEEEEE.EE	East distance in meters.	Always 0
±NNNNNNNN.NN	North distance in meters.	Always 0
±UUUUUUUU.UU	Upward distance in meters.	Always 0
DDDD.DD	Range to bottom in meters	Current altitude
TTT.TT	Time since last good velocity estimate in seconds.	Always 0

Bottom track, ship referenced distance data (BS)

- In the 2.4.0 software release, the BS values are always zero.
- As of the 2.4.4 software release, the BS values are given by the actual velocity.

`:BS,±TTTTTTTT.TT,±LLLLLLLL.LL,±NNNNNNN.NN,S <CR><LF>`

Field	Explanation	Value
±TTTTTTTT.TT	Transverse movement, (+ = Port to Starboard velocity relative to bottom) in mm/s	Y axis velocity
±LLLLLLLL.LL	Longitudinal movement, (+ = Aft to Forward velocity relative to bottom) in mm/s	X axis velocity

Field	Explanation	Value
±NNNNNNN.NN	Ship velocity away from bottom in mm/s	Z axis velocity
S	Status of velocity	A = good. V = bad

Example output

```
:SA, +0.00, +0.00,  0.00
:TS,22020812061800, 0.0, +0.0,   0.0,1475.0,  0
:WI,    +0,    +0,    +0,    +0,V
:WS,    +0,    +0,    +0,V
:WE,    +0,    +0,    +0,V
:WD,       +0.00,       +0.00,       +0.00,   0.00,  0.00
:BI,  +123,  -420, +2000,    +0,A
:BS,  -420,  +123, +2000,A
:BE,    +0,    +0,    +0,V
:BD,       +0.00,       +0.00,       +0.00,   5.32,  0.00
```

PD4

The PD4 string is intended for use with equipment that already has a PD4-compatible interface, reducing the need to create a driver based on the Water Linked serial protocol.

PD4 is supported for output over serial and Ethernet. PD4 over TCP is always enabled. The port in use is configurable in the web GUI. The default port is TCP 1038.

Note

PD4 format support was added in software release 2.5.2 and is experimental. Please give feedback on this feature.

Data format

PD4 is a binary format where fields are defined by their position in one message. Data fields which use more than one byte are LittleEndian encoded.

Byte(s)	Data type	Used	Unit
0	DVL Data ID 7Dh	y	
1	Data structure (always equal to 0)	y	
2,3	Number of bytes	y	
4	System Config (0x10100011[1])	y	

Byte(s)	Data type	Used	Unit
5,6	X velocity bottom	y	mm/s
7,8	Y velocity bottom	y	mm/s
9,10	Z velocity bottom	y	mm/s
11,12	E velocity bottom	y	mm/s
13,14	BM1 range to bottom	y	cm
15,16	BM2 range to bottom	y	cm
17,18	BM3 range to bottom	y	cm
19,20	BM4 range to bottom	y	cm
21	Bottom status	y	bool
22,23	X-Velocity reference layer	n	
24,25	Y-Velocity reference layer	n	
26,27	Z-Velocity reference layer	n	
28,29	E-Velocity reference layer	n	
30,31	Reference layer start	n	
32,33	Reference layer end	n	
34	Reference layer status	n	
35	Time of first ping - hour	y	hours
36	Time of first ping - minute	y	minutes
37	Time of first ping - second	y	seconds
38	Time of first ping - hundredths	y	centi-seconds
39,40	Bit result	n	
41,42	Speed of Sound	y	m/s

Byte(s)	Data type	Used	Unit
43,44	Temperature	n	
45,46	Checksum	y	N/A

BM1-BM4 mapping

To use BM1-BM4 correctly, use this translation. The numbering comes from the transducer numbering shown in the image below.

- **BM1** is transducer 3.
- **BM2** is transducer 1.
- **BM3** is transducer 4.
- **BM4** is transducer 2.

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.



PD0

PD0 is planned for DVL A100 and DVL A250. It is not available yet.

When ready, PD0 is planned for both TCP and serial output.

---

1. Indicates that the velocities are in ship coordinates, tilt is used, three-beam is not computed, and the frequency is 600 kHz. ↩

DVL triggering

The DVL has different options for triggering acoustic pings to perform a velocity measurement. Triggering can be useful when you need to synchronize the DVL with other sensors.

Model	Software triggering	Hardware triggering
A50/A125	Yes	No
A100/A250	Yes	Yes

Hardware triggering

Use the RS232 RX line of the DVL to trigger a ping after enabling hardware triggering in the configuration.

The trigger is detected on the falling edge of the RX signal. The signal must then remain low for approximately 78.1 µs before returning high.

For a connection configured as 115200 baud, 8N1, this pulse corresponds to transmitting a `"NUL"` ASCII character (`0x00` , Ctrl + @).

Because the trigger is defined by its pulse timing, sending a `NUL` character at another baud rate may **not** produce a valid trigger.

The trigger can be sent as often as needed. If the DVL is already transmitting or receiving, incoming trigger signals are discarded.

Warning

Acoustics must be enabled. Hardware triggering will not work while acoustics are disabled.

Software triggering

Software triggering is available for all DVL models. It triggers a ping over the TCP JSON API or serial protocol. Acoustics must be disabled for this to work.

Software triggering differs from hardware triggering because it is sent as a request to the DVL and placed in a ping queue. The queue can store up to 15 requests, which are executed one after another until the queue is empty.

See the software ping command in Commands.

Because this method depends on the internal state of the DVL, the ping queue, and Ethernet jitter if Ethernet is used, the exact ping time is less predictable than with hardware triggering.

Warning

Disable acoustics before using software triggering. This is different from hardware triggering.

Software resources

This page lists Water Linked software resources that may be useful when integrating DVL products.

GitHub organization

Water Linked publishes software examples, tools, and integration resources on GitHub:

https://github.com/waterlinked

The repositories below were identified from the Water Linked GitHub organization and checked against repository descriptions or README files on 2026-05-07. Repository availability, names, and APIs may change over time. Check each repository README before using it in an integration.

DVL software repositories

Repository	Description	Typical use	Maintained
waterlinked_dvl	C++ library and ROS 2 driver for Water Linked DVL devices, including DVL A50 and DVL A125.	ROS 2 integration, C++ integration, and starting from the included library examples.	Yes
dvl-python	Example code for working with the Water Linked DVL serial protocol and TCP JSON API.	Legacy Python examples for reading DVL protocol data over serial or TCP. Use together with the current protocol documentation.	No
dvl-a50-ros-driver	Archived ROS 1 package for the Water Linked DVL A50.	Legacy ROS 1 integrations that still depend on the original DVL A50 driver.	No

Examples and integration helpers

Use these repositories as starting points when you need code-level integration examples:

- waterlinked_dvl for ROS 2 and C++ projects using DVL A50 or DVL A125.
- dvl-python for legacy Python examples using the DVL serial protocol and TCP JSON API.
- dvl-a50-ros-driver for legacy ROS 1 projects using DVL A50.

Where a repository README states model support, use that scope as the source of truth. For other DVL models or custom integrations, use the protocol documentation and the integration guide linked below.

Related resources

- DVL integration guide
- DVL A50/A125 TCP JSON API
- DVL A100/A250 TCP JSON API
- DVL A50/A125 serial protocol
- DVL A100/A250 serial protocol
- PD formats
- PD integration overview
- Electrical connection
- Network setup
- DVL diagnostic log
- How to diagnose

Notes

Software repositories are provided as integration aids. They may not cover every DVL model, software version, or customer platform. For production integrations, verify behavior against the current DVL protocol documentation and the README in the repository you plan to use.

DVL for BlueROV2

Introduction

The Water Linked DVLs are designed to integrate with most ROVs and AUVs through Ethernet and/or a serial interface (UART). To minimize the time between receipt of a DVL and having it up and running, the DVLs are shipped with an attached I/O interface for power, Ethernet, and serial connectivity. However, for most ROVs and AUVs, the DVL has to be integrated with the on-board electronics through a penetrator or a subsea connector.

This guide details how the DVL A50 can be integrated with the BlueROV2. The same procedure can be carried out for the DVL A125.

Parts and tools

The parts and tools needed depend on whether you intend to permanently attach the DVL A50 to the BlueROV2, have it semi-permanently attached, or use a subsea connector for quick installation/removal.

This guide details how to permanently attach the DVL A50 to a BlueROV2.

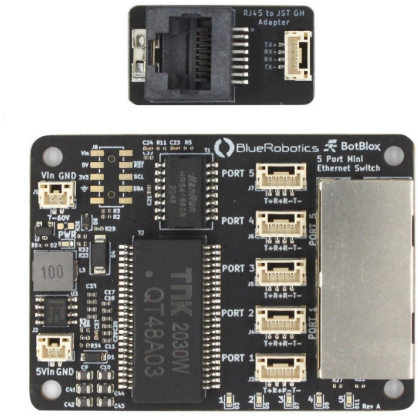
You will need



- DVL A50



- BlueROV2



- Small Ethernet switch. The one pictured here is from Blue Robotics, and is ideal for use in the BlueROV2.

dvl-bracket

- DVL A50 bracket. This is optional, but protects the DVL A50 and allows for down to 0 cm minimum altitude.

You will also need:

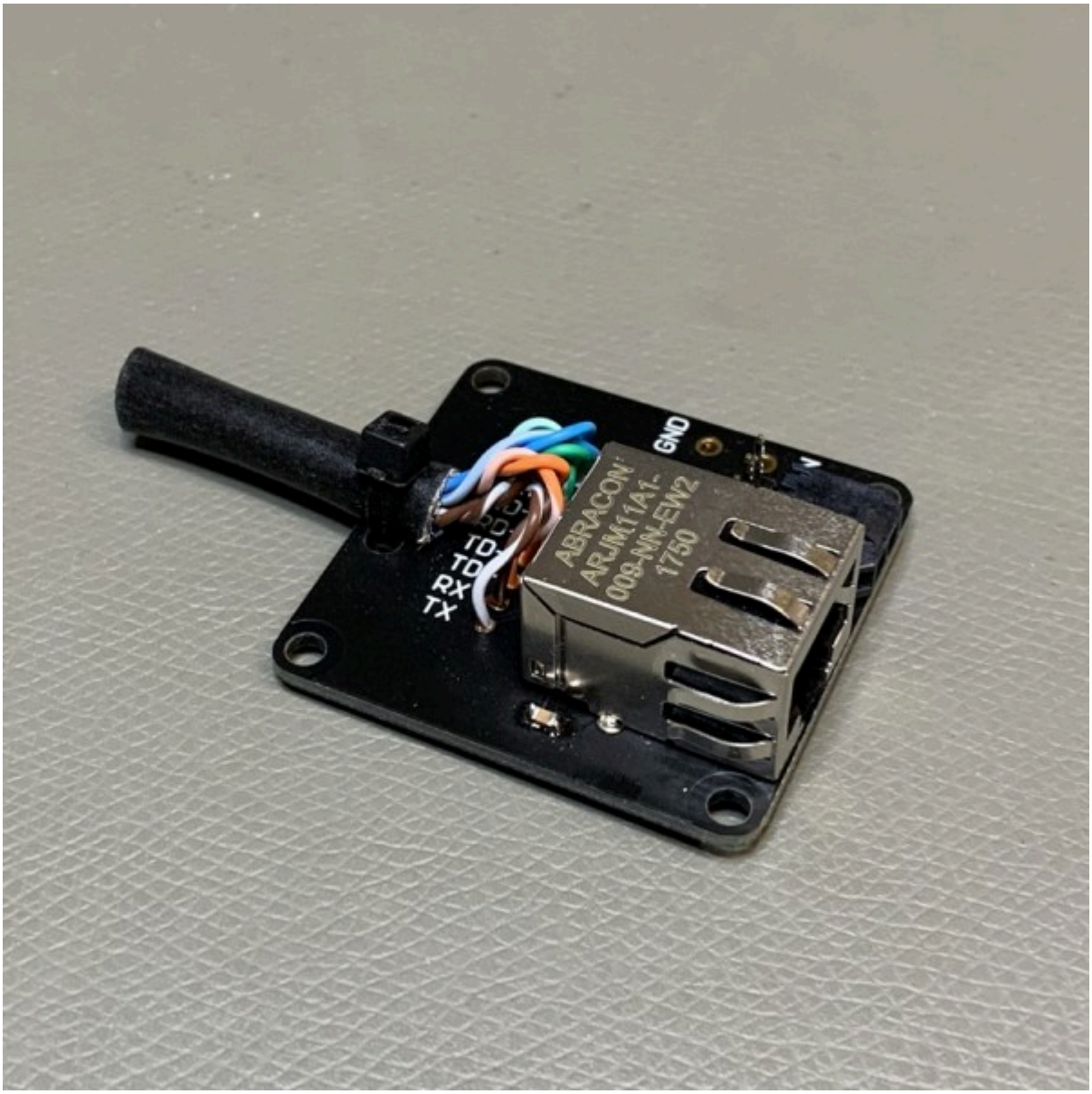
- Hex Key Set from Blue Robotics
- Bulkhead Wrench from Blue Robotics
- Plug Wrench from Blue Robotics
- (Optional) Cable Jacket Stripper from Blue Robotics
- M10-7.5mm-HC WetLink Penetrator from Blue Robotics
- Silicone grease
- 5 mm universal drill bit
- 2 x M5x12 mm stainless steel pan head screws (included with BlueROV2 mounting bracket)
- 4 x M3x4 mm stainless steel countersunk flat head screws (included with BlueROV2 mounting bracket)

- Soldering iron
- Solder wire
- Wire cutter
- Tweezers or nose pliers
- Solder wick and/or desoldering pump
- Utility knife
- Wire stripping tool
- Zip ties
- 4 x 30 cm (12") wires (options discussed under Connect DVL A50 and BlueROV2). If possible, use 2 x red and 2 x black wires.
- 15 cm (6") Ethernet cable

Preparation

Separate I/O Interface from DVL A50

First, cut the DVL A50 cable to separate the I/O Interface board. We recommend cutting near the I/O Interface board end of the cable.



Warning

The DVL A50 cable is permanently attached and non-replaceable inside the DVL A50. If you cut the cable too short, you will either have to splice the cable with another cable or order a new DVL A50. Measure twice, cut once!

Attaching the BlueROV2 mounting bracket

- Attach the BlueROV2 mounting bracket to the DVL A50 using the 4 x M3x6 screws provided with the DVL A50 BlueROV2 Integration Kit.



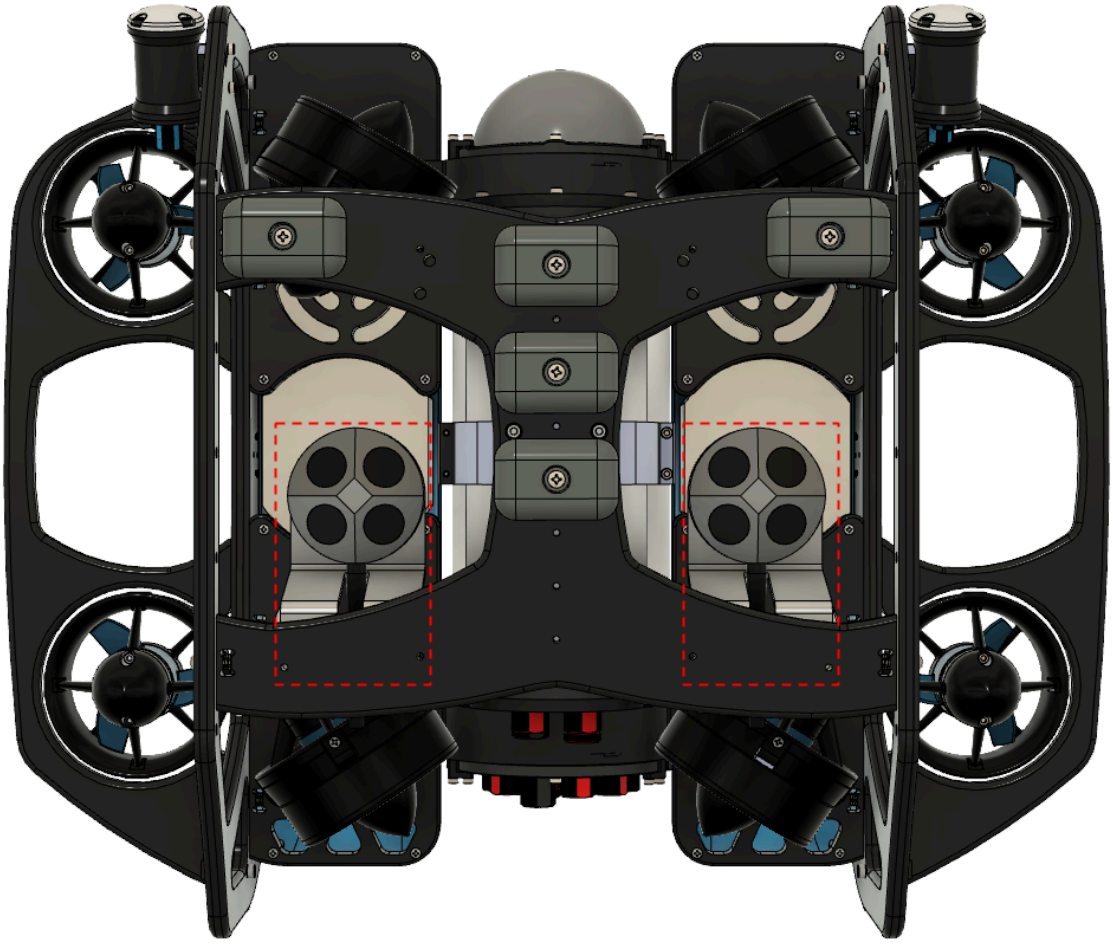
The BlueROV2 mounting bracket can be attached to the BlueROV2 in one of the two places indicated in the image below (standard BlueROV2).

Note

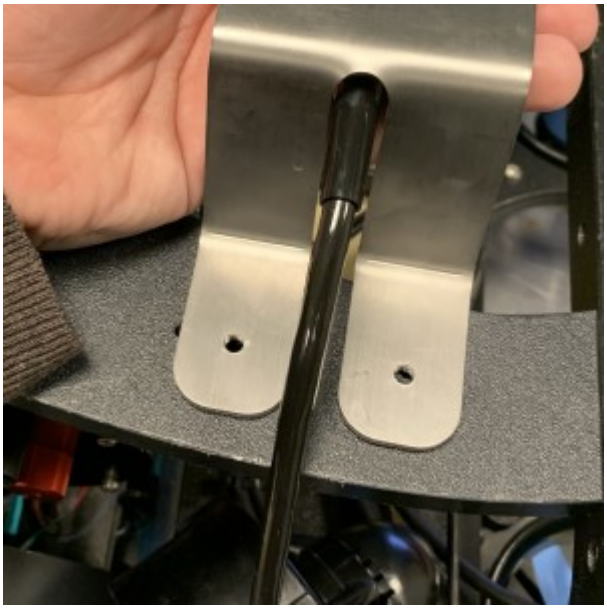
In this configuration, the DVL A50 is pointing backwards, meaning both the x-direction and y-direction are flipped. Multiplying the velocity data in x- and y-direction by -1 will flip the coordinates back to the same perspective frame as the BlueROV2.



If your BlueROV2 has the Heavy Configuration upgrade, the BlueROV2 mounting bracket can be attached such that the DVL A50 is within the BlueROV2 frame. This allows for a minimum working distance of 0 cm for the DVL A50.

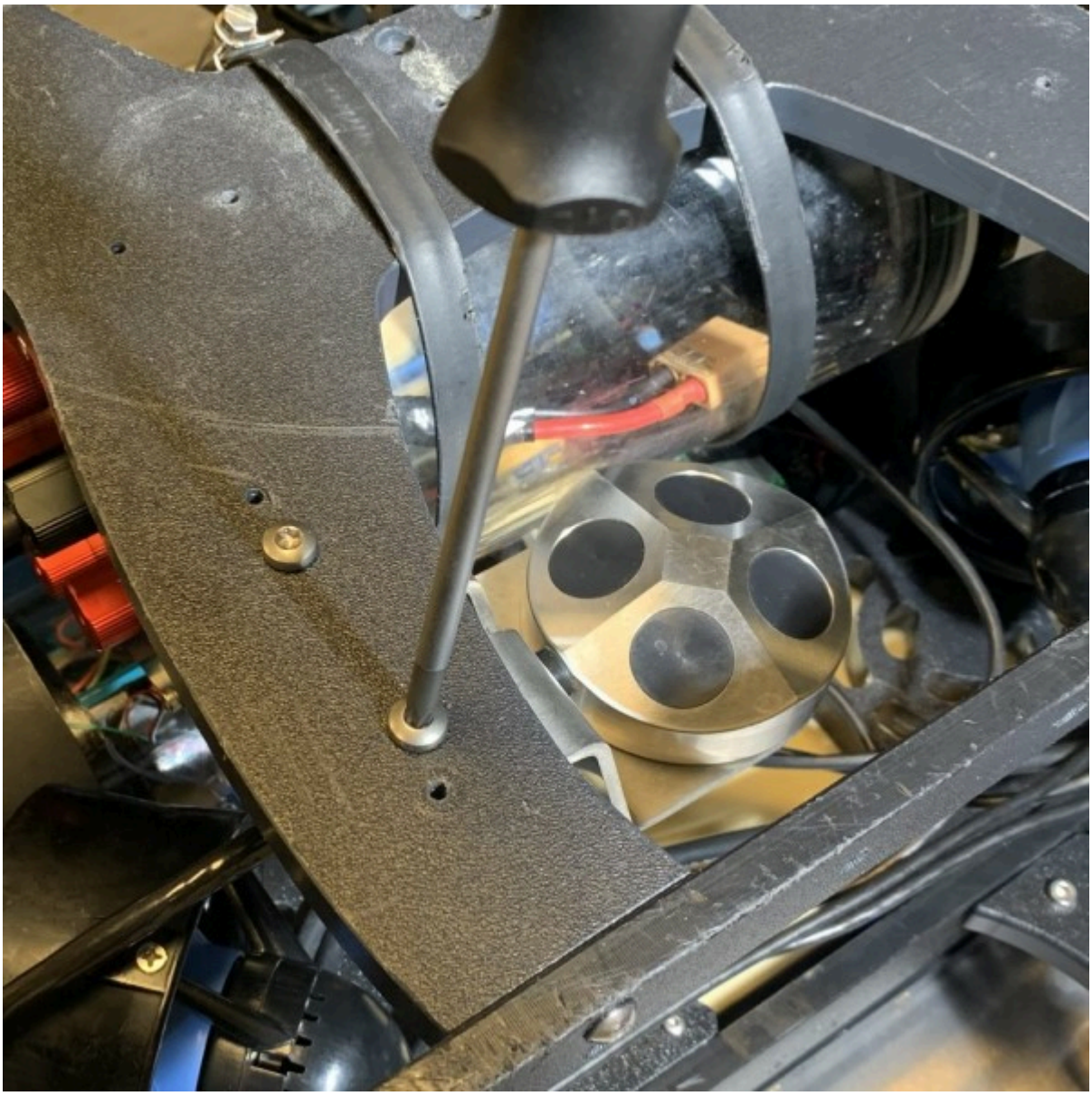


- Mark 2 hole positions on the BlueROV2 frame for attaching the BlueROV2 mounting bracket and drill through the frame using a 5 mm universal drill bit.



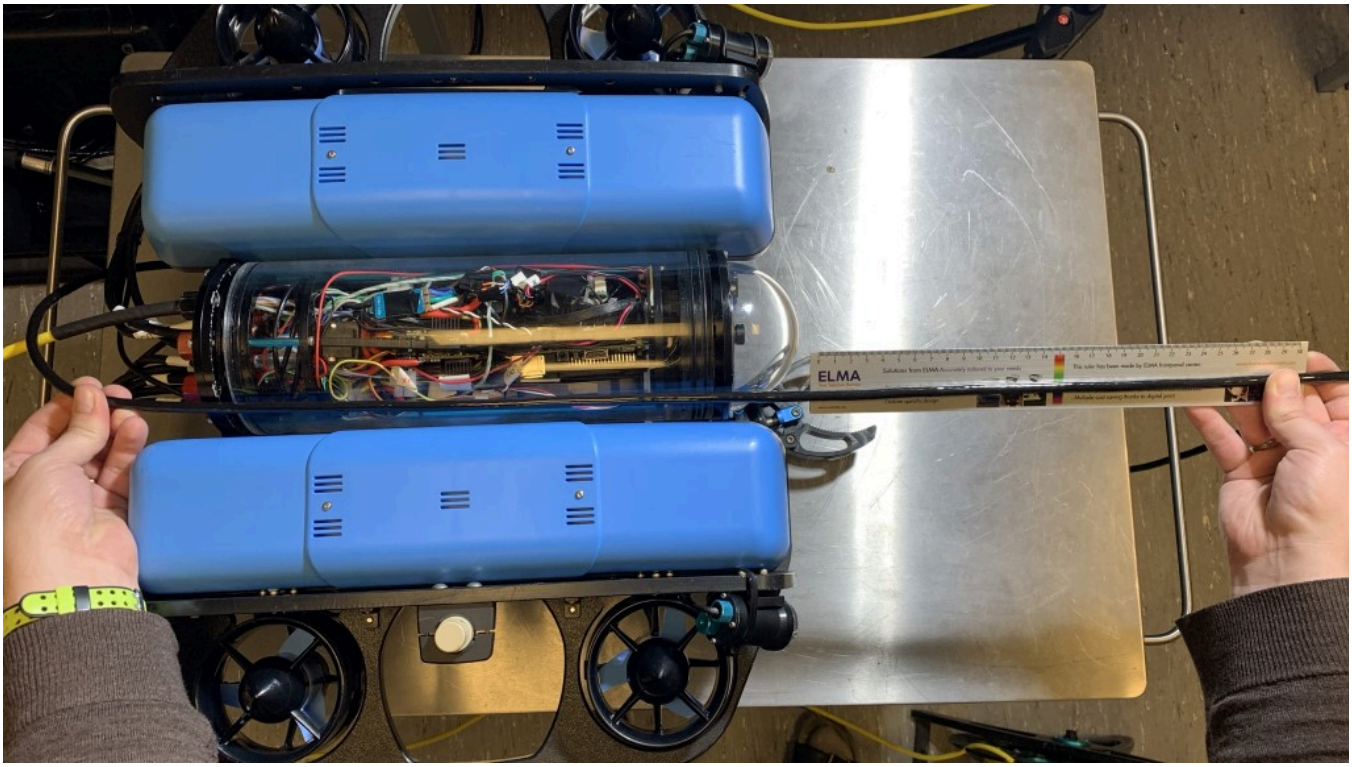


Check that the BlueROV2 mounting bracket will attach properly to the BlueROV2 frame using 2 x M5x16 mm screws. No need to secure the bracket in place as more work must be done on the cable end.



Final cable length

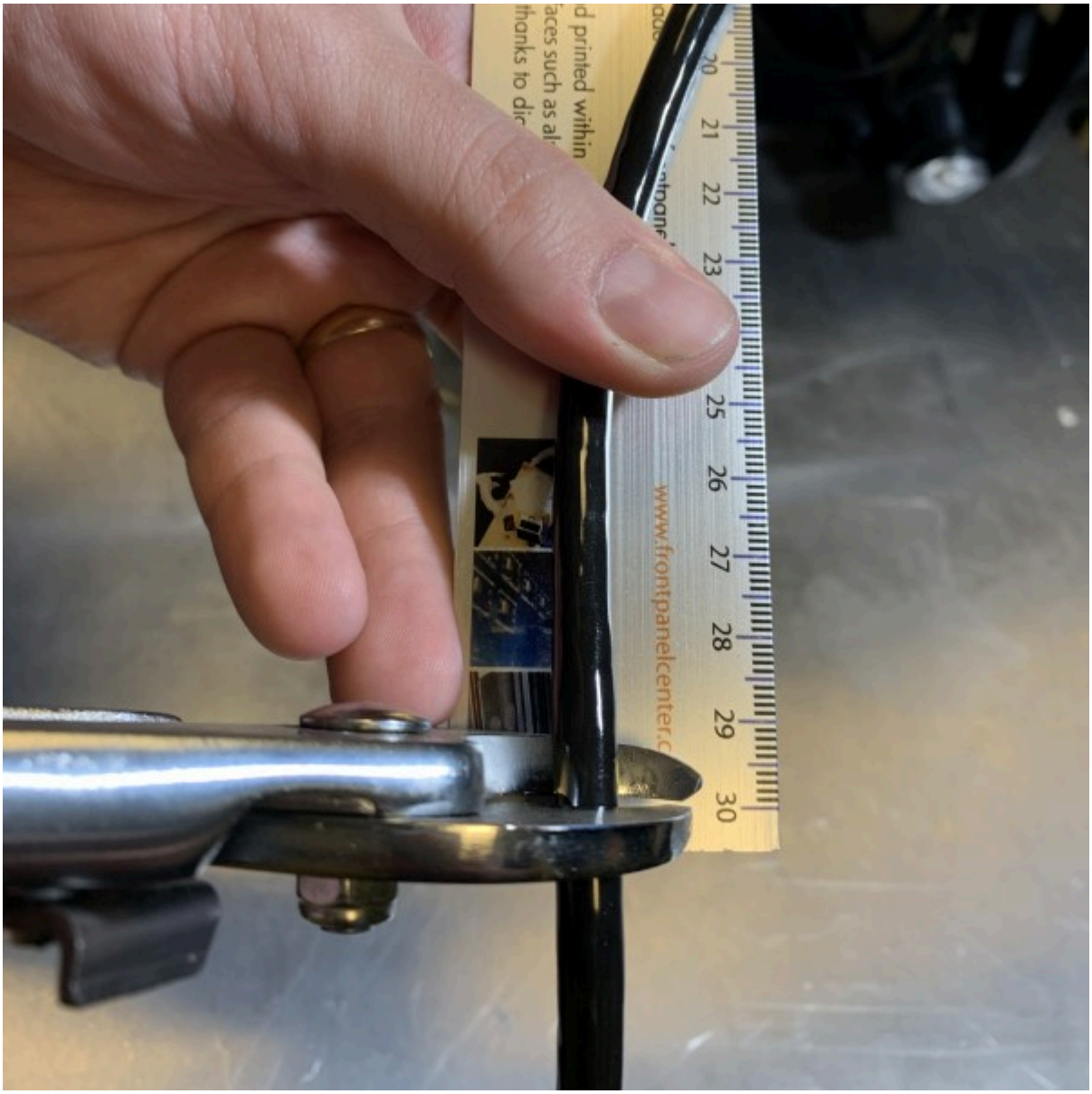
With the DVL A50 now attached to the BlueROV2 frame, find the appropriate cable length by leading the cable back to the penetrator end cap of the BlueROV2 and adding the length of the electronics enclosure (30 cm) to the cable. This is the *minimum* required cable length.



Tip

We recommend adding at least another 30 cm to the cable to provide enough service loop in case the penetrator fails or the leads get damaged.

Cut the cable to the final length.



Installation

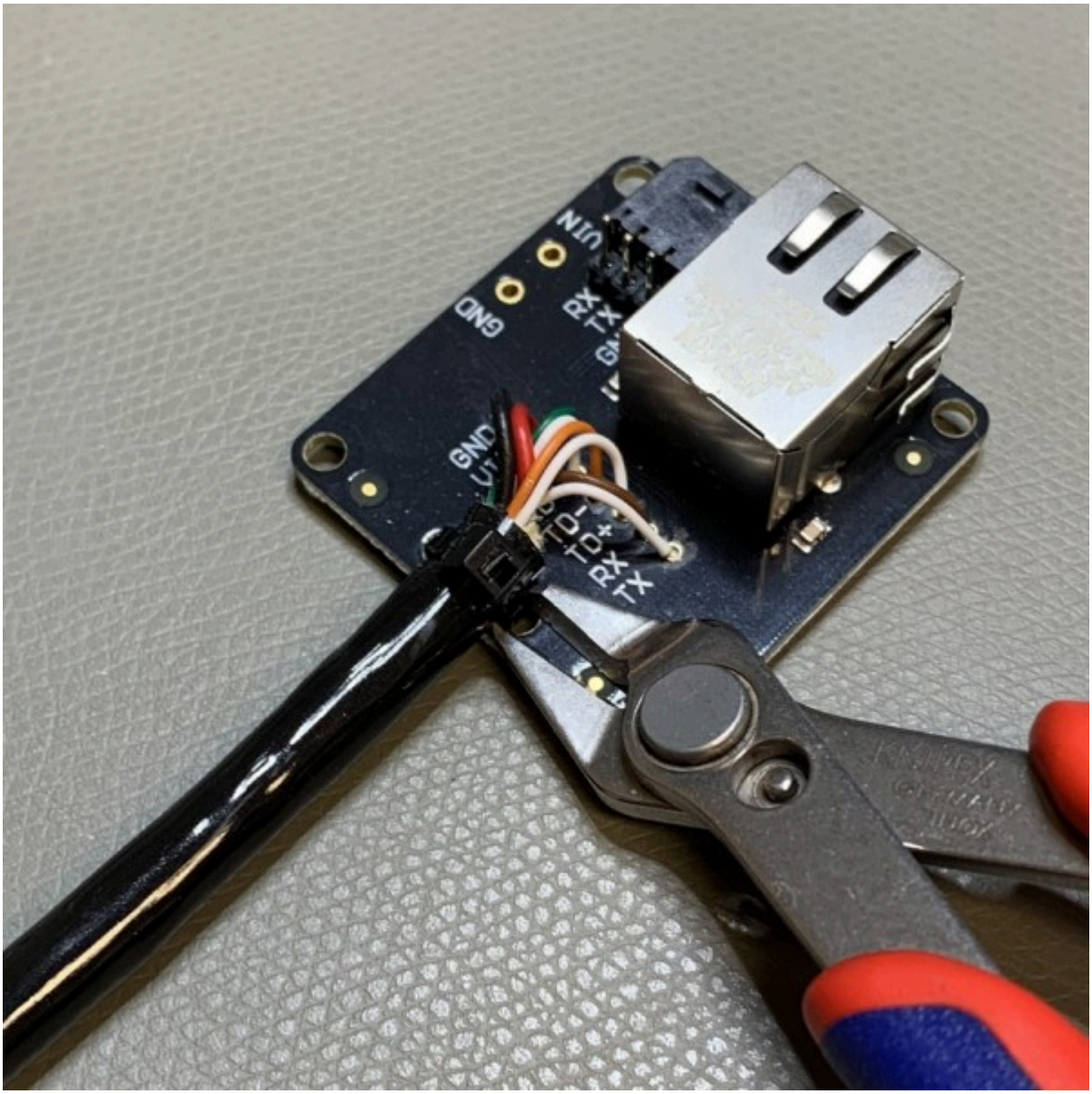
The DVL A50 will be permanently installed on the BlueROV2 with the I/O Interface board supplied with the DVL A50.

I/O Interface preparation

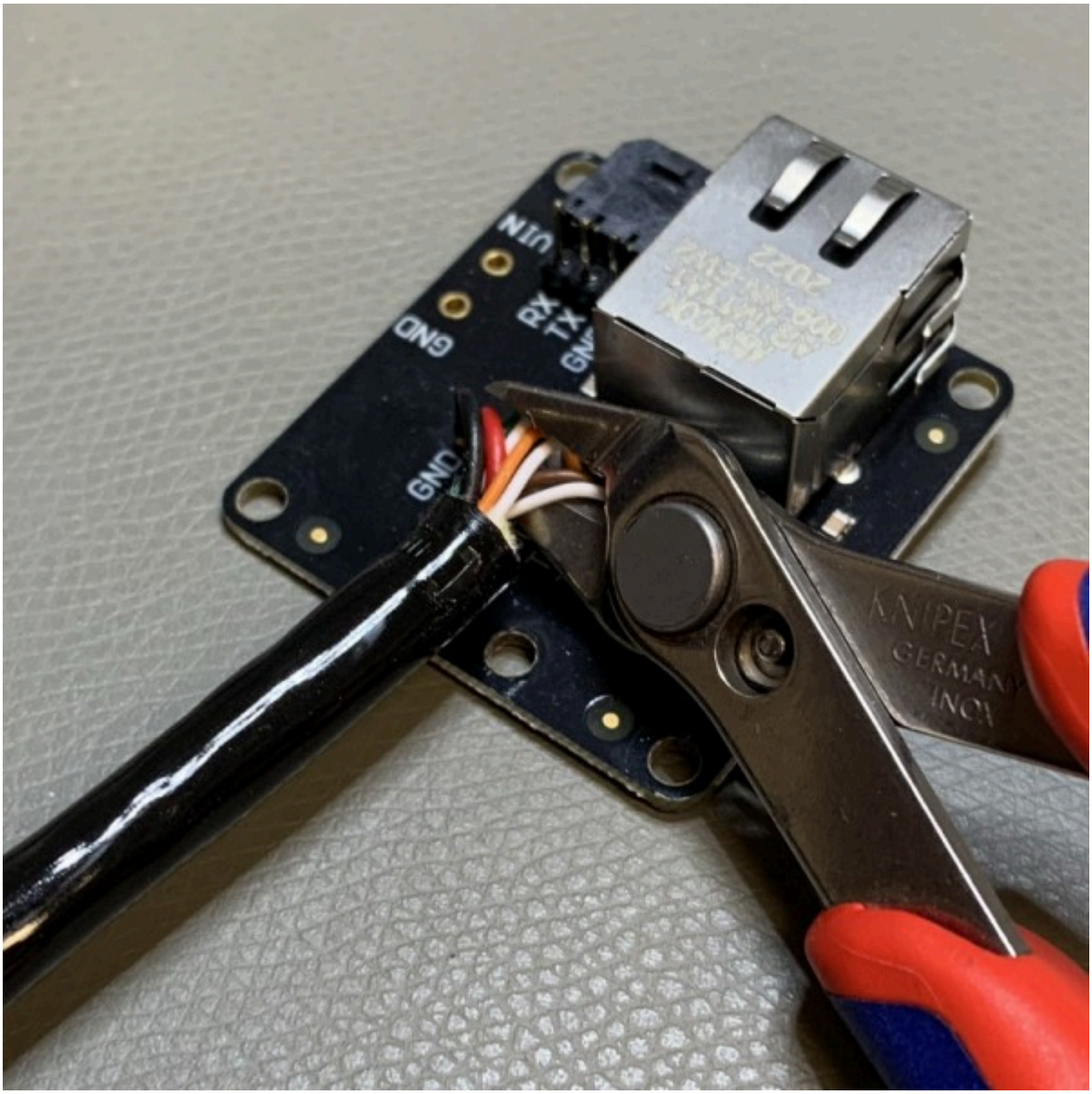
To prepare the I/O Interface Board you will need:

- Soldering iron
- Wire cutter
- Tweezers or nose pliers
- Solder wick and/or desoldering pump

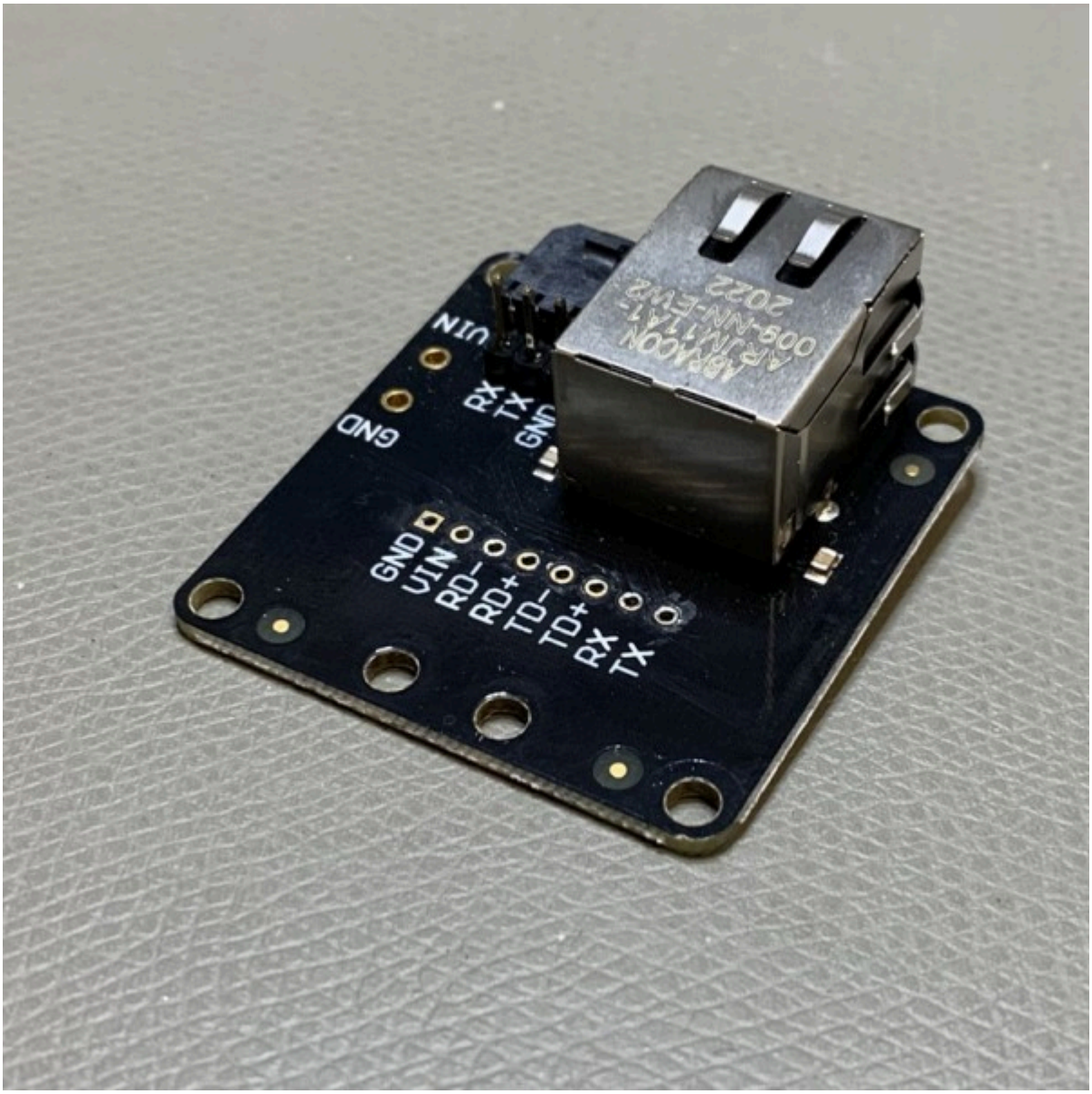
Cut off the zip tie holding the cable onto the I/O Interface board.



Cut off the individual wires soldered onto the I/O Interface board. Leave enough wire to grab onto with tweezers or nose pliers.



Desolder all the cut wires. Use solder wick and/or a desoldering pump to remove any excess solder from the now desoldered through hole pads.



Warning

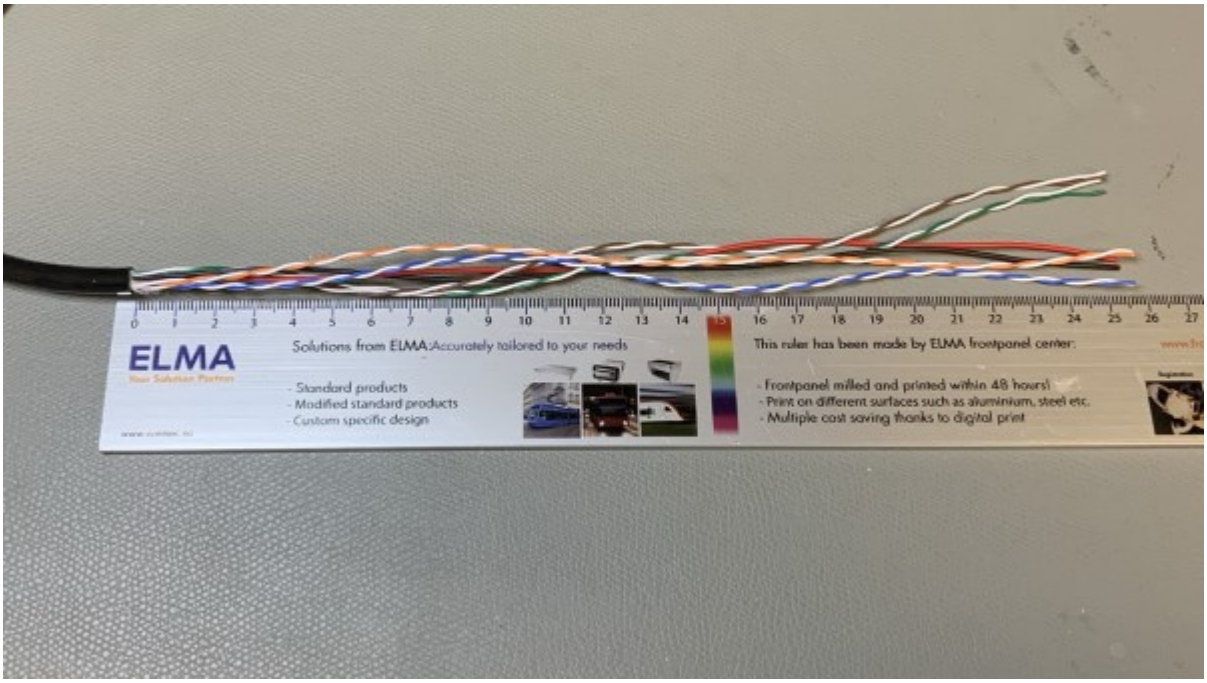
Using too much force or heat may delaminate the I/O Interface PCB, rendering the I/O Interface board useless.

Adding a penetrator

To add a penetrator to the cable you will need:

- Plug Wrench from Blue Robotics
- M10-7.5mm-HC WetLink Penetrator from Blue Robotics
- (Optional) Cable Jacket Stripper from Blue Robotics
- Utility knife

Strip off 25-30 cm of the cable jacket using a cable jacket stripper, a utility knife or a sharp blade. Take care not to cut into the leads inside the cable. Blue Robotics has a great guide on cable stripping here.



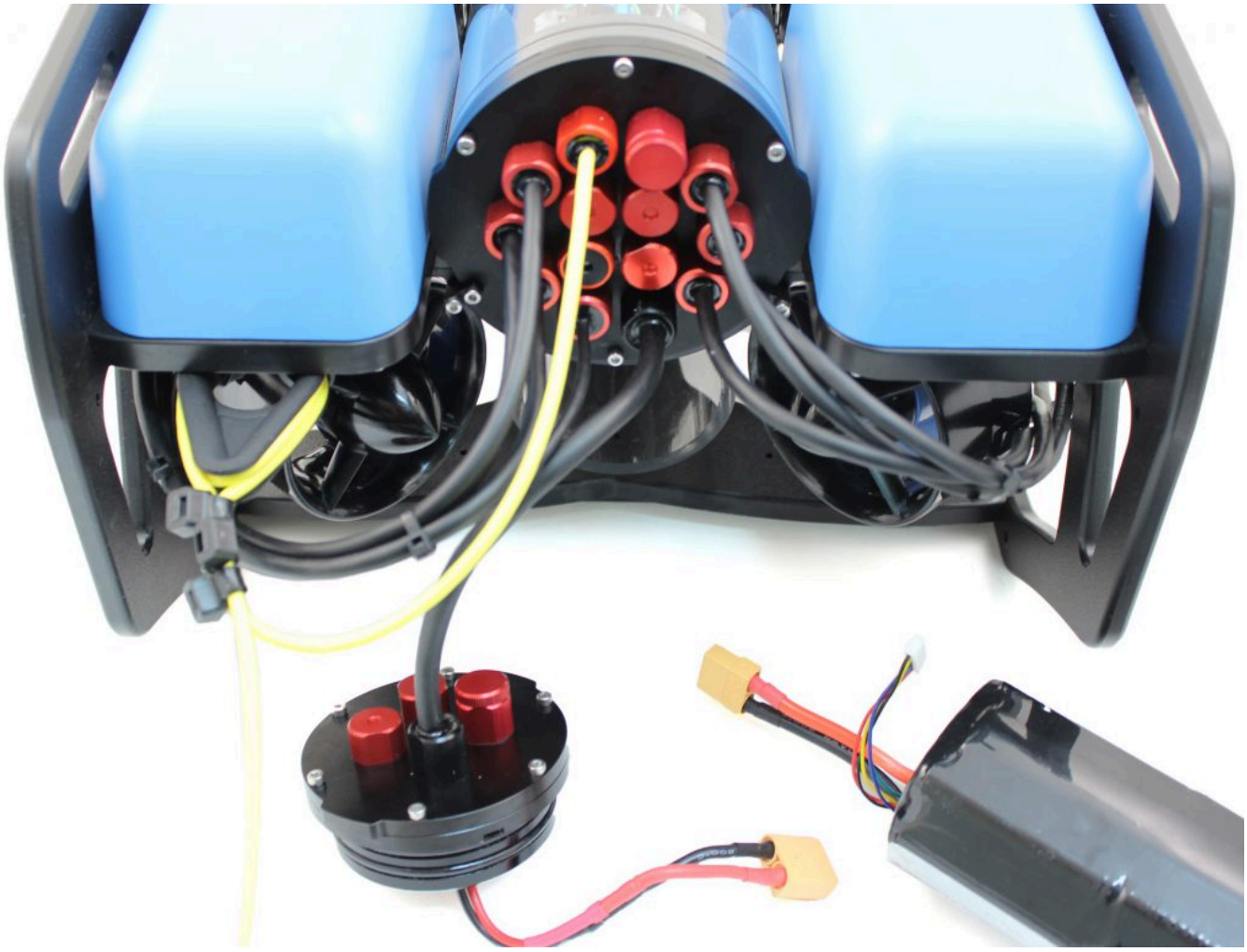
Follow the WetLink Penetrator Assembly Guide by Blue Robotics to add a penetrator on the DVL A50 cable.

Removing a blank penetrator

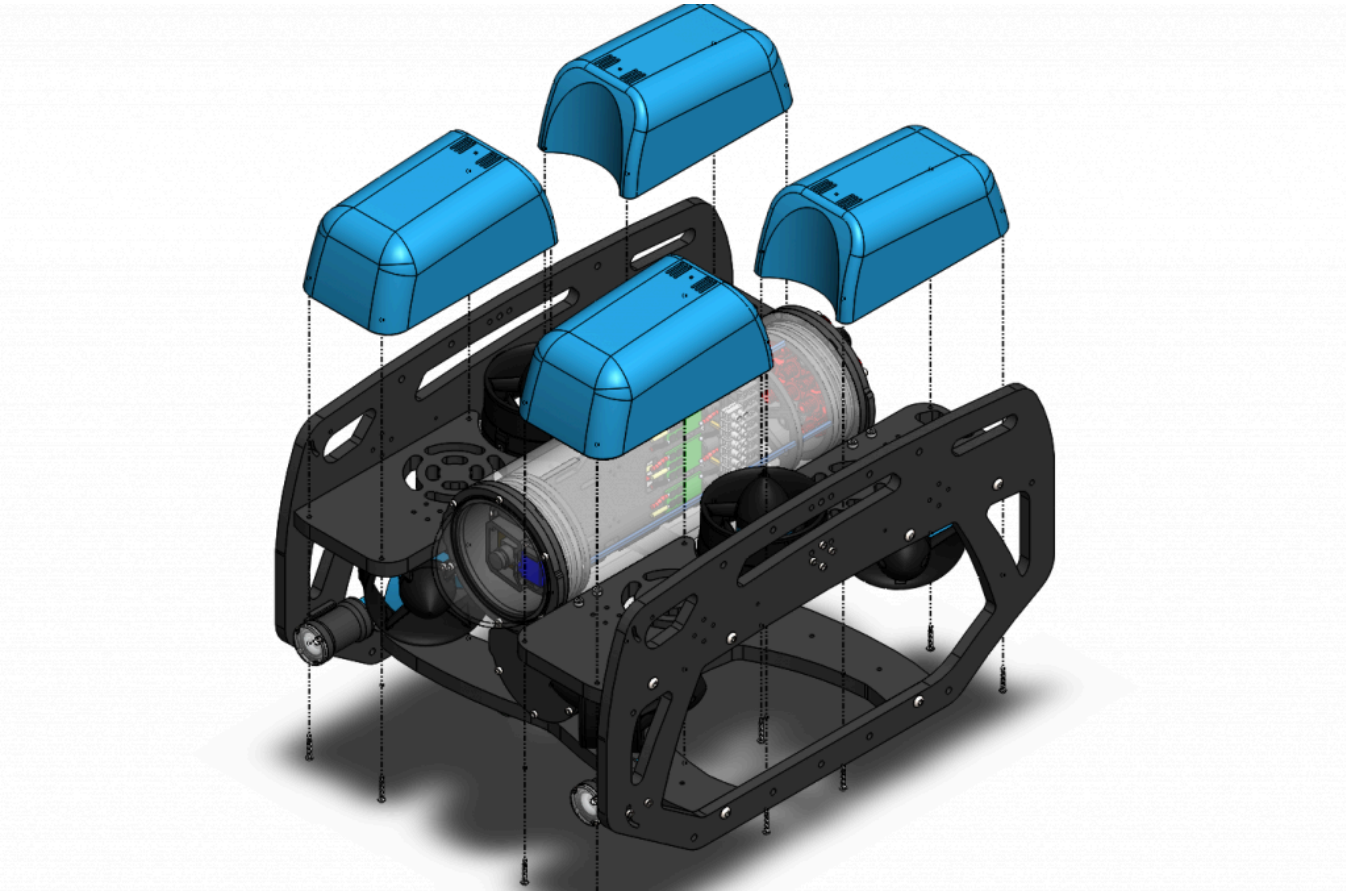
To remove a blank penetrator from your BlueROV2, you will need the following tools:

- 2.5 mm hex driver
- Bulkhead Wrench from Blue Robotics

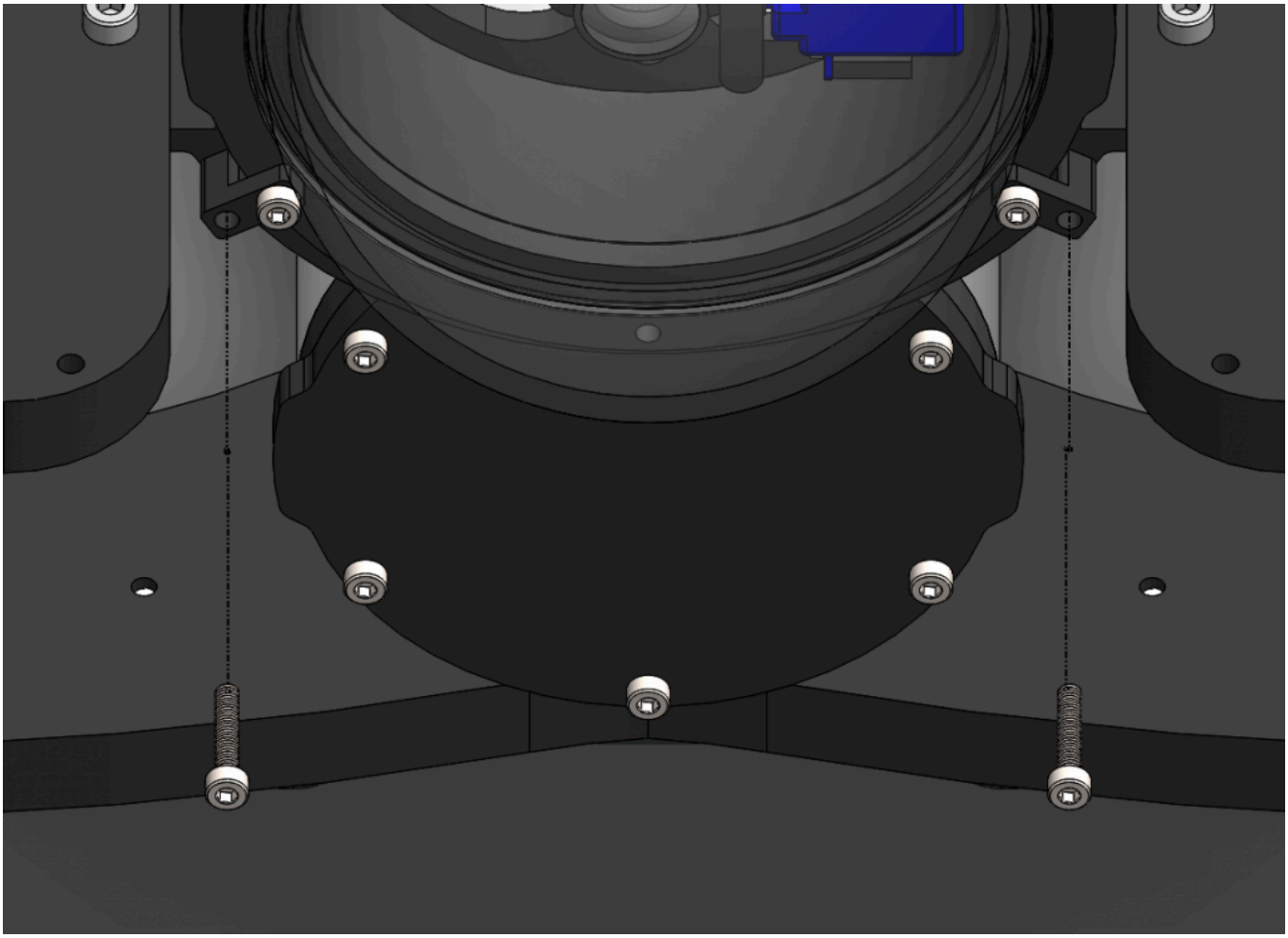
To ensure your ROV is completely powered off, please remove the battery completely from the 3” enclosure and place to the side.



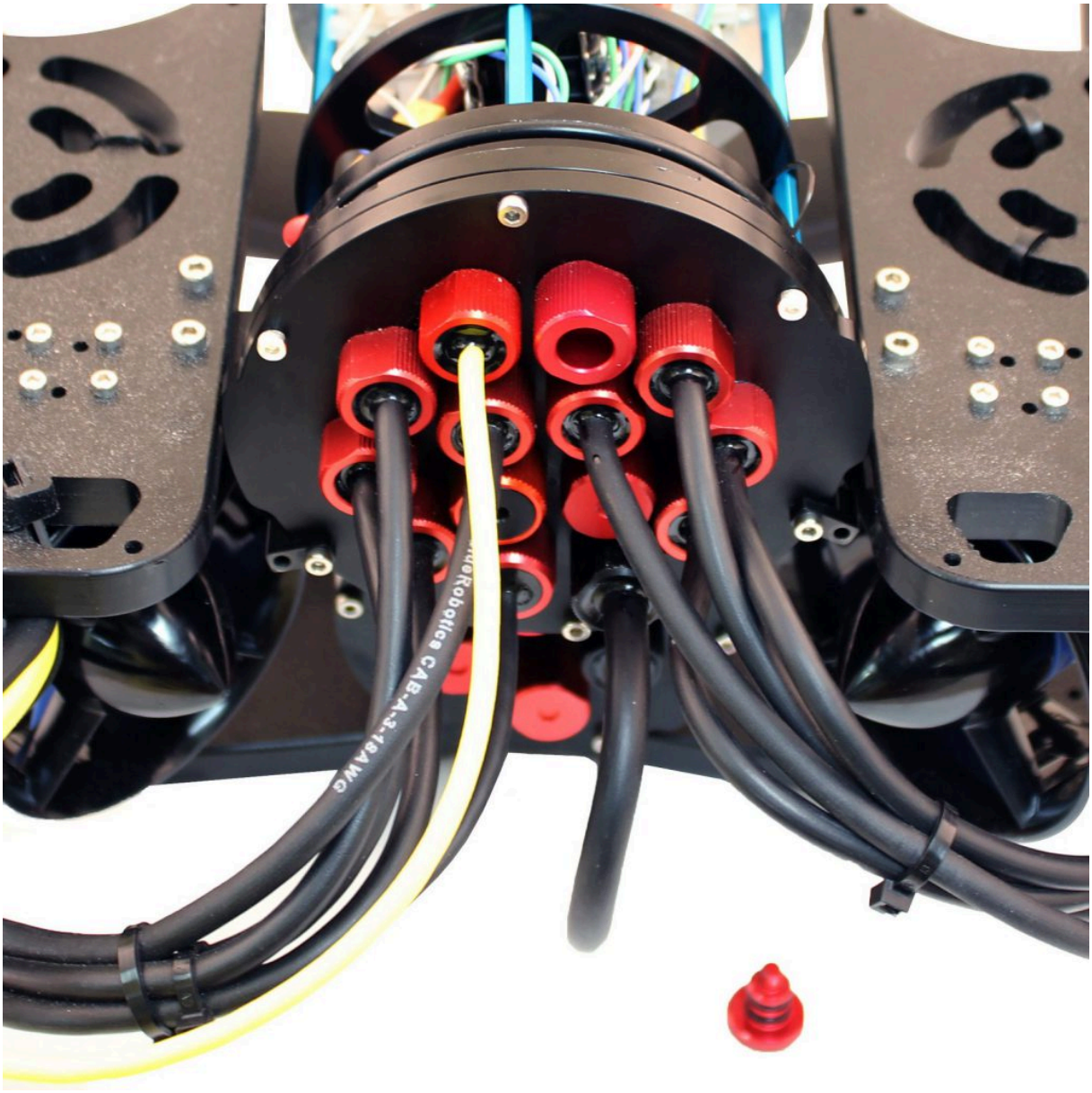
(Optional) Remove the fairings and buoyancy blocks by removing the self-tapping screws that hold the fairings to the frame.



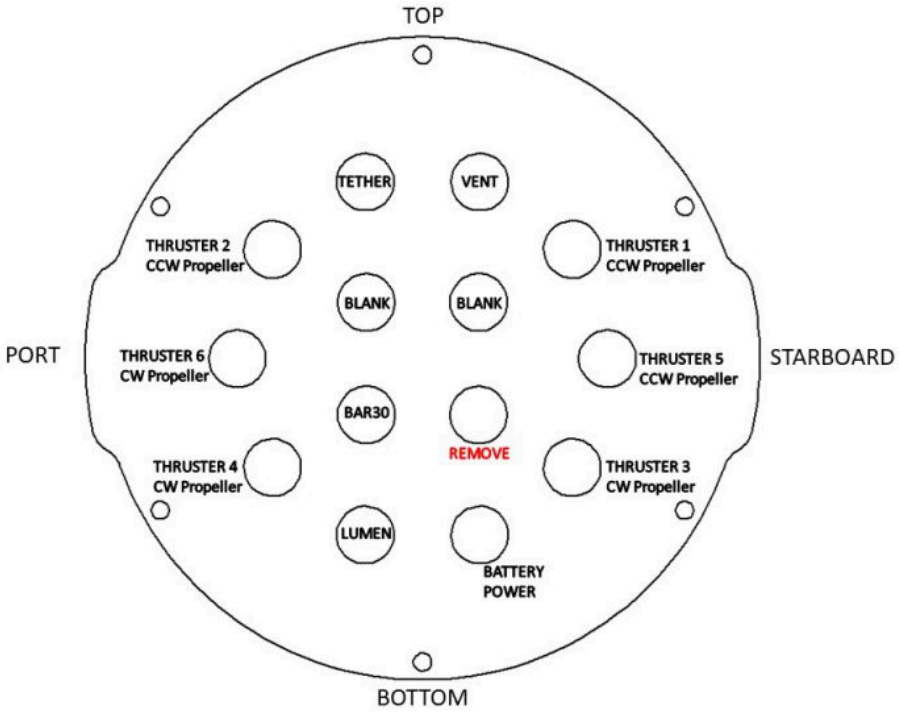
Remove the 4” electronics enclosure from the ROV by removing the M3x16 screws that mount the enclosure to the ROV cradle.



Remove the Vent Plug from the Vent Penetrator Bolt on the electronics enclosure. Remove the 4″ tube and forward dome assembly from the rear end cap.



Remove a blank penetrator, such as the one pictured below, from the 4” End Cap with the penetrator wrench.



Install DVL A50 penetrator

Note

The following images show the old penetrator from Blue Robotics. The procedure is the same with the new WetLink penetrators.

To install the DVL A50 into the end cap, you will need the following parts and tools:

- DVL A50 with installed cable penetrator
- Penetrator O-ring (included with M10-7.5mm-HC WetLink Penetrator from Blue Robotics)
- Bulkhead Wrench from Blue Robotics
- Silicone Grease
- Isopropyl Alcohol Wipe

Wipe the exterior surface of the electronics enclosure end cap clean with isopropyl alcohol or isopropyl alcohol wipes, and make sure it is free of any particles in the areas where the penetrator O-ring will sit.

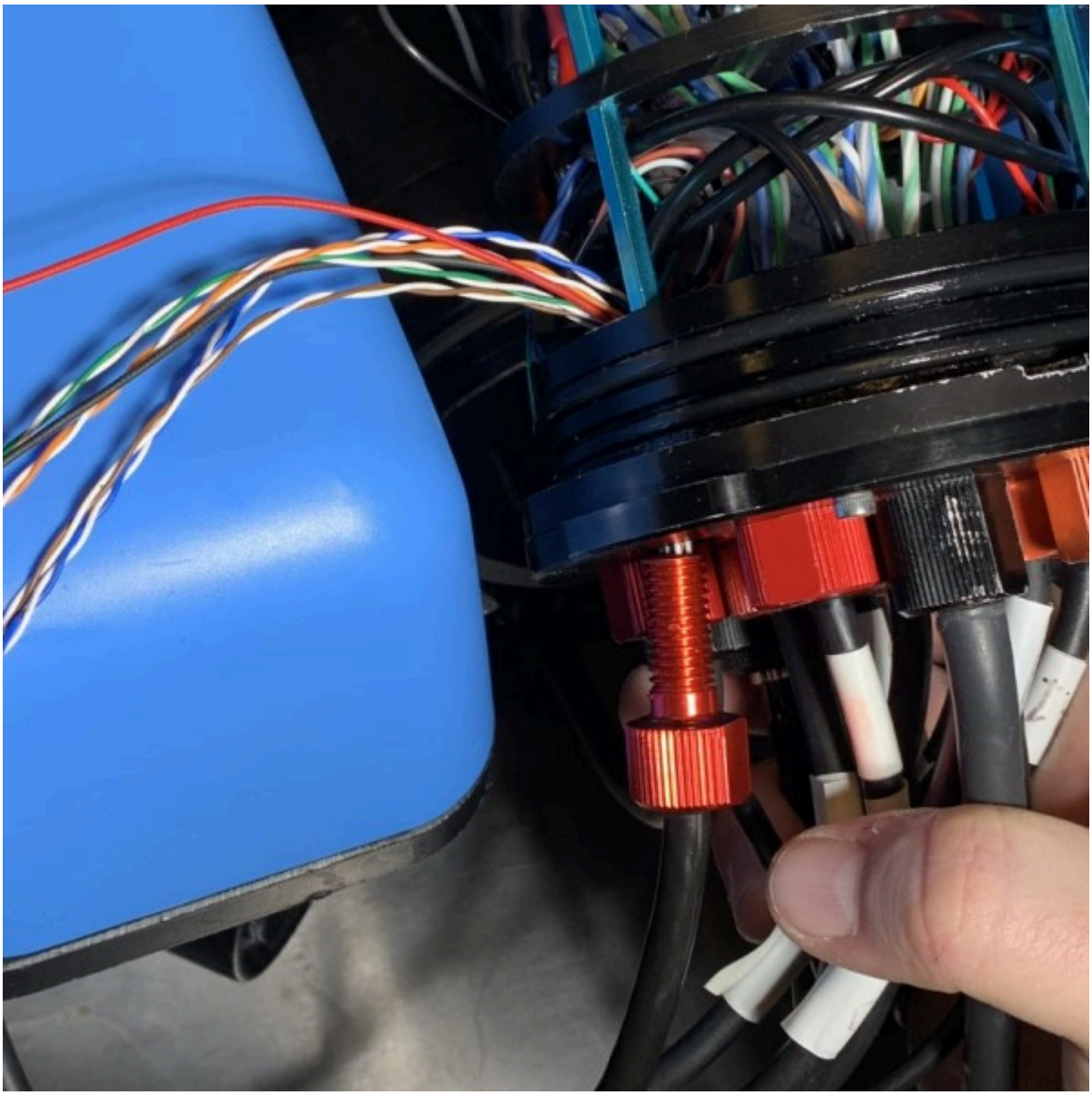
Remove the O-ring from the bag

Install the O-ring onto the DVL A50 cable penetrator and apply silicone grease to it.



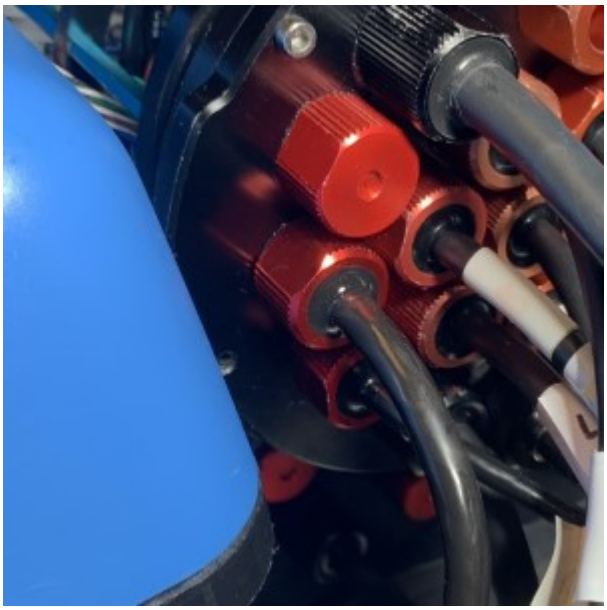
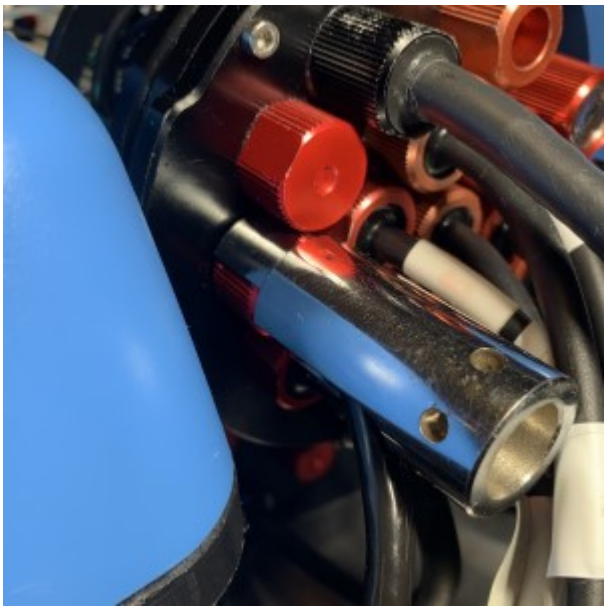


Push the wires through the hole in the end cap where you previously removed a blank penetrator.



> **Tip**
>
> Since the penetrator has to be screwed on, adding a bit of tape around the end of the wires helps them stay untangled while turning. Twisting the DVL A50 cable clockwise will also help when screwing on the penetrator.

Screw the DVL A50 cable penetrator onto the end cap. Tighten to finger tight, then use the penetrator wrench to tighten it an additional ~1/16 of a turn. If you can't loosen it with your fingers, it is tight enough.

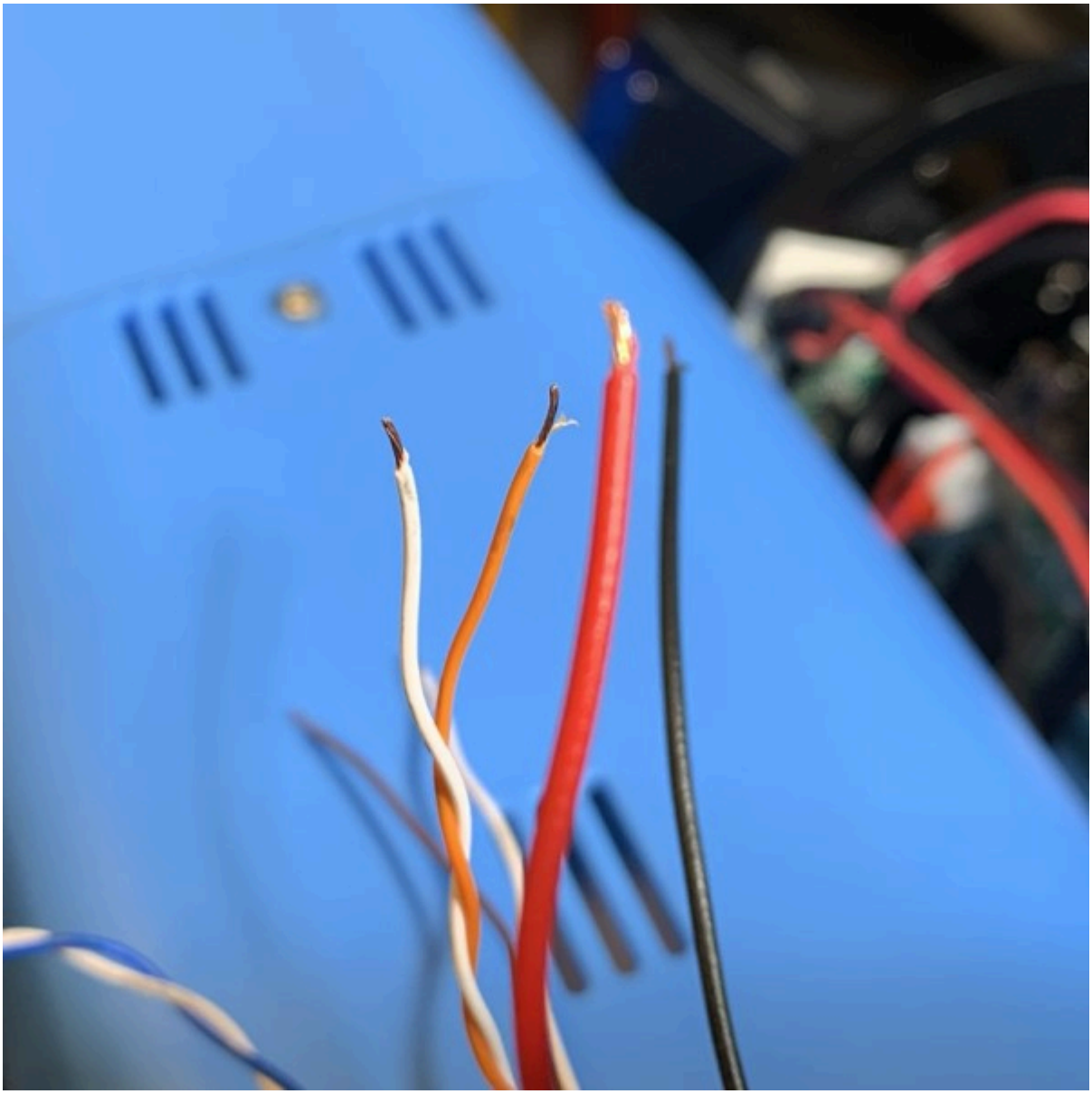



Reinstall I/O Interface Board

To reinstall the I/O Interface board onto the DVL A50 cable, you will need:

- Wire stripping tool
- Soldering iron
- Solder wire
- Zip tie

With the DVL A50 penetrator now through the 4" End Cap, strip off 2-3 mm of the ends of all the wires. Wire sizes are detailed in the table below. The blue pair can be removed because it is not used for the DVL A50.



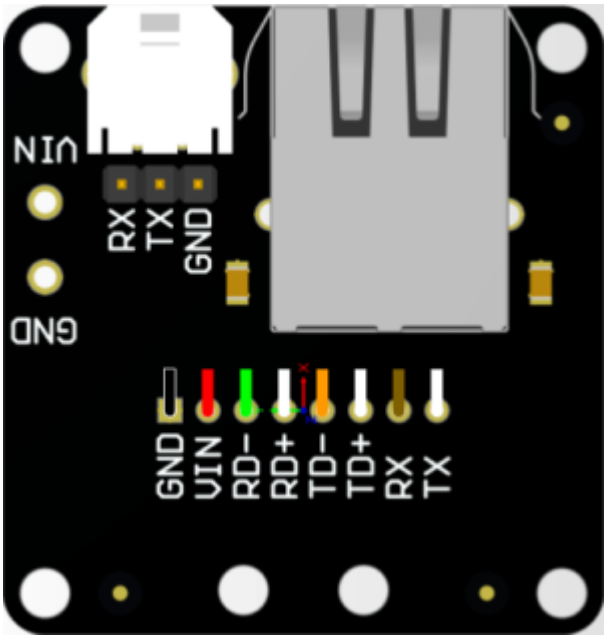
Warning

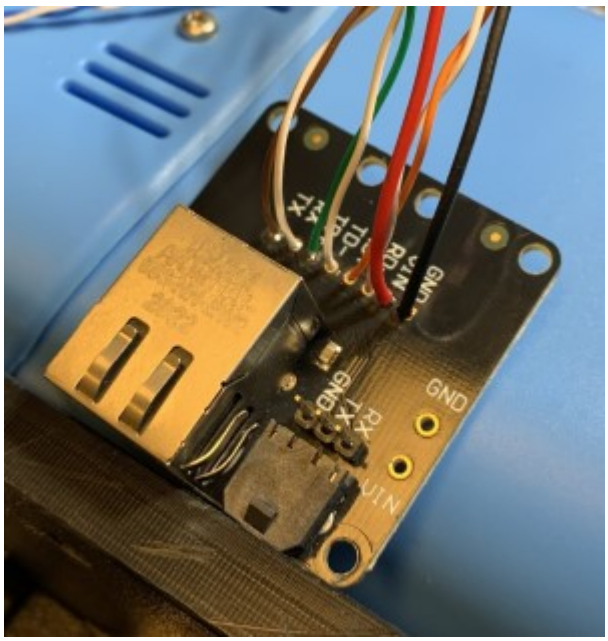
The two wires of a twisted wire pair are fully colored and fully white. The table below shows the white wires with a stripe of the complementing color, but this is only for reference. Keep track of which white wire is paired with which colored wire. The wiring in the images below also differs from the table below. Please follow the table.

Re-solder the leads of the DVL A50 cable to the I/O Interface board following the table below:

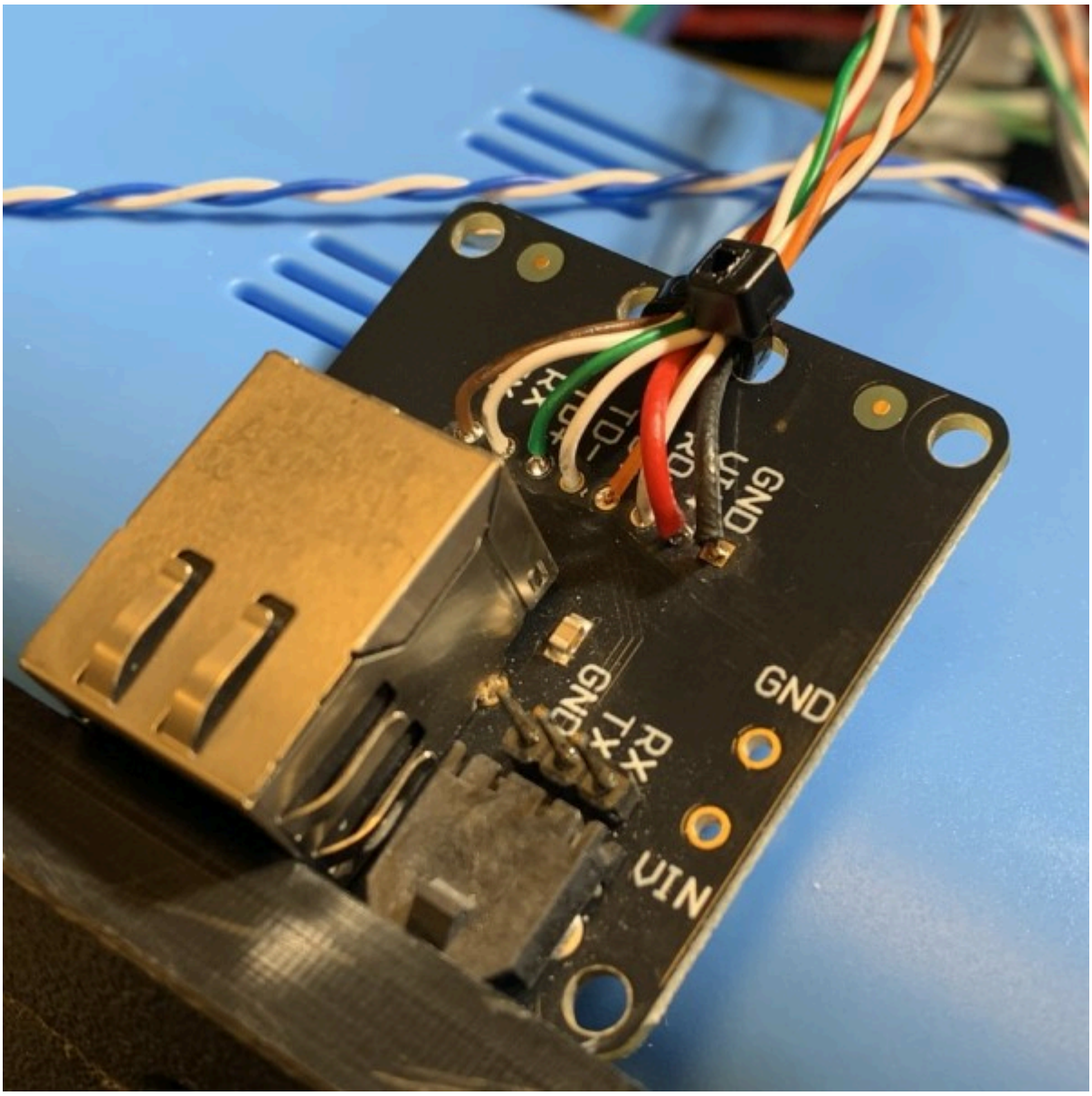
DVL A50 cable	I/O Interface pad	Function	Diameter (mm)	AWG#
	GND	Power ground	0.6 mm	22

DVL A50 cable	I/O Interface pad	Function	Diameter (mm)	AWG#
	VIN	Power in	0.6 mm	22
	TD+	Ethernet TX+	0.6 mm	22
	TD-	Ethernet TX-	0.4 mm	26
	RD+	Ethernet RX+	0.4 mm	26
	RD-	Ethernet RX-	0.4 mm	26
	TX	Serial UART TX	0.4 mm	26
	RX	Serial UART RX	0.4 mm	26





Zip tie the soldered wires to the I/O Interface board to protect the solder joints.

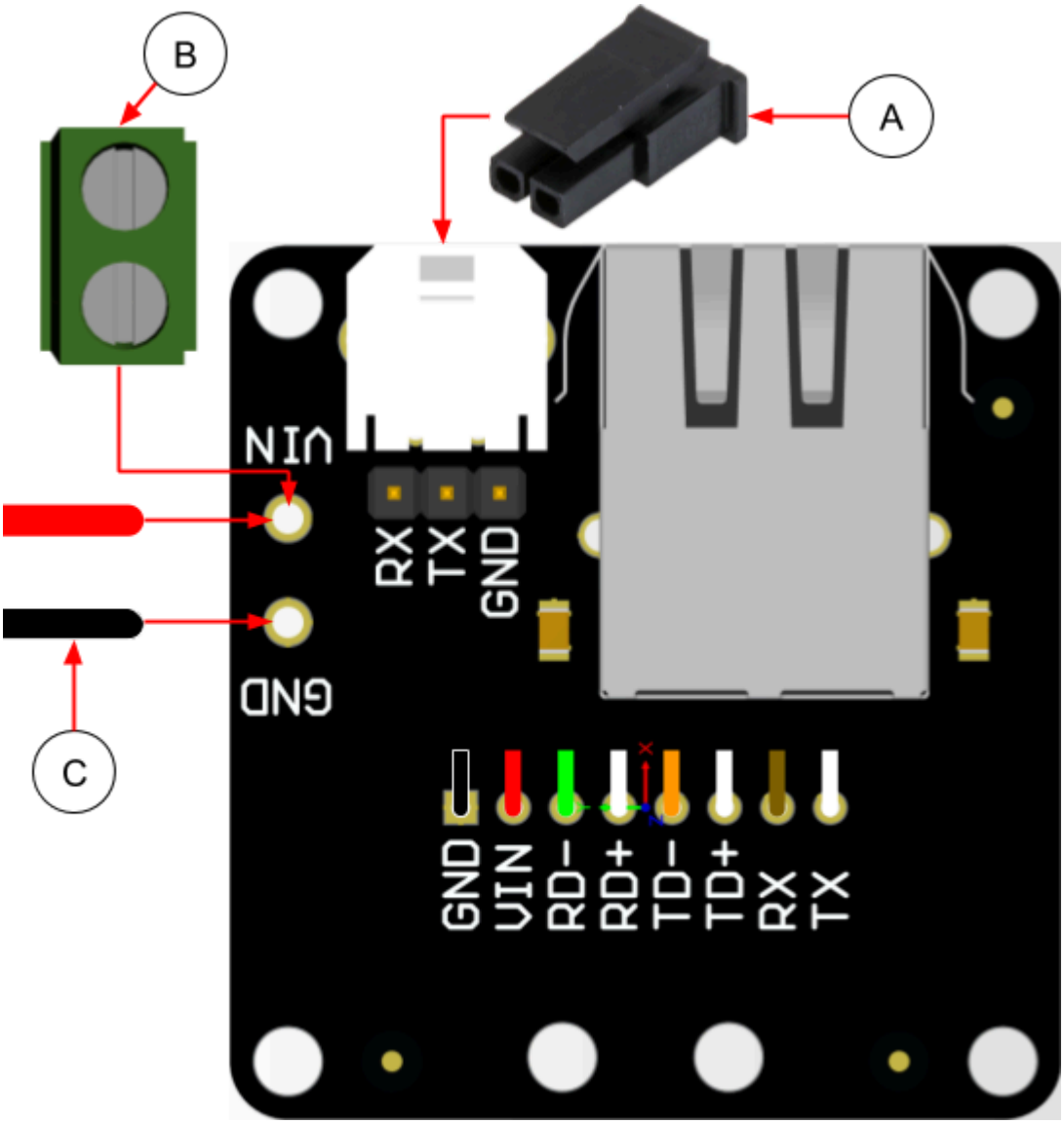


Connect to BlueROV2

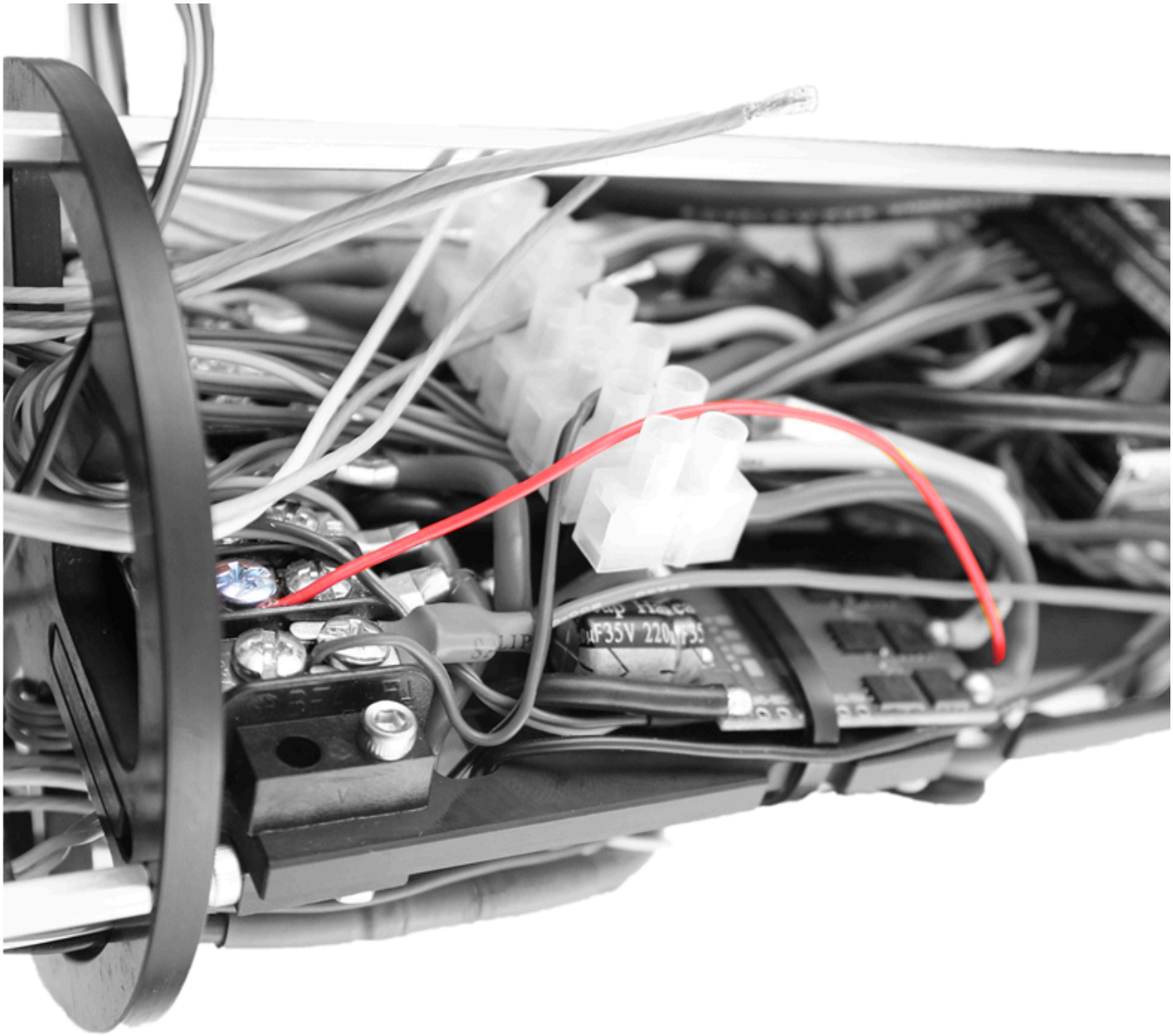
To connect the DVL A50 to the BlueROV2, you will need:

- 2 x 30 cm (12") wires for power to the DVL A50
- Blue Robotics Ethernet switch
- 6" Ethernet cable
- Zip ties

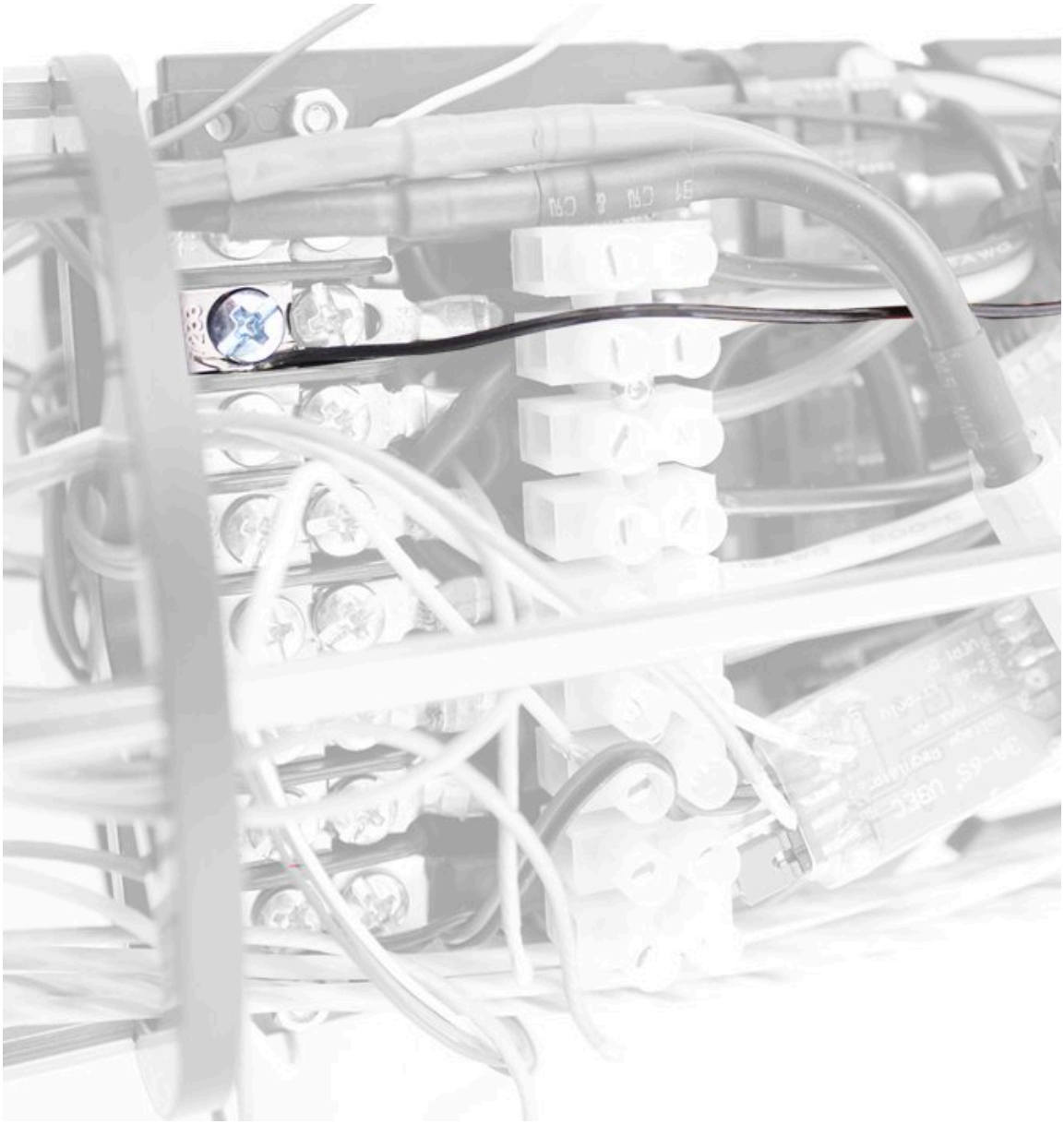
To provide power to the DVL A50, you can either use (A) a 2-pin Molex MicroFit3.0 cable assembly, (B) solder on a Würth screw terminal, or (C) directly solder on 2 wires to the power pads.



- Connect VIN to the positive Power Terminal Block inside the BlueROV2 by either stripping off 6-8 mm of the wire and screw it in place, or add a spade terminal for a # 6 screw (M3.5) and screw in place.



- Connect GND to the negative Power Terminal Block by either stripping off 6-8 mm of the wire and screw it in place, or add a spade terminal for a # 6 screw (M3.5) and screw in place.



Warning

The Power Terminal Blocks are unregulated and directly connected to the battery. Any short may generate several amps, which will damage most circuits. Check that the wires are secured properly at both ends.

Install the Blue Robotics Ethernet switch following the Ethernet Switch Installation Guide for the BlueROV2 by Blue Robotics.

With power and the Ethernet switch now sorted, connect a 6" Ethernet cable from the Ethernet switch to the I/O Interface board.

Tip

Some thick double-sided sticky tape and zip ties can be used to secure the I/O Interface to the electronics frame inside the BlueROV2.

Reassemble BlueROV2 electronics enclosure

To reassemble your BlueROV2 Electronics Enclosure, you will need the following parts and tools:

- 4 x M3x16 screws that were placed off to the side during disassembly
- Silicone Grease
- 2.5 mm hex driver

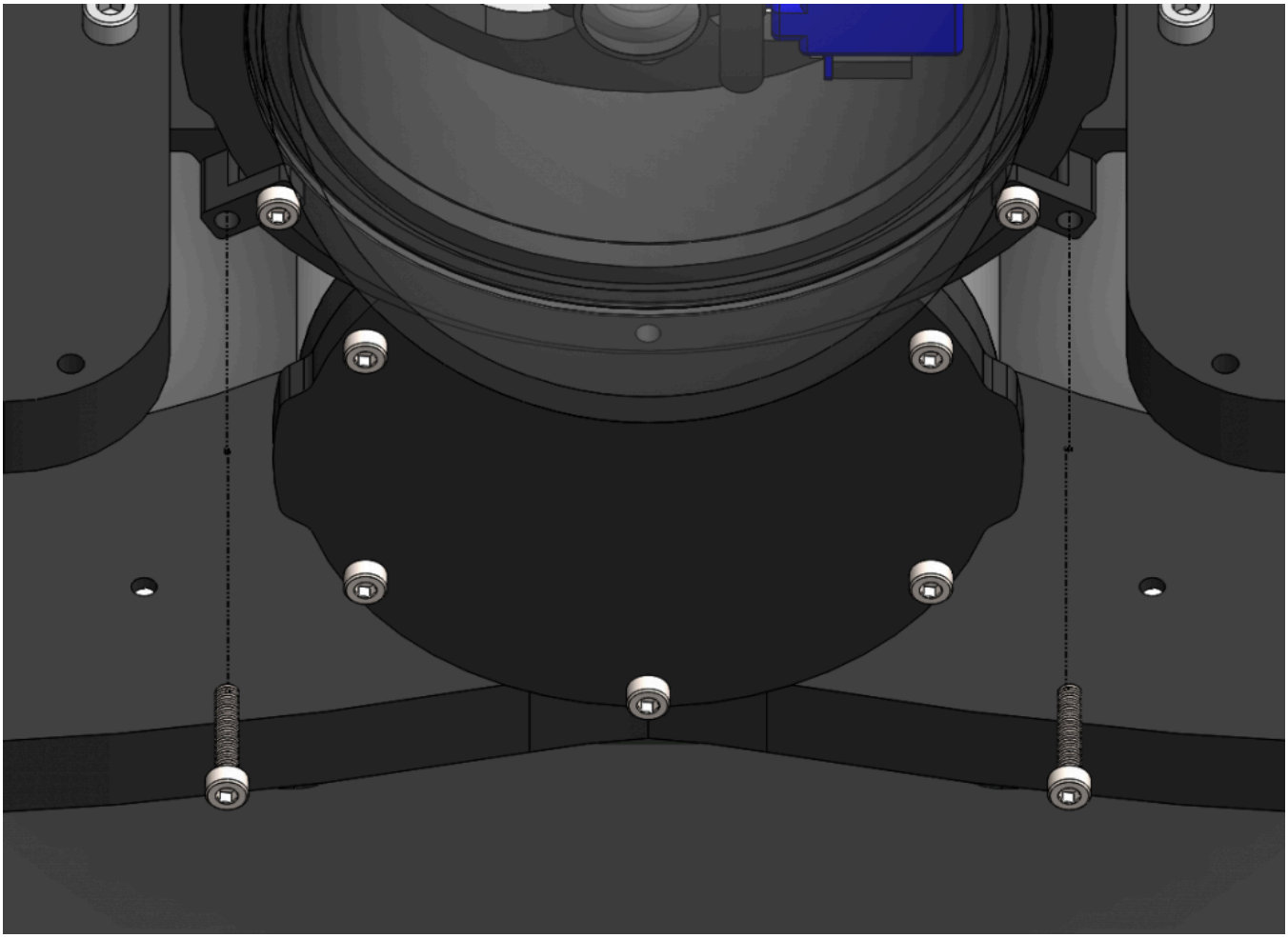
Reinstall 4” Watertight Enclosure onto ROV with the following steps:

- Apply silicone grease to the two radial O-rings on the O-Ring Flange (4” Series) that is attached to the Electronics Tray.
- Install the Watertight Enclosure (4” Series) with installed Dome End Cap to the O-Ring Flange (4” Series).

Warning

Use caution when sliding the 4" Watertight Enclosure over the electronics. Space may be tight, and wires can get trapped between the 4" tube and the 4" end cap.

Mount the Electronics Enclosure to the frame using the M3x16 screws so that the dome is on the same side as the front center panels (the center panels without the 3 large holes). Install the M3x16 screws through the clips and into the Enclosure Cradle (4” Series). It is easier to install these screws if the clips are not fully tightened until all screws are through the clips and threading into the Enclosure Cradle (4” Series). This allows clips to rotate so you can find the threaded hole in the Enclosure Cradle (4” Series) easily.

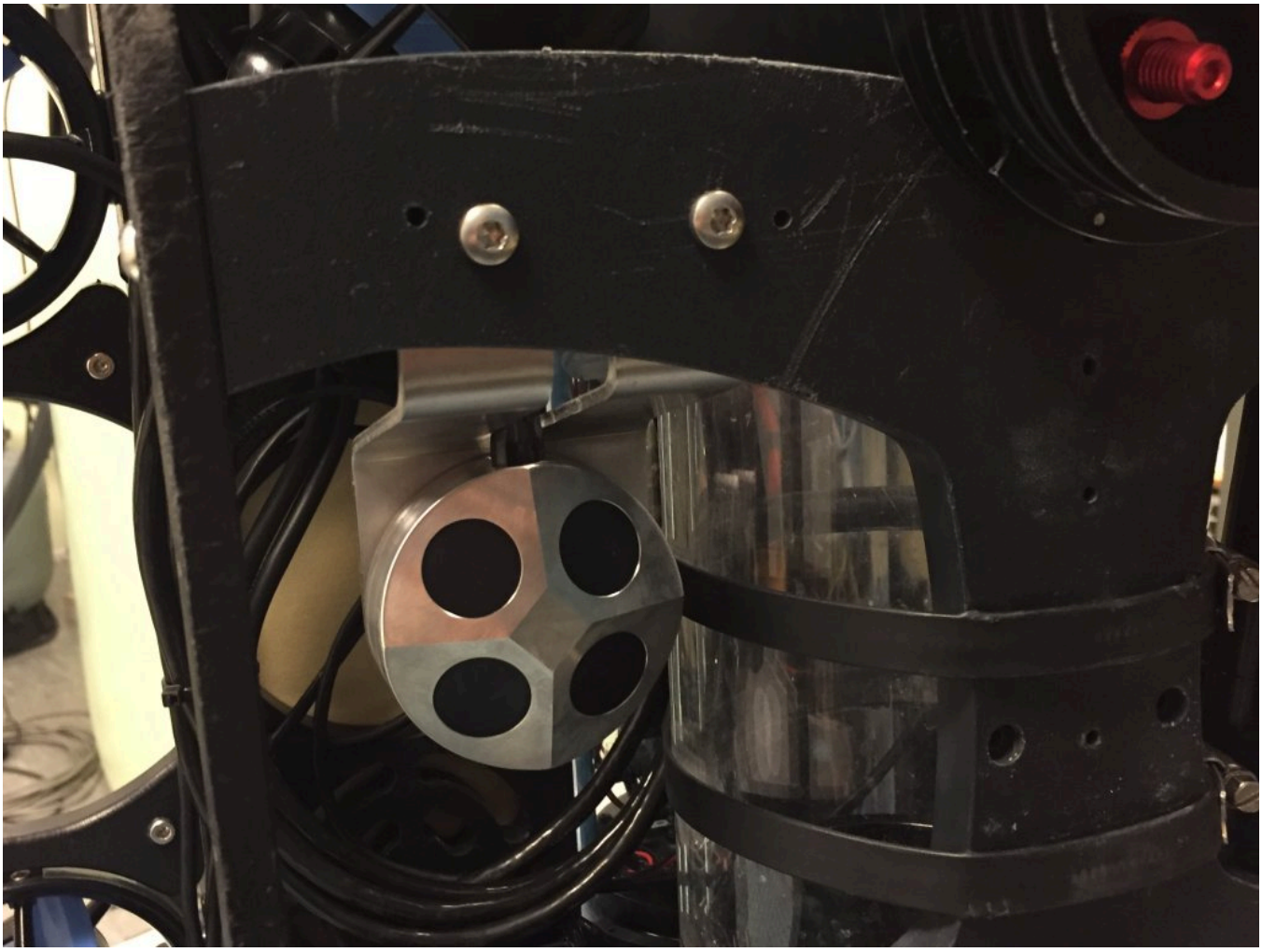


Attach DVL A50 to BlueROV2

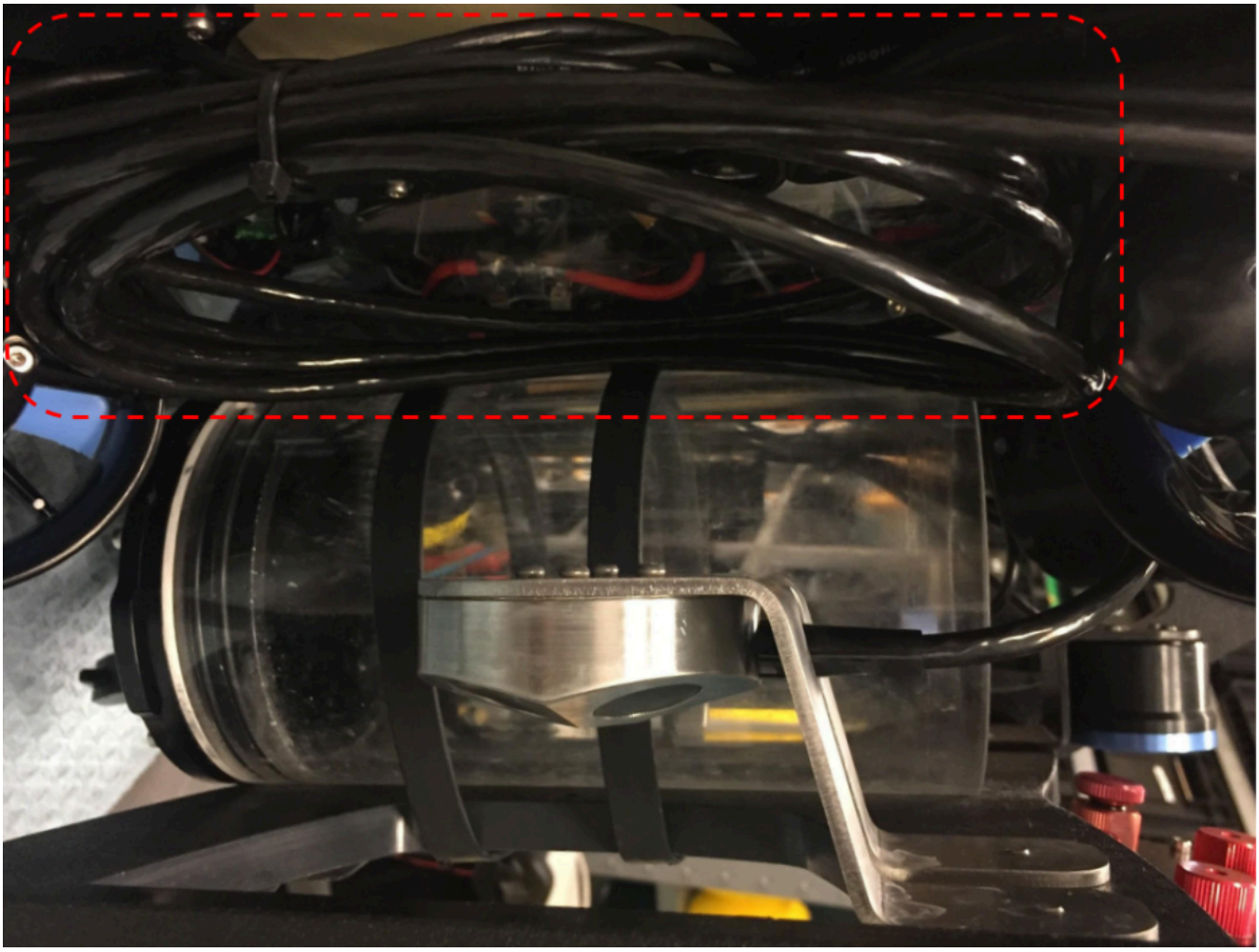
Note

The following images show the old version of the DVL A50 with steel housing. The new housing has the same dimensions as the old.

Attach the DVL A50 to the BlueROV2 using the 2 x M5x16 screws provided with the DVL A50 BlueROV2 Integration Kit through the two holes you made previously.



Excess cable can be bundled together and attached to the BlueROV2 frame using zip ties.



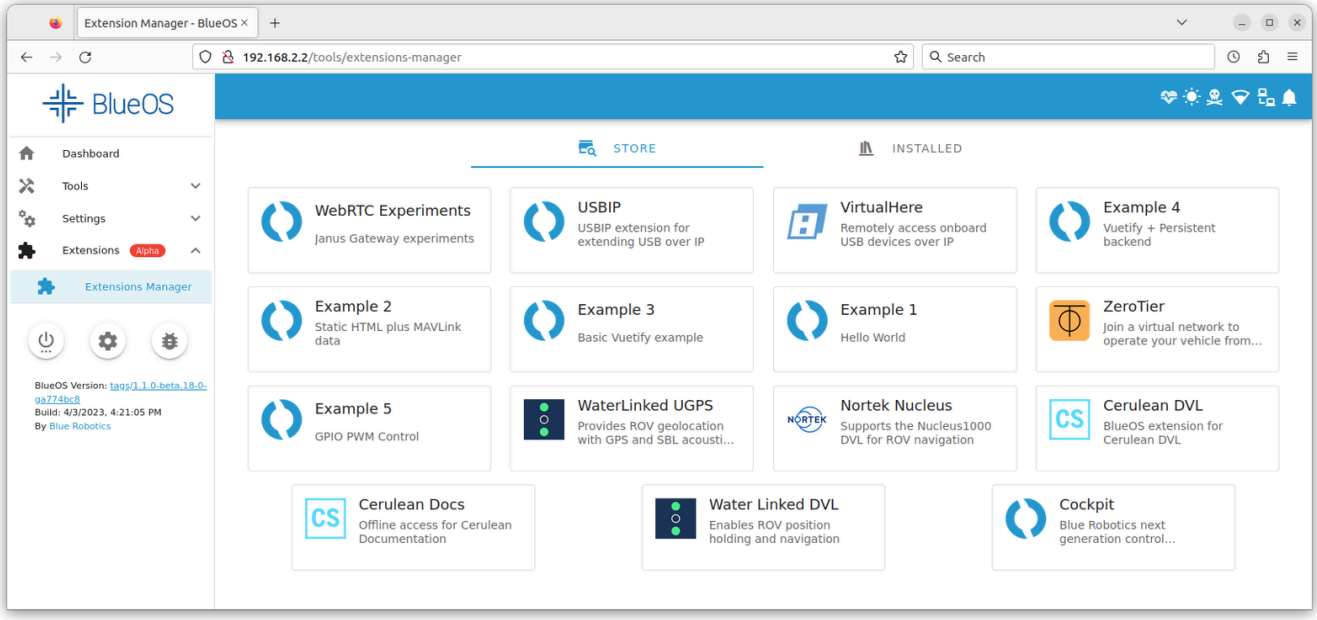
Software

The DVL A50 requires an extension in BlueOS to communicate with the autopilot (Pixhawk or Navigator flight controller). BlueOS has to be updated to **BlueOS 1.1.0** or newer to assure that extensions are supported sufficiently.

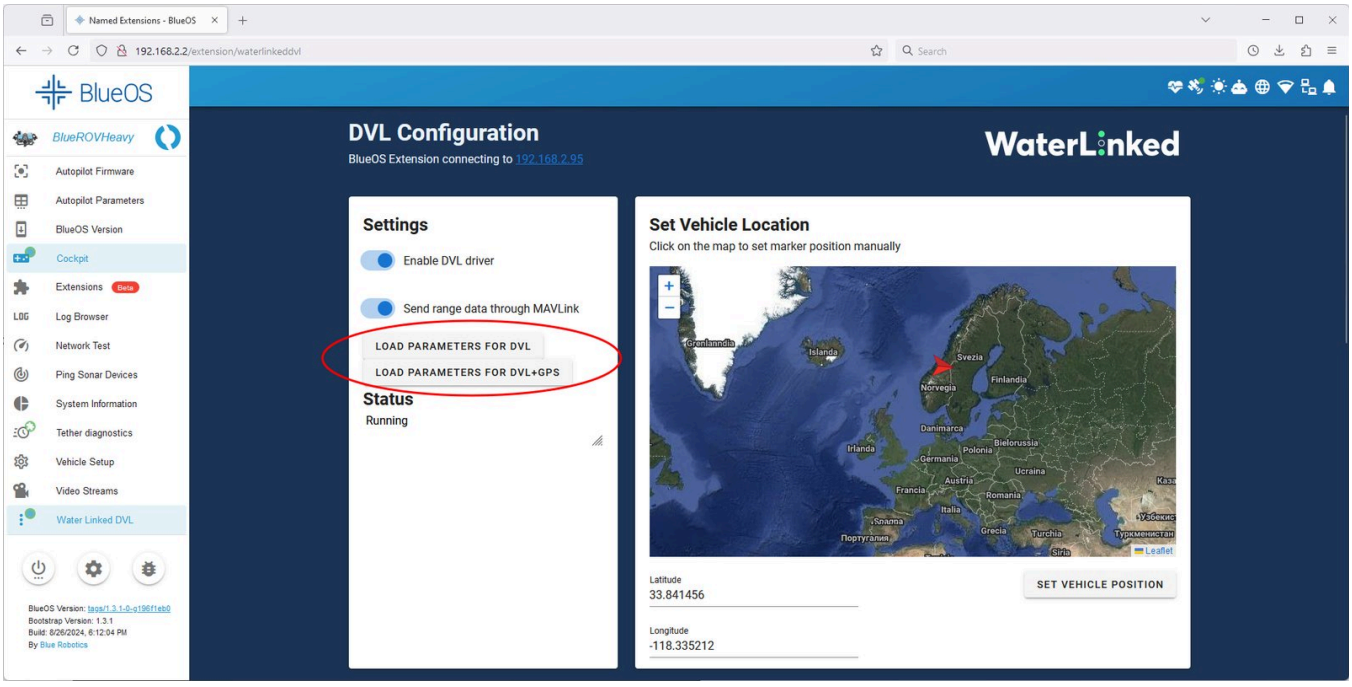
- After the update, install the latest BlueOS-Water-Linked-DVL extension from the BlueOS extension manager. It is best to disable the BlueOS "Water Linked UGPS" extension if you have that installed.

Note

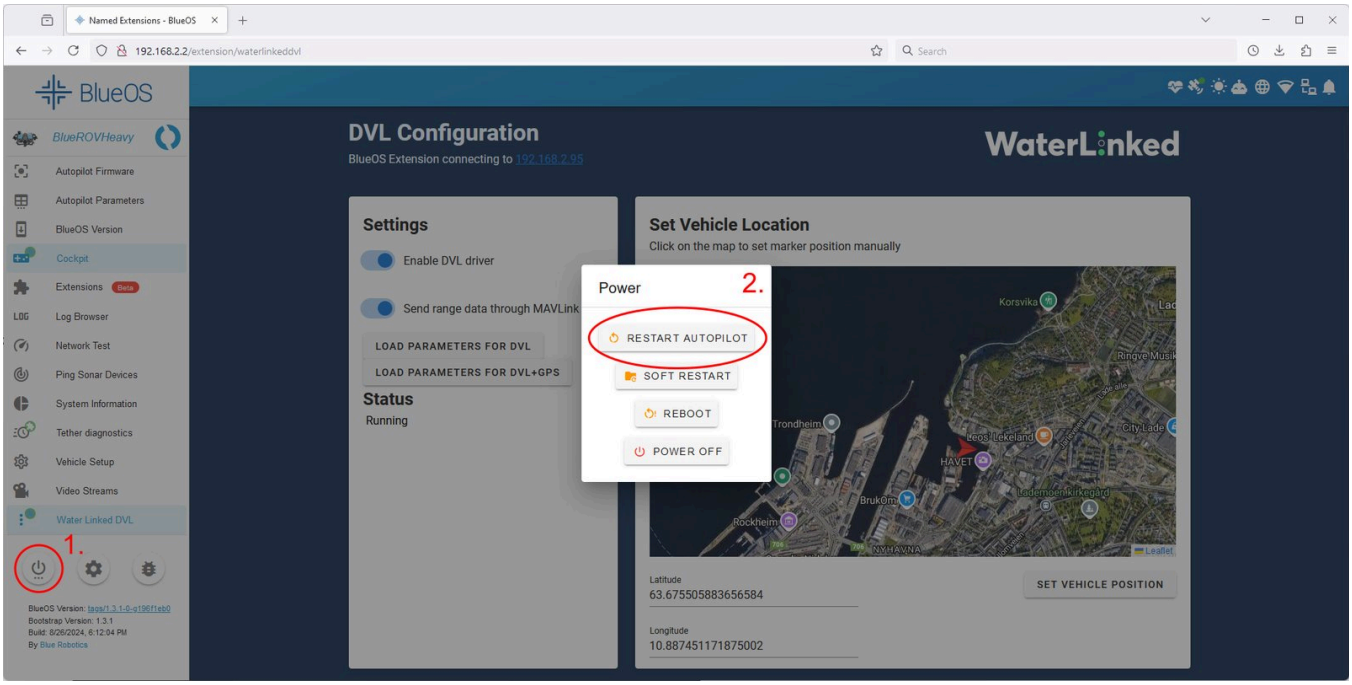
If you want to use both DVL and UGPS, see the DVL with UGPS for BlueROV2 guide.



- Open the interface of the DVL extension and click the button "Load parameters for DVL". Those parameters are persistent over reboots, which means that this button only needs to be pressed once and not every time you restart the ROV.



- Restart the autopilot for the parameter changes to take effect. It is not necessary to restart BlueOS.



- Set the global vehicle position in the DVL extension every time you start the ROV.

> **Note**
>
> There is a known bug where the I2C address of the pressure sensor is set wrong, and so depth is not received properly. This can be fixed by setting the parameter `BARO_PROBE_EXT` to `768` and then rebooting the ROV.

DVL with UGPS for BlueROV2

Optimizing performance when using DVL and UGPS together

Capabilities of the DVL as a Standalone System

The Water Linked DVL can provide accurate velocities relative to the seabed. It has a relatively high update rate and dynamic response. For the calculation of an absolute position, the velocities have to be integrated over time with the help of inertial measurement unit (IMU) data. Furthermore, the user has to specify a starting position for this dead-reckoning algorithm. Alternatively, a normal GPS can be used to initialize the position before a dive.

One advantage of using a DVL only is that no infrastructure outside the ROV (such as antennas, beacons, etc.) is needed. As a downside, small errors in sensor values accumulate over time and the position estimate drifts from the actual position. A typical use case for the DVL is station keeping even in water currents. It also helps to track the approximate ROV position on a map and enables simple autonomous missions.

Capabilities of the Underwater GPS as a Standalone System

The Water Linked Underwater GPS is able to provide an absolute position below water. It is based on the combination of a standard GPS to measure the Topside position and the Short Baseline (SBL) acoustic positioning to determine the ROV/locator position relative to the Topside.

The characteristic of this combined position estimate is that it is global, with little to no drift over time under the right conditions. The update rate is in general lower compared to the DVL. A typical use case for using the Underwater GPS by itself is to visualize the ROV position on a map for manual control and finding a specific geolocation below water.

Enhanced Functionality with Combined DVL and UGPS Integration

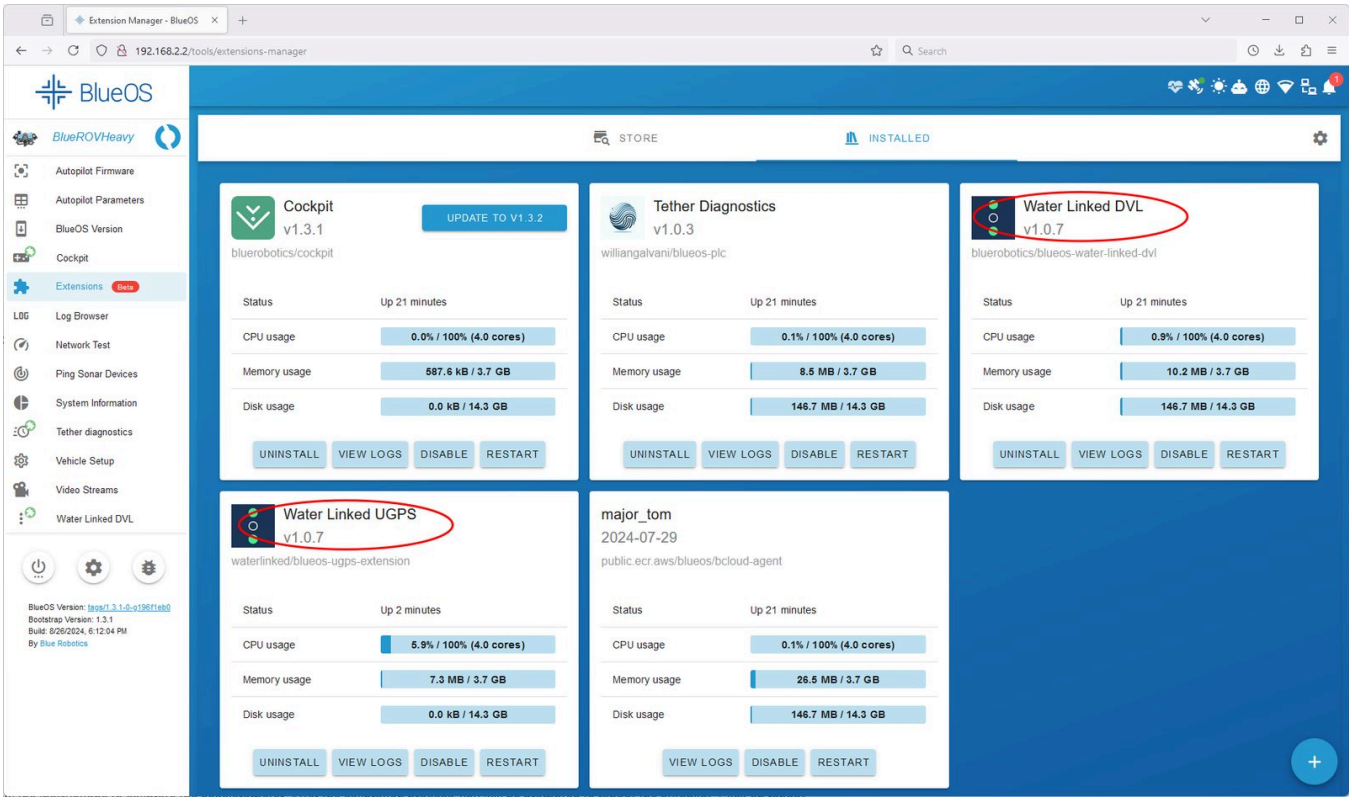
When both sensor types are combined well, they can compensate for each other's limitations. For instance, the UGPS helps compensate for DVL position drift over time. The DVL increases the position update rate, which is necessary for better control. This is particularly useful if you want to use the positioning not only for visualization during manual operation or position holding, but also for longer autonomous missions, such as waypoint navigation. The section "Mode DVL+UGPS" describes how to enable this in the BlueROV control system.

Mode "DVL+UGPS": Fusing the data of Underwater GPS and DVL for a position estimate of the BlueROV

The BlueROV uses a highly-customizable autopilot Ardusub. One part of that autopilot is an Extended Kalman filter, which is able to combine inputs from all types of sensors (gyroscope, accelerometer, compass, pressure sensor, GPS, DVL, etc.) to calculate a position estimate. This is then used to either simply show the position on a map in manual mode or as part of a feedback loop to actively control the ROV. This data fusion algorithm is not part of a Water Linked product but is executed in the BlueROV control system. So the user is in full control of the parameters and used algorithm.

Before successful sensor fusion of Underwater GPS and DVL, the following conditions must be met:

- A BlueROV running BlueOS with a version 1.1.0 (stable) or newer.
- The DVL is installed, configured, and tested successfully. Guides can be found here and on the Blue Robotics website
- Make sure you are running version 1.0.7 or newer of the BlueOS Water Linked DVL extension. The extension version can be checked / updated in BlueOS by going to Extensions > Installed. The version is shown next to the extension name, see below figure.



- The Underwater GPS is installed, configured, and tested successfully. The latest UGPS integration guide can be found here.
- Make sure you are running version 1.0.7 or newer of the BlueOS Water Linked UGPS extension.
- The accelerometer, gyroscope, compass and pressure sensor must be calibrated: Calibration of all sensors can be performed from QGroundControl. Detailed steps are described in the BlueROV Software Setup
- Make sure that the compass calibration was successful by comparing the ROV orientation shown in the map in QGroundControl to the actual ROV orientation. Landmarks like a coastline or an analog compass can help.

Note

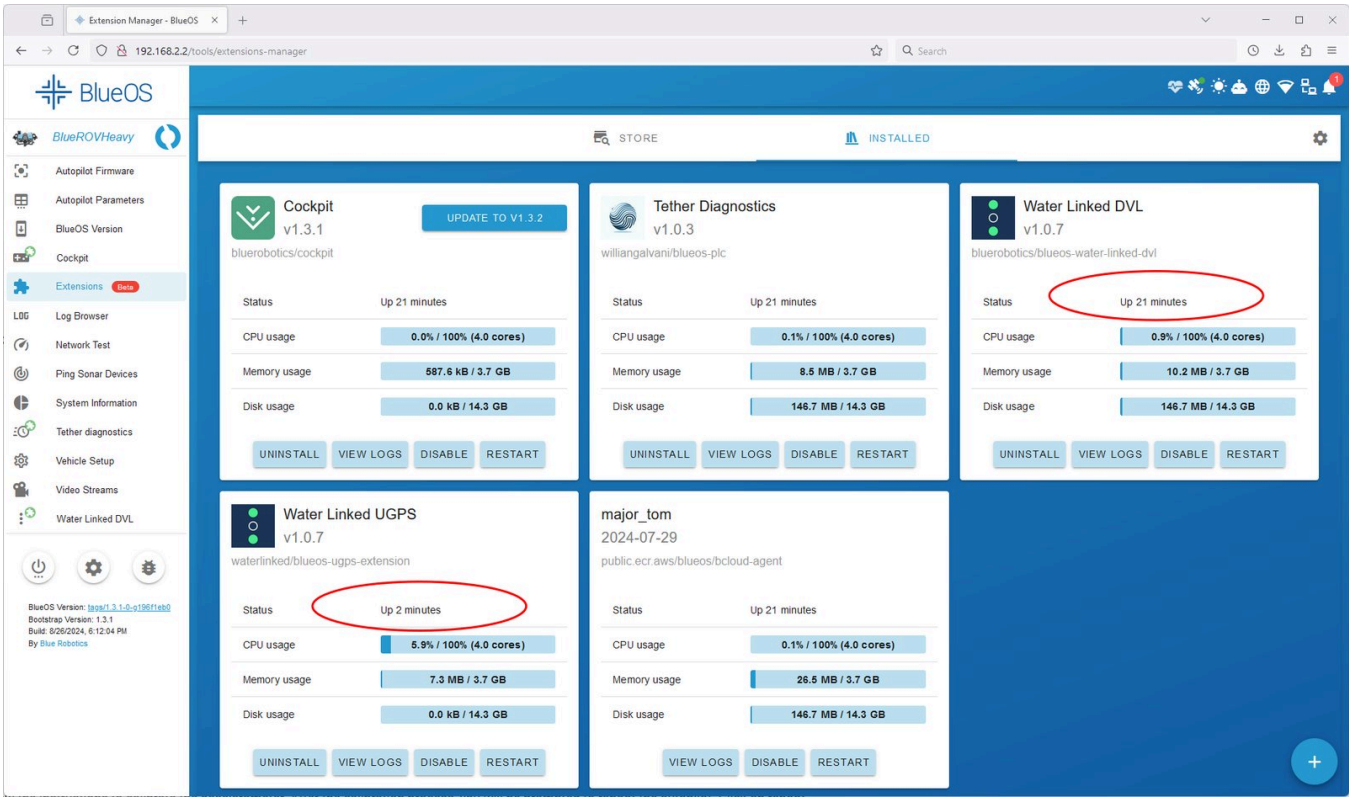
When you want to use the position estimate for *control* of the ROV and the UGPS Topside is on land, it is highly recommended to set up the Underwater GPS to use static position and static heading mode. Alternatively, when the UGPS Topside position is not static (on a boat or similar), you need to input heading and position data from an external GPS compass. Details can be found here.

If you don't follow this advice, the position provided by the Underwater GPS will have more noise ("jump" too much) and also drift over time. This might cause the position hold mode to not work properly.

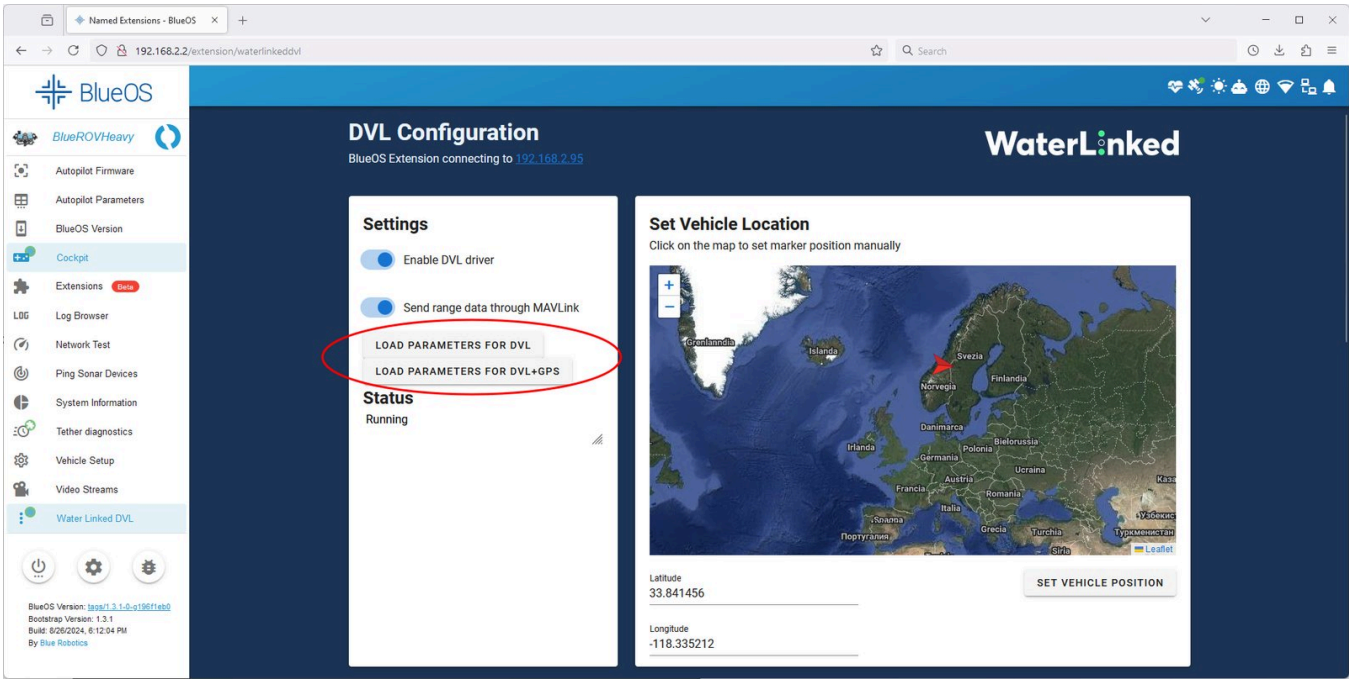
The Underwater GPS together with the BlueOS UGPS extension provide a standard deviation of the global position to the autopilot and the Kalman filter. The Kalman filter uses this standard deviation to weight the provided position in relation to other sensor inputs. The standard deviation is currently optimized for static position and static heading mode.

The steps to enable sensor fusion are:

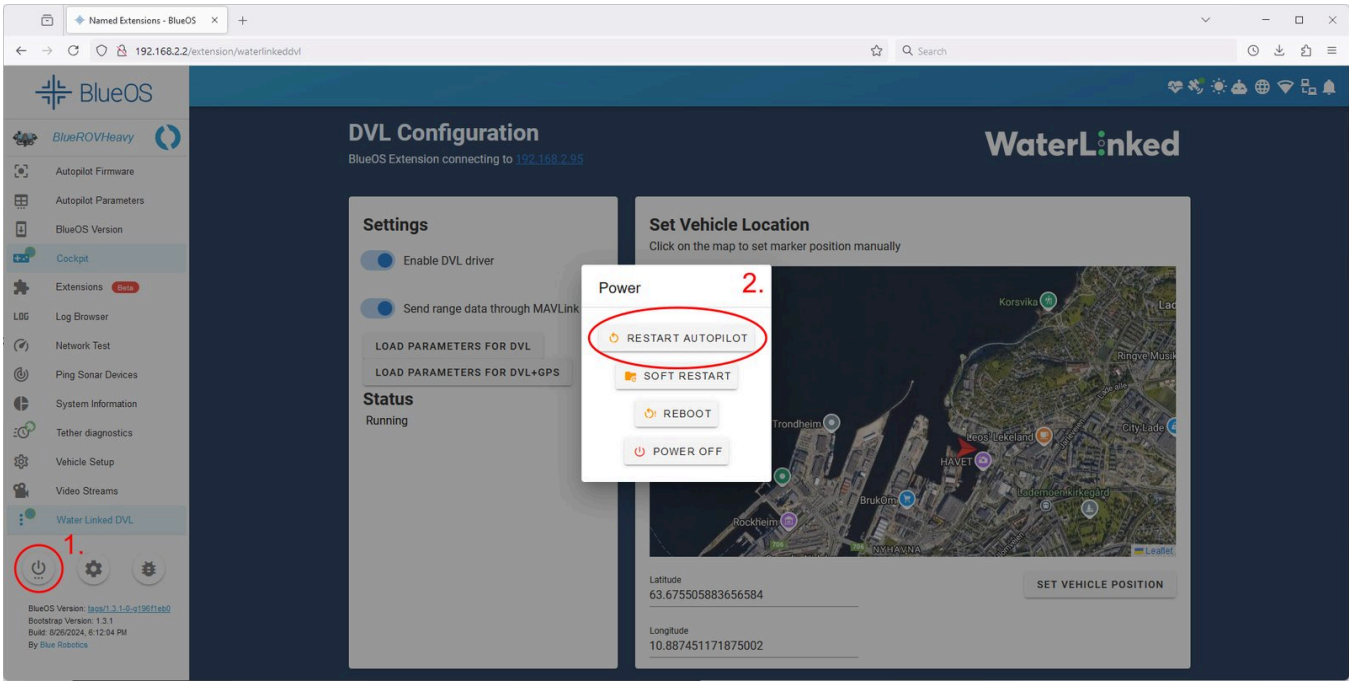
- To enable fusion of Underwater GPS and DVL, enable both the "Water Linked DVL" extension and "Water Linked UGPS" extension in the extensions manager. You see that they are running when an uptime is shown.



- Open the interface of the DVL extension and click the button "Load parameters for DVL+UGPS". This will set parameters for the Kalman filter to consider both sensors. Those parameters are persistent over reboots, which means that this button only needs to be pressed once and not every time you restart the ROV.



- Restart the autopilot for the parameter changes to take effect. It is not necessary to reboot BlueOS.



- If both Underwater GPS and DVL are connected, it is not necessary to set the vehicle position manually in the DVL extension.

Now both sensors should be considered for the position estimate of the ROV. You can test this in a simple way by switching the ROV to position hold mode around close (e.g. 3 m) to the UGPS-antenna and in sight. The ROV should stand still. When you now slowly turn the UGPS antenna, the ROV should follow your rotation.

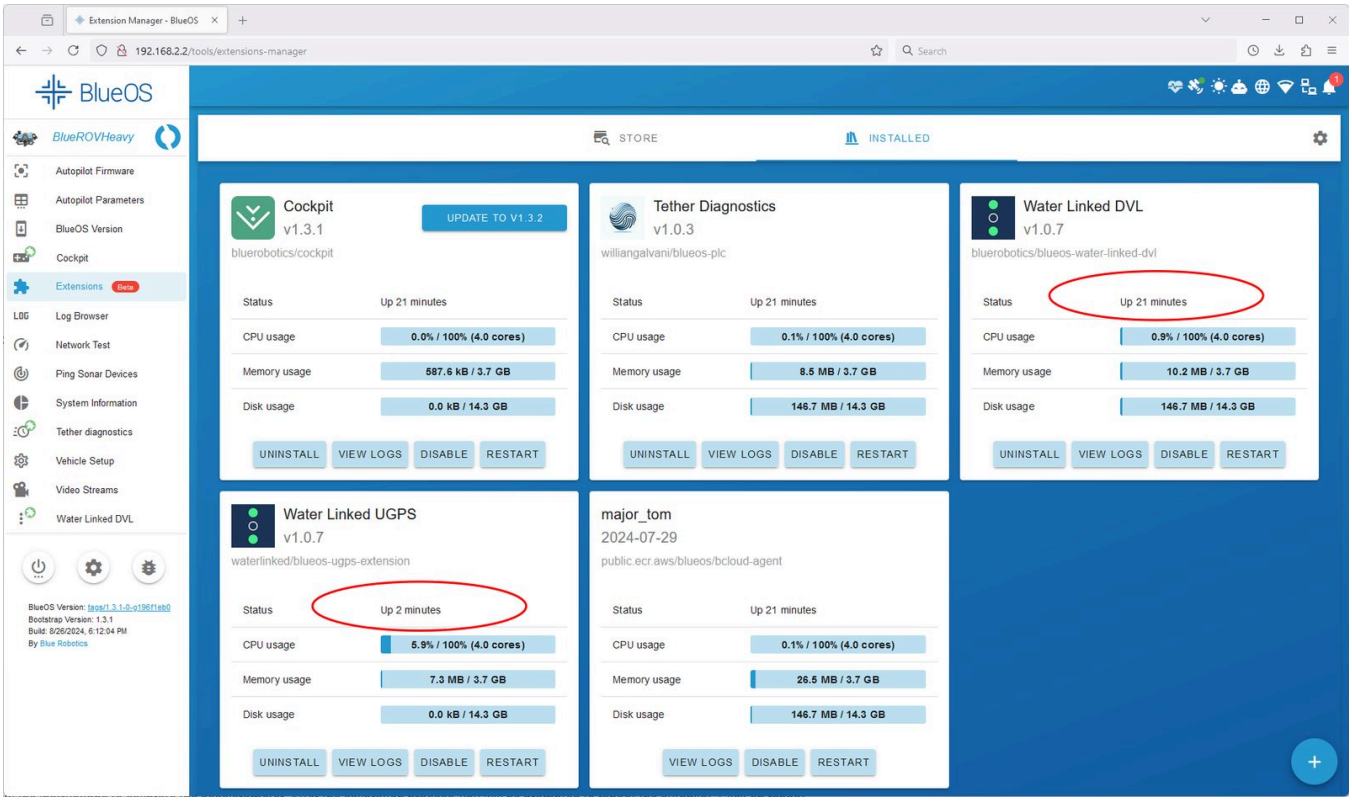
Alternative mode "DVL only": How to base the ROV position estimate only on the DVL while the Underwater GPS is running

This mode can be useful when you experience issues with sensor fusion (Mode "DVL+UGPS" above), or when not all the described conditions in the section above are met. Otherwise, we recommend using Mode "DVL+UGPS".

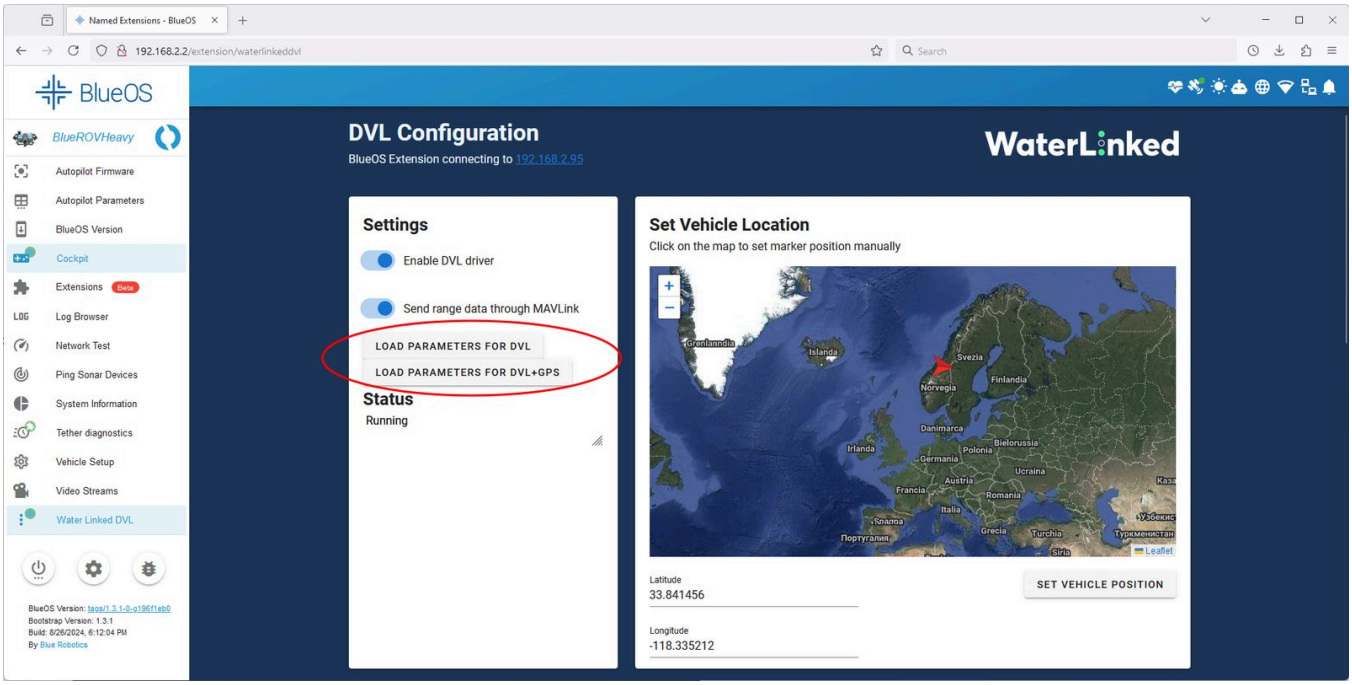
In this configuration, the BlueROV will base its position estimate *updates* on velocity input from the DVL as well as accelerometer, gyroscope, compass, and pressure sensor data, but not the Underwater GPS. Although the Underwater GPS is connected and running, it will only be used to *initialize* the global ROV position at the beginning. QGroundControl (or the new Cockpit) will show the ROV position estimate based on the DVL only, while the Underwater GPS GUI will show the position based on the Underwater GPS.

> **Note**
>
> The setup procedure below is very similar to when you only use the DVL without a running Underwater GPS. That case is described here.

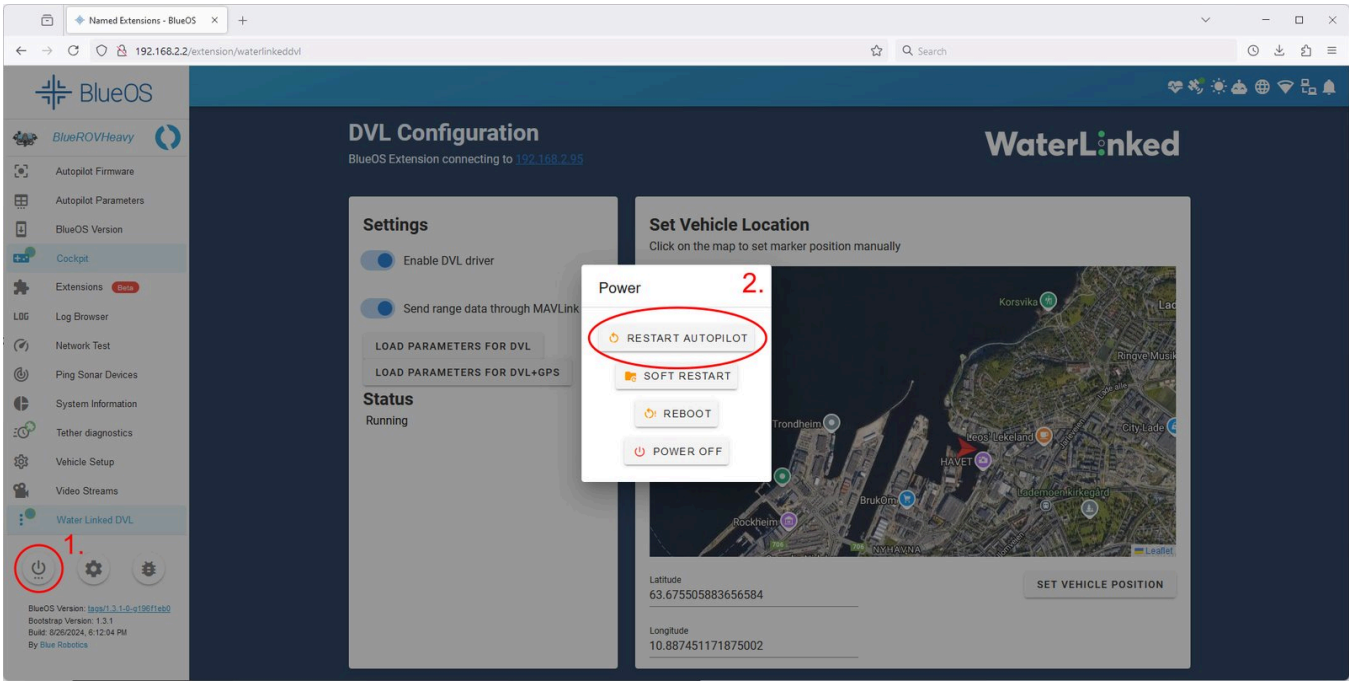
- Enable both the "Water Linked DVL" extension and "Water Linked UGPS" extension in the extensions manager. You see that they are running when an uptime is shown. Make sure you use the latest versions of both extensions.



- Open the interface of the DVL extension and click the button "Load parameters for DVL". This will set parameters for the Kalman filter to ignore the Underwater GPS input. Those parameters are persistent over reboots, which means that this button only needs to be pressed once and not every time you restart the ROV.



- Restart the autopilot for the parameter changes to take effect. It is not necessary to reboot BlueOS.



- If both Underwater GPS and DVL are connected, it is not necessary to set the vehicle position manually in the DVL extension.

FAQ for sensor fusion of Underwater GPS and DVL

How do I start an autonomous mission?

Blue Robotics wrote a guide on how to start an autonomous mission with the DVL only. The steps are the same when you use DVL and Underwater GPS together after you have followed this guide for Mode "DVL+UGPS".

At the time of writing (November 2024) QGroundControl is still more suited for advanced usage like missions compared to the new Cockpit.

Which mode am I currently using? Is Underwater GPS input enabled for sensor fusion or not?

You can check if the correct parameters are set by navigating to "Autopilot Parameters" in BlueOS. It shows a searchable list of all currently set parameters.

The table below lists both default values of the parameters for two Ardusub versions and which values the DVL extension sets when you click on the respective buttons. The parameter set by the Underwater GPS extension is listed, too.

Parameters can be "not set" which is shown as an empty cell in the table.

Parameter	Ardusub 4.5.0 default	Ardusub 4.1.2 default	DVL Extension, Mode "DVL+UGPS"	DVL Extension, Mode "DVL only"	UGPS extension
AHRS_EKF_TYPE	3 ekf3	3 ekf3	3 ekf3	3 ekf3	
EK2_ENABLE		0 disabled	0 disabled	0 disabled	
EK3_ENABLE	1 enabled	1 enabled	1 enabled	1 enabled	
VISO_TYPE	1 mavlink	0 none	1 mavlink	1 mavlink	
RNGFND1_TYPE	10 mavlink	10 mavlink	10 mavlink	10 mavlink	
EK3_GPS_TYPE			0 v=3d-pos=2d	3 no-gps	
EK3_SRC1_POSXY	3 gps	3 gps	3 gps	6 external-nav	
EK3_SRC1_VELXY	3 gps	3 gps	6 external-nav	6 external-nav	
EK3_SRC1_POSZ	1 baro	1 baro	1 baro	1 baro	
GPS_TYPE	1 auto	1 auto			14 mavlink

Documentation for Ardusub parameters can be found here.

Some parameters in Ardusub require a reboot of the autopilot to take effect while others take effect immediately. If in doubt, reboot the autopilot.

The ROV moves back and forth in position hold mode. What is wrong?

When the ROV holds its position well in mode "DVL only" and starts moving back and forth in mode "DVL+UGPS" this is due to more than expected noise of the UGPS position. Make sure to use static position and heading of the Underwater GPS Topside for best results.

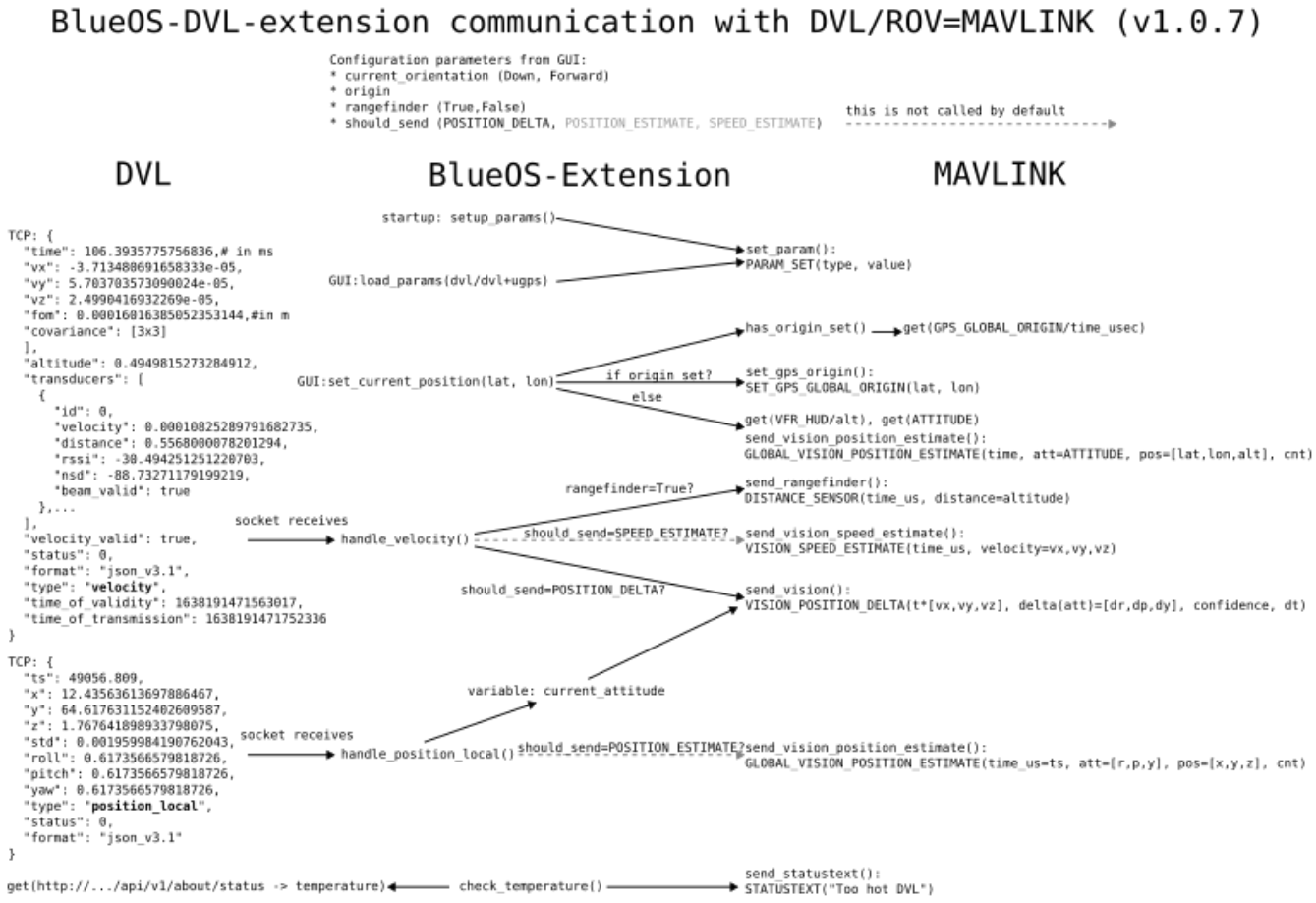
How does the sensor fusion work?

This section is for advanced users and developers who try to understand what happens behind the scenes. It contains links to documentation of the relevant open source projects and describes how Water Linked products interact.

- The DVL and UGPS extensions are sending messages following the mavlink-protocol to the autopilot, e.g. the message GPS_INPUT.
- The ROV-specific autopilot is called Ardusub, which is part of a larger project Ardupilot
- The extended Kalman filter is described in the Ardupilot documentation. The EKF3 implementation is used by default.
- In the current version of Ardusub together with the navigator flight controller only one EKF-lane is used.

How the BlueOS Water Linked DVL extension interacts with the autopilot

- This figure illustrates which events trigger which function calls and which mavlink messages:

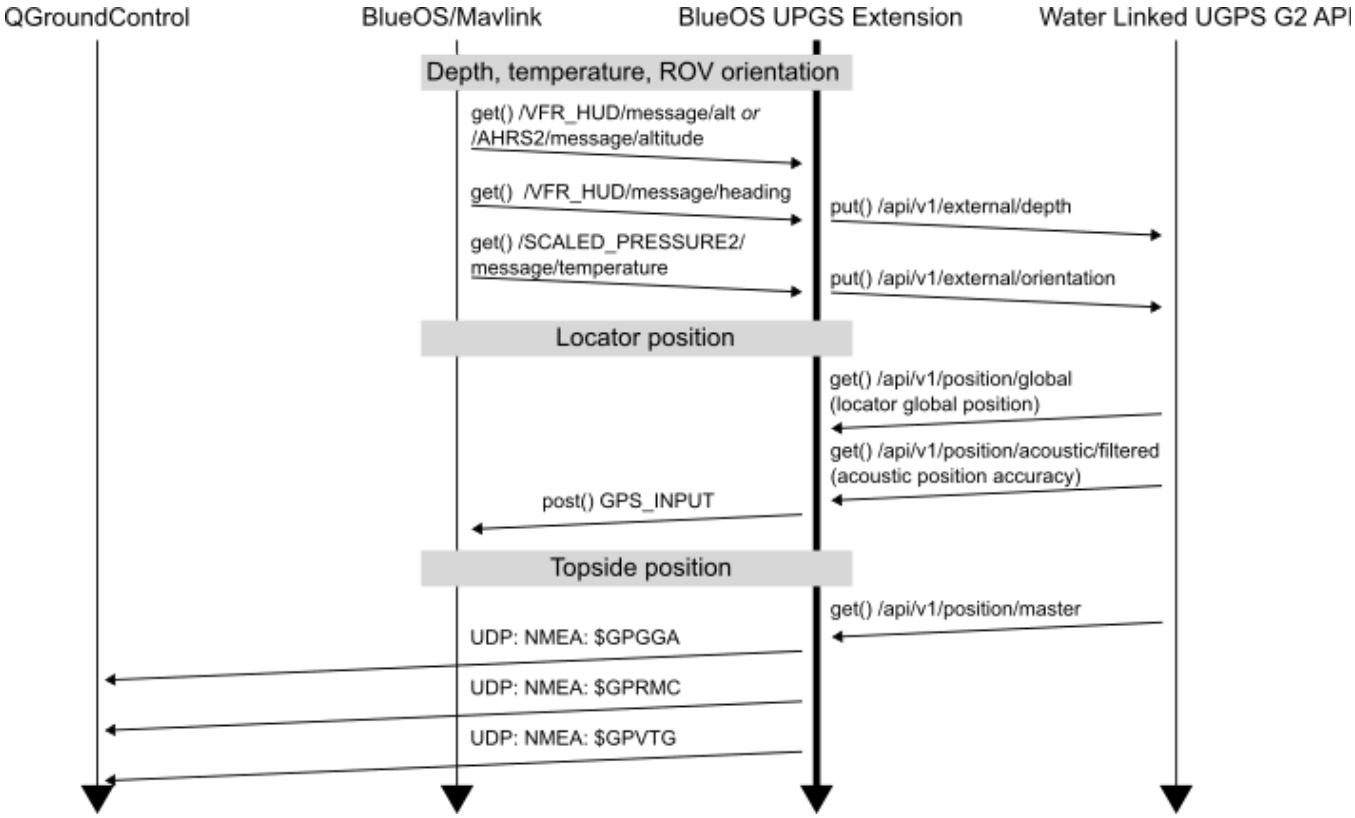


- When the user sets the vehicle location on the extension map the first time, the origin of the autopilot global position estimate is set (SET_GPS_GLOBAL_ORIGIN). When the latter is already set, the position estimate is updated with GLOBAL_VISION_POSITION_ESTIMATE.
- The main input from the DVL to the autopilot is the mavlink message VISION_POSITION_DELTA. It contains the change of position and attitude since the last message in body frame of the ROV together with the time interval since the last measurement. This corresponds directly to velocities and angular rates. The figure of merit (FOM) provided by the DVL is scaled to a confidence value in percent.
- The DVL-internal dead-reckoning exposed through the position_local-TCP-packet is currently not used on a BlueROV. A function in the extension is implemented, but it is disabled. So GLOBAL_VISION_POSITION_ESTIMATE messages are sent based on DVL-measurements.

- The attitude measured by the DVL-IMU (current_attitude) is sent in the VISION_POSITION_DELTA-message. The attitude values sent in VISION_POSITION_DELTA are currently not considered by the Ardusub autopilot.
- More details can be found in the source code in the GitHub repository.

How the BlueOS Water Linked UGPS extension interacts with the autopilot

This is an overview figure of the sent mavlink messages:



- The main input from the Underwater GPS to the autopilot is the mavlink message GPS_INPUT.
- The GitHub README provides more details.

Diagnostic log

A diagnostic log contains DVL logs and measurements that help Water Linked support investigate issues such as loss of bottom lock, invalid velocity, weak signal, communication problems, or unexpected behavior.

When to collect a diagnostic log

Collect a diagnostic log when:

- Water Linked support requests it.
- The DVL loses bottom lock unexpectedly.
- Velocity becomes invalid or drops out.
- Measurements are weak or noisy.
- You experience communication or web GUI issues.

How to collect a diagnostic log

1. Put the DVL in the same environment where the issue occurs.
2. Open the DVL web GUI.
3. Open the diagnostic log or diagnostics page for your DVL model.
4. Follow the instructions in the web GUI to create and download the diagnostic log.
5. Send the diagnostic log to Water Linked support together with relevant installation and test information.

Demo GUI links

- DVL A50/A125 diagnostic log demo
- DVL A100/A250 diagnostics demo

For more context on the web GUI pages, see DVL A50/A125 web GUI and DVL A100/A250 web GUI.

What to include when contacting support

Use the support package checklist when preparing a support request.

See How to diagnose for troubleshooting guidance before contacting support.

Transducer numbering and protocol IDs

Mechanical/transducer diagrams number the transducers from 1 to 4. In protocol messages and diagnostic logs, the transducer `id` uses zero-based numbering from 0 to 3. The `id` is therefore the transducer number minus 1.

How to diagnose

This page applies to all our DVLs unless otherwise noted.

Use this guide when the DVL is not performing as expected, cannot get bottom lock, reports invalid velocity, or produces noisy measurements.

Jump to a symptom:

- Invalid velocity
- Loss of bottom lock
- Weak signal or noisy measurements
- Air bubbles
- Thrusters, propellers, and hull turbulence
- Mechanical vibration and flexible mounts
- Operation close to the water surface
- Obstructed beam paths / line of sight
- Active acoustic interference from other sensors
- Thermal shutdown

Before contacting support

1. Confirm that the DVL is powered and submerged.
2. Check the LED status:
   - LED signals
3. Check wiring and cable condition:
   - DVL electrical connection
   - DVL A50/A125 serial interface
   - DVL A100/A250 serial interface
4. Open the web GUI:
   - DVL A50/A125 web GUI
   - DVL A100/A250 web GUI
5. Confirm that the software is up to date:
   - Software updates
6. Review Installation for mounting, bubbles, vibration, and acoustic interference.
7. Use the Support package checklist before contacting Water Linked support.

Download diagnostic log

Download a diagnostic log when bottom lock is unreliable, velocity is invalid, the web GUI reports warnings, or Water Linked support asks for one.

Use the common Diagnostic log page for collection instructions and model-specific demo links.

See the Support package checklist before contacting Water Linked support.

Invalid velocity

Invalid velocity means the DVL does not currently have a valid velocity estimate. Check:

- The DVL is submerged.
- Acoustics are enabled.
- The DVL has a clear line of sight to the seabed.
- The selected range mode includes the current altitude.
- The vehicle or boat speed is reasonable for the operating conditions.
- Bubbles, vibration, and acoustic interference are not affecting the measurement.

If the DVL is in water tracking mode, remember that velocity is relative to the water column, not the bottom. See Water tracking.

Loss of bottom lock

Bottom lock depends on altitude, bottom conditions, beam paths, acoustic noise, and vehicle motion.

If bottom lock is lost:

1. Check that the bottom is within the operating range.
2. Check that range mode is not too restrictive. See Range mode.
3. Check that all beam paths are unobstructed.
4. Check for bubbles and turbulence.
5. Check for other active acoustic instruments.
6. Review periodic cycling, especially in reflective tanks, pools, or open-water tests where every measurement matters.
7. Test in a simpler environment if possible.

Weak signal or noisy measurements

Weak or noisy measurements can be caused by soft bottoms, acoustic reflections, bubbles, vibration, and other acoustic sources.

Practical checks:

- Compare signal quality with thrusters or motors off and on.

- Test away from tank walls, dock structures, or polished metal surfaces.
- Check whether transducer distances are stable and similar in a flat test area.
- Collect a diagnostic log while the issue is present.

Air bubbles

Air bubbles can block or scatter the acoustic signal.

Check for bubbles from:

- Thrusters and propellers.
- Hull turbulence.
- Pumps and hoses.
- Waves or surface aeration.
- The DVL moving in and out of the water.

If bubbles are present, try a calmer test condition or a mounting location with cleaner water flow.

Thrusters, propellers, and hull turbulence

Thrusters, propellers, and hull edges can create turbulent or aerated water. This can reduce signal quality and cause invalid velocity.

When testing:

1. Check DVL output with thrusters or propellers stopped.
2. Check again at the expected operating speed.
3. If performance changes significantly, consider moving the DVL away from the turbulent flow.

Do not assume a fixed safe distance; the required separation depends on the vehicle, speed, and water flow.

Mechanical vibration and flexible mounts

The DVL should be mounted rigidly to the vehicle or boat. Flexible poles, loose brackets, or vibrating mounts can make measurements noisy.

Check:

- Fasteners are tight.
- The DVL cannot move relative to the vehicle.
- Poles or brackets do not visibly vibrate.
- Motors, pumps, or thrusters are not exciting the mount.

Operation close to the water surface

Near the surface, waves and bubbles can affect acoustic measurements. The DVL should stay fully submerged during operation.

Check:

- Waves are not exposing the transducers to air.
- The DVL is not passing through surface bubbles.
- Boat pitch, roll, or heave is not making the beam paths unstable.

Boat-mounted installations

For boat-mounted DVLs:

- Use a rigid mount.
- Keep the DVL submerged at the expected operating speed.
- Avoid aerated flow from the hull or propeller.
- Check for vibration in the pole or bracket.
- Verify bottom lock and velocity at the intended speed before relying on the data.

See Installation.

Obstructed beam paths / line of sight

The DVL needs a clear acoustic path from each transducer to the bottom.

Check model-specific line-of-sight information:

- DVL A50 line of sight
- DVL A100 line of sight
- DVL A125 line of sight
- DVL A250 line of sight

TODO: Add final DVL A100 line-of-sight asset when available. TODO: Add final DVL A250 line-of-sight asset when available.

If one beam shows a different distance or flickers more than the others, check whether that beam is seeing a wall, frame, skid, cable, or other obstruction.

Active acoustic interference from other sensors

Other acoustic instruments can interfere with DVL measurements, especially if they transmit at the same time.

Check:

1. Run the DVL with other acoustic instruments off.

2. Run the DVL with the other instruments on.
3. If performance changes, consider trigger or synchronization. See Integration.

Thermal shutdown

The DVL has a thermal shutdown mechanism to avoid damage. It issues a warning before shutting down at 55℃. Once it cools below 50℃, it automatically restarts. If the overheating issue is not addressed, it may repeat this shutdown/restart cycle.

If thermal shutdown occurs, confirm that the DVL is submerged and that acoustics are disabled when the DVL must be powered out of water for a short time.

What to send to Water Linked support

Submit a support ticket with the information listed in the Support package checklist.

Support package checklist

Use this checklist before contacting Water Linked support about bottom lock, invalid velocity, noisy measurements, connection issues, or integration problems.

The diagnostic log is most useful when it is recorded in the same setup and environment where the issue happens.

Before contacting support

1. Check the relevant symptom in How to diagnose.
2. Open the web GUI and look for changes on the diagnostics or status page while the issue is happening.
3. If possible, compare with another DVL of the same type, another vehicle, or a simpler test setup.
4. If the issue only happens in one vehicle or environment, collect one diagnostic log when the DVL works as expected and one when the issue is visible.
5. Review Installation for mounting, bubbles, vibration, line of sight, and acoustic interference.

> **Note**
>
> Many DVL issues are caused by the installation, vehicle, or acoustic environment. A support package helps Water Linked separate DVL behavior from vehicle and environment effects.

Download diagnostic log

Use the common Diagnostic log page for collection instructions and model-specific demo links.

Keep the generated file and include it with the support request.

Include in the support request

Include as much of this information as possible:

- DVL model.
- Serial number, if available.
- Software/firmware version.
- Diagnostic log from the web GUI.
- Description of the problem and when it happens.
- Description of how the DVL is mounted.
- Photos of the DVL and the installation.
- Operating altitude or range to seabed.
- Vehicle or boat speed.

- Range mode and tracking mode.
- Environment description, such as pool, tank, harbor, river, lake, or open sea.
- Whether thrusters, propellers, pumps, motors, echo sounders, sonars, modems, or other acoustic instruments were active.
- Screenshots from the web GUI if they show status, warnings, invalid velocity, or bottom-lock behavior.

If the issue may need service

Start by collecting diagnostics and sending the support package to Water Linked through the support page. Water Linked support may ask for additional checks before deciding the next step.

Do not send the DVL for service until Water Linked support has reviewed the issue and asked for the next action.

FAQ

This FAQ applies to DVL A50, DVL A100, DVL A125, and DVL A250 unless a question states otherwise.

1. How do I access the web GUI?

Applies to: all DVL models

Connect power and Ethernet, then open the DVL address in a web browser. Ethernet is recommended for first setup because it gives access to the web GUI, diagnostics, configuration, software updates, and the TCP data stream.

See Network setup for mDNS, fallback IP, direct Ethernet connection, and TCP data stream information.

2. What should I check if I cannot connect to the web GUI?

Applies to: all DVL models

Check:

- Power and LED status.
- Ethernet cable and connector condition.
- Computer network configuration.
- Whether mDNS is available on the network.
- The fallback IP address if mDNS does not resolve.
- Whether the DVL is in thermal shutdown.

See Network setup, DVL electrical connection, and How to diagnose.

3. Why does the DVL lose connection?

Applies to: all DVL models

Common causes include insufficient power, poor cable or connector contact, thermal shutdown, and network configuration issues.

Check Network setup, DVL electrical connection, and How to diagnose.

4. Why does the DVL lose bottom lock?

Applies to: all DVL models

Bottom lock depends on altitude, bottom conditions, line of sight, acoustic noise, vehicle motion, and range mode. If bottom lock is lost, check that the bottom is within the selected range, all beams have a clear path, and the DVL is not operating in bubbles or disturbed water.

See Loss of bottom lock and Installation.

---

5. Why does the DVL velocity become invalid or drop to zero?

Applies to: all DVL models

Velocity can become invalid when the DVL cannot form a valid velocity estimate. Common causes include loss of bottom lock, wrong range mode for the current altitude, blocked beam paths, bubbles, vibration, acoustic interference, or operation outside the conditions where the DVL can track reliably.

See Invalid velocity.

---

6. How do I download a diagnostic log?

Applies to: all DVL models

Open the web GUI diagnostics or diagnostic log page, put the DVL in the same environment where the issue occurs, enter a short issue description, create the log, and download it.

See Diagnostic log and Download diagnostic log.

---

7. What information should I send to Water Linked support?

Applies to: all DVL models

Use the Support package checklist. The most useful items are the diagnostic log, DVL model, serial number if available, software version, failure description, mounting description, installation photos, operating altitude, speed, range mode, tracking mode, environment, active acoustic instruments, and relevant web GUI screenshots.

---

8. How do I choose range mode?

Applies to: all DVL models

Use automatic range mode unless you know that the DVL should only search within a limited altitude range. If bottom lock is unreliable, make sure the selected range mode includes the actual altitude.

See Range mode.

TODO: Confirm final DVL A100/A250 range-mode tables before adding model-specific range-mode examples.

---

9. What is periodic cycling?

Applies to: all DVL models

Periodic cycling is a setting that is enabled by default. Every 10 seconds, the DVL checks that the bottom it has locked onto is the actual bottom and that there are no closer reflections that should be treated as the real bottom.

It is useful in reflective environments with many multipath reflections, such as tanks or pools. Some measurements are lost during the validation check. If you operate in open water with stable bottom conditions and need as many measurements as possible, consider turning periodic cycling off.

See Periodic cycling.

---

10. What is the difference between bottom tracking and water tracking?

Applies to: all DVL models

Bottom tracking estimates velocity relative to the seabed or another reflecting surface. Water tracking estimates velocity relative to the water column.

Water tracking can be useful when bottom lock is not available or when velocity relative to the water column is required. It should not be treated as bottom-relative velocity.

Water tracking is available on all DVL models, and behaves the same way on all of them.

See Water tracking.

---

11. Can air bubbles affect DVL performance?

Applies to: all DVL models

Yes. Air bubbles can scatter or block the acoustic signal and cause weak signal, noisy measurements, invalid velocity, or loss of bottom lock.

Check for bubbles from thrusters, propellers, pumps, hull turbulence, waves, or the DVL moving in and out of the water. See Air bubbles and Installation.

---

12. Can thrusters or propellers affect DVL performance?

Applies to: all DVL models

Yes. Thrusters, propellers, hull edges, pumps, and other moving water sources can create turbulence and bubbles in the beam paths. Compare DVL performance with thrusters or propellers off and on, and consider moving the DVL away from disturbed flow if performance changes.

See Thrusters, propellers, and hull turbulence.

13. Can mechanical vibration affect DVL performance?

Applies to: all DVL models

Yes. Loose brackets, flexible poles, or vibrating mounts can make measurements noisy because the DVL moves relative to the vehicle or boat.

Use a rigid mount and check whether motors, pumps, or thrusters excite the bracket. See Mechanical vibration and flexible mounts and Rigid mounting and vibration.

14. Can the DVL be used on a boat?

Applies to: all DVL models

Yes, but the installation should keep the DVL rigidly mounted, fully submerged, and away from aerated or turbulent flow. Verify bottom lock and velocity at the intended boat speed before relying on the data.

See Boat-mounted installations and Boat-mounted use.

15. How close to the water surface can the DVL be mounted?

Applies to: all DVL models

The DVL should stay fully submerged during operation. Near the surface, waves, air ingestion, turbulence, and bubbles can affect the acoustic signal. Exact surface-clearance limits depend on the installation and operating conditions.

See Operation close to the water surface and Operation close to the surface.

16. Can other acoustic instruments interfere with the DVL?

Applies to: all DVL models

Yes. Other acoustic instruments can interfere if they transmit at overlapping times or create strong acoustic noise in the DVL operating environment.

Test the DVL with the other instruments off, then on. If performance changes, consider physical separation, beam geometry, or trigger/synchronization. See Active acoustic interference from other sensors and Triggering and synchronization.

17. Which interface should I use: Ethernet, serial, PD0, PD4, or PD6?

Applies to: all DVL models

Use Ethernet and the web GUI for first setup, diagnostics, configuration, and testing. Use the Water Linked TCP JSON API over Ethernet for custom software integrations. Use the Water Linked serial protocol for direct vehicle integration over the serial interface.

Use PD4 or PD6 only when the receiving system already supports that format. PD0 is planned for DVL A100 and DVL A250, but is not available yet.

See Integration overview, PD formats, DVL electrical connection, and the protocol pages for your model.

---

18. Does the DVL have an RS232 interface?

Applies to: model-specific

DVL A50 and DVL A125 do not use RS232. They use a 3.3 V UART electrical interface that is 5 V tolerant.

DVL A100 and DVL A250 use RS232.

The serial data protocol is separate from the electrical interface. See DVL electrical connection, DVL A50/A125 serial protocol, and DVL A100/A250 serial protocol.

---

19. What is the latency when using the API over Ethernet?

Applies to: all DVL models

In our measurements, TCP JSON message latency was typically around 4 ms average, with 2 ms standard deviation and an observed maximum of 13 ms.

This is not a hard real-time guarantee. Actual latency can depend on network setup, host system, load, and integration method.

See Integration, DVL A50/A125 TCP JSON API, and DVL A100/A250 TCP JSON API.

---

20. How does triggering and synchronization work?

Applies to: model-specific

Triggering lets an external system control DVL pinging. This can be useful when several acoustic instruments are installed on the same vehicle and should avoid transmitting at the same time.

Triggering and synchronization are handled through the integration interface, not as a web GUI workflow. See Integration overview and the protocol page for your model.

TODO: Confirm exact timing requirements and supported trigger configurations before publishing detailed trigger setup rules.

---

21. What is the operating range?

Applies to: all DVL models

See Overview for model comparison and Range mode for range-mode configuration notes.

---

22. What is the velocity output frequency?

Applies to: all DVL models

Velocity output frequency varies with altitude and range mode. See Range mode for the common range-mode concept and model applicability notes.

TODO: Confirm A100 and A250 range mode values before publishing model-specific output frequency limits.

---

23. What is the source level or SPL?

Applies to: all DVL models

See the product comparison table in Overview. The FAQ does not repeat these values so that specifications are maintained in one place.

TODO: Confirm A100 and A250 source level/SPL before publishing it in the overview comparison table.

---

24. Can I test or operate the DVL in air?

Applies to: all DVL models

You will not get valid velocity measurements in air. It is possible to power the DVL out of water for a short time, but it warms up quickly. If possible, keep the DVL submerged when powered.

A practical lab setup is to keep the DVL submerged in a small bucket of water.

If the DVL must be powered out of water for a short time, disable acoustics in the web GUI to reduce heating.

---

25. Can the DVL overheat?

Applies to: all DVL models

Yes. The DVL has thermal shutdown to avoid damage. It issues a warning before shutting down at 55°C. Once it cools below 50°C, it automatically restarts. If the overheating issue is not addressed, it may repeat this shutdown/restart cycle.

Keep the DVL in water when powered for normal operation.

---

26. Do I need to calibrate the IMU each time I power on the DVL?

Applies to: all DVL models

No. The calibration is saved in memory.

27. Will temperature affect the IMU?

Applies to: all DVL models

Yes. The IMU can perform differently under temperatures that differ significantly from the conditions where it was calibrated.

28. Why does the yaw angle change rapidly?

Applies to: all DVL models

Yaw angle in the web GUI may increase or otherwise change rapidly if the gyro has not been calibrated or if the temperature changes.

See:

- Dead reckoning

> **Note**
>
> If only velocity output is required, you can ignore yaw drift. Yaw is only relevant if dead reckoning is used.

29. Can the DVL be used as an ADCP?

Applies to: all DVL models

No. The DVL algorithms are designed to track reflections from the seafloor or another reflecting surface. An ADCP measures reflections from water-borne particles at various depths, which uses a different processing approach.

30. Will the DVL work over a soft seabed, such as sand?

Applies to: all DVL models

Yes, the DVL can operate over soft bottoms. Softer seabeds can absorb more acoustic energy, which may reduce the maximum altitude where the DVL can reliably track. The effect varies by environment and may require testing.

31. How do I update software or firmware?

Applies to: all DVL models

Use the common Software updates page for all DVL models.

---

32. Which DVL model should I choose?

Applies to: all DVL models

Choose the DVL model based on required altitude range, platform size, depth rating, velocity range, physical interface, and integration requirements.

See Overview for the product comparison table and the model pages for mechanical details.

---

33. How should I maintain the DVL?

Applies to: all DVL models

After operating in seawater, brackish water, chlorinated pools, or any medium other than freshwater, rinse the DVL thoroughly with fresh water to remove residues and salt deposits. Once dry, wipe the exterior with a clean, dry cloth. Avoid solvents or abrasive cleaners.

DVL A50/A125 changelog

This page applies to DVL A50 and DVL A125.

See Software updates for more information on software updates.

2.7.2 (2026-06)

- Added an integration API for NTP configuration and status to provide tighter time synchronisation
  - Users with a NTP configuration from prior versions must apply this configuration again
- Improve NTP status feedback in GUI
- Add get_version_info TCP JSON command
- Fix issue where velocity updates for water tracking mode are not included in 'WL - Serial V2'
- Move changelog to docs.waterlinked.com

2.7.1 (2026-04)

- Add water tracking support for DVL A125
- Fix DVL can get stuck in an incorrect range mode if periodic cycling is disabled

2.7.0 (2026-03)

- Beta release only
- Add water tracking support for DVL A50
- Update serial output command (wcs) to include water tracking velocity
- Update serial protocol to version 2.7.0
- Update JSON protocol version to v3.2
- Production optimization

2.6.4 (2025-06)

- Improved signal to noise performance

2.6.3 (2025-04)

- Adjust DHCP timing to request IP address more frequently

2.6.2 (2024-12)

- Add serial baud rate configuration
- Fix FOM calculation error (introduced in 2.4.0)
- Fix NTP configuration to be persistent following reboot
- Fix PD6 to output configured speed of sound instead of 1475 m/s
- Fix PD6 string to output time of validity instead of time of transmission
- Fix an issue which could cause the user interface to stop responding
- Fix chip id not correctly sent over serial
- Update serial output command (wcg) to include configuration for PD4
- Update serial protocol to version 2.6.0
- Update serial protocols 'Latest' to 'WL - Serial V2' and 'Backward compatible' to 'WL - Serial V1+V2' for clarity
- [Beta] Add PD4 protocol support

2.6.1 (2024-11)

- Beta release only
- Add serial baud rate configuration
- Fix FOM calculation error (introduced in 2.4.0)
- Fix NTP configuration to be persistent following reboot
- Fix PD6 to output configured speed of sound instead of 1475 m/s
- Fix PD6 string to output time of validity instead of time of transmission
- Fix an issue which could cause the user interface to stop responding
- Update serial output command (wcg) to include configuration for PD4
- Update serial protocol to version 2.6.0
- Update serial protocols 'Latest' to 'WL - Serial V2' and 'Backward compatible' to 'WL - Serial V1+V2' for clarity
- [Beta] Add PD4 protocol support

Known issues:

- Chip id not correctly sent over serial

2.6.0 (2024-08)

- Beta release only
- Add serial baud rate configuration
- Fix NTP configuration to be persistent following reboot
- Fix PD6 to output configured speed of sound instead of 1475 m/s
- Fix PD6 string to output time of validity instead of time of transmission
- Update serial output command (wcg) to include configuration for PD4

- Update serial protocol to version 2.6.0
- Update serial protocols 'Latest' to 'WL - Serial V2' and 'Backward compatible' to 'WL - Serial V1+V2' for clarity

2.5.2 (2024-05)

- Improve velocity calculation
- Fix issue that causes Ethernet connectivity problems with certain switches
- Fix network configuration incorrectly showing as "static" after factory reset
- [Beta] Add support for PD4 protocol endpoint

Known issues:

- NTP configuration is not persistent

2.5.0 (2024-04)

- Beta release only
- [Beta] Add support for PD4 protocol endpoint
- Fix issue that causes Ethernet connectivity problems with certain switches

Known issues:

- NTP configuration is not persistent

2.4.5 (2024-01)

- Reduce boot time when ethernet is not connected

Known issues:

- NTP configuration is not persistent

2.4.4 (2023-12)

- Fix bug where altitudes <0.2m can be unstable (bug introduced in 2.4.0)
- Add PD6 BS sentence (Bottom track, ship referenced velocity)
- Add storing gyro calibration automatically when using the API
- Change gyro calibration to be single button for calibrate and store in the GUI
- Change warning messages to include a popup in the GUI
- Improve temperature shutdown warning in GUI
- Fix minor upgrade issue affecting a minority of users

Known issues:

- NTP configuration is not persistent

2.4.0 (2023-06)

- Add IMU compensation for fast velocity changes
- Improve accuracy of time of validity for altitudes >10m
- Improve dead reckoning position estimation when exposed to high acceleration
- Improve accuracy of altitude
- Improve accuracy of velocity valid
- Add ability to trigger pings though API
- Add configuration for periodic cycling
- Fix dead reckoning map clears when dead reckoning is reset using the API
- Fix NTP configuration persistent over reboots
- Fix velocities containing NaN or Inf not outputted to API/GUI
- Update serial protocol version to 2.5.0
- Update JSON protocol version to v3.1

Known issues:

- NTP configuration is not persistent

2.3.2 (2023-02)

- Beta release only
- Add ability to trigger pings though API
- Add configuration for periodic cycling
- Fix dead reckoning map clears when dead reckoning is reset using the API
- Fix NTP configuration persistent over reboots
- Fix velocities containing NaN or Inf not outputted to API/GUI
- Update serial protocol version to 2.5.0
- Update JSON protocol version to v3.1

Known issues:

- NTP configuration is not persistent

2.2.1 (2022-02)

- Add PD6 protocol support
- Add system time configuration
- Update serial protocol to version 2.4.0
- Fix missing line ending in JSON protocol
- Bugfixes

Known issues:

- NTP configuration is not persistent

2.1.0 (2021-12)

- Add configuration over serial/TCP
- Add reset of dead reckoning over serial/TCP
- Add additional information in serial protocol (updated to 2.3.0)
- Add covariance to JSON protocol (updated to v3)
- Add timing information (time of validity, time_of_transmission)
- Improve GUI visuals
- Fix mounting rotation offset bug
- Change ethernet auto-negotiation to 10/100

2.0.8 (2021-08)

- Bugfixes

2.0.2 (2021-06)

- Add thermal shut down
- Improve velocity calculation
- Update serial protocol to version 2.2.0
- Add output of orientation angles
- Add position estimation (dead reckoning) (Beta)

1.6.0 (2021-03)

- Improve range

1.5.1 (2021-02)

- Improve velocity valid
- Improve noise immunity
- Add transducer details in serial protocol
- Add vertical speed visualisation
- Add ping rate in GUI
- Improve web gui update rate using websockets

1.4.2 (2020-12)

- Fix bug in setting of fallback IP
- Add hardware revision in API

1.4.1 (2020-11)

- Fix instability introduced by 1.4.0

1.4.0 (2020-11)

- Add factory reset
- Fix TCP stops working after some time

1.3.7 (2020-08)

- Add generation of diagnostic report

1.3.5 (2020-06)

- Improve reliability of velocity data
- GUI fixes

1.3.3 (2020-06)

- Improve power control

1.3.2 (2020-04)

- Higher speed supported on longer range
- Improved velocity valid
- Increased ping rate
- Add high temperature warning
- Update serial protocol to version 2.1.0

1.3.1 (2020-04)

- Initial public release

DVL A100/A250 changelog

This page applies to DVL A100 and DVL A250.

See Software updates for more information on software updates.

3.5.2 (2026-09-29)

- Disable serial output by default
- Add output protocol configuration
- Add option to configure range mode
- Improve diagnostic log dialog
- Improve charts and plotting
- Improve dashboard layout and eligibility on smaller screens
- Improve dead-reckoning positioning

3.4.0 (2026-07-02)

- Add support for A100
- Add periodic cycling configuration in the GUI (already configurable through the API)
- Accessibility improvement (WCAG2 Compliant)
- Improve dead-reckoning positioning

3.3.0 (2026-06-18)

- Add water tracking
- Add `get_version_info` command to the TCP JSON API
- Improve dead-reckoning positioning
- Time API v2

3.2.0 (2026-03)

- Initial public release